# 大模型训练与推理

## 课堂投影片内容整理与总结分析

On-Site Lecture Slides: Content Digest, Technical Synthesis & Analysis

| 项目 | 内容 |
| --- | --- |
| 资料来源 | `大模型训练与推理\IMG_20261016_*.jpg`（手机翻拍投影片，共 96 张） |
| 拍摄信息 | 2026-10-16 13:32:19 — 16:26:59（全程约 2 小时 55 分，单机位连续翻拍） |
| 原件规格 | 4064×3048 px（94 张）/ 3264×2448 px（2 张），横向 4:3 |
| 整理方式 | RapidOCR（PP-OCRv4 ONNX，中英混合）逐页识别 → 结构化摘要 → 人工校核与综合 |
| 识别规模 | 96 页 / 60,024 字 / 平均置信度 0.947 |
| 配套文件 | ①《大模型训练与推理_图片影印本.pdf》（96 页原图影印，含可检索文本层）② 本文件 |

说明：本文全部技术要点、数字与结论均来自幻灯片文字与图注的识别结果，未引入外部资料； 凡识别不清、跨行串读或无法确认之处，均在原位标注 `(?)` 或说明"识别内容不足"。 本文的评述性内容（第三章之后的判断、趋势与实践建议）为分析者观点，已与事实陈述分开表述。

目录

摘要

一、资料与方法

1.1 资料的基本事实

1.2 处理流程

1.3 录音转写与对齐（新增）

1.4 可靠性说明

二、全篇脉络：从"一个内核"到"一个系统"

2.1 时间轴与主题分布

2.2 五个部分之间的内在联系

三、技术主线剖析

3.1 主线一：把 MoE 的通信与计算压进一个内核（第 1–34 页）

3.2 主线二：大规模训练的"装得下 / 跑得快"四步法（第 35–52 页）

3.3 主线三：KV Cache 是推理侧的第一性瓶颈（第 53–67 页）

3.4 主线四：从"算得快"到"找得准"——KV Router 与 Flash Indexer（第 68–78 页）

3.5 主线五：智能体推理重写了性能度量（第 79–96 页）

3.6 贯穿全场的方法论

四、总结

4.1 十二个关键结论

4.2 三句话的电梯陈述

五、分析与评论

5.1 这份材料的技术价值

5.2 我的四条趋势判断

5.3 局限与待核验之处

5.4 对实践的启示（按角色）

5.5 如果继续深挖，值得补的八个问题

六、逐页要点整理

6.1 第一部分 · MegaMoE：单内核融合的 MoE 算子（第 1–34 页）

6.2 第二部分 · 大规模训练优化 playbook（第 35–52 页）

6.3 第三部分 · KV Cache 压缩：TurboQuant / KVTC / nvCOMP-ANS（第 53–67 页）

6.4 第四部分 · KV Router 与 Flash Indexer（第 68–78 页）

6.5 第五部分 · 智能体推理与 ThunderAgent（第 79–96 页）

附录 A：逐页索引表（96 张）

附录 B：OCR 处理与可靠性说明

B.1 处理方法

B.2 质量统计

B.3 二次（高分辨率）识别补充

B.4 引用时的注意事项

附录 C：术语与缩写对照

# 附录 D：产出文件与复现方式

# 摘要

这 96 张照片记录的是 **2026-10-16 NVIDIA 开发者日（Developer Day）中国站"大语言模型训练与推理"分会场**的技术分享——据现场录音确认，它是**同一分会场的 4 场分享、5 位讲者**（详见配套《大模型训练与推理_讲稿整理（录音对照）.pdf》），全程约三小时，拼出一条完整的技术链条：**从单个 GPU 内核内部的指令级优化，一路推到跨千卡集群的调度与路由，最后落到智能体（Agent）时代的服务指标重写。**

**第 1 场（第 1–34 页，13:31–14:11；孙方宇、张毅接力）** 是全场技术密度最高的部分：Megatron/DeepSeek 风格的 MoE 推理内核 **MegaMoE** 如何把 dispatch、FC1（SwiGLU）、FC2、combine 全部塞进"一个 persistent kernel"，并把通信与计算在时间轴上做细粒度交织——理论上界从"朴素顺序执行"的 **1.42×**（Comet 式两路重叠）提升到 **1.92×**。随后是 30 余项具体优化：256×256 大 tile、swapAB 动态 tileN、TMEM 内 epilogue 转置、单 dispatch warp、16 字节向量化写出等，最终在 4×H200 / EP4 / DeepSeek-V4-Pro 形状上拿到 **几何平均 1.16×（per-tensor FP8）到 1.30×（blockwise FP8）、峰值 1.35×–1.72×** 的端到端 MoE 层加速。

**第 2 场（第 35–52 页，14:12–15:00；白洪潇，NVIDIA Megatron 团队）** 视角抬高到集群：以 **DeepSeek-V4-Pro（1.6T 总参数 / 49B 激活）** 为例，把大规模训练拆成"装得下（make it fit）"与"跑得快（make it fast）"两类问题，并给出一套四步 playbook（运行开销与显存权衡 → 注意力计算效率 → 专家执行与显存占用 → 上下文并行负载均衡），实测吞吐 **64K 上下文 +59%、1M 上下文 +32%。**

**第 3 场（第 53–67 页，15:00–15:44 含问答；徐文杰）** 转到推理侧的**第一性瓶颈——KV Cache**：用在线量化（TurboQuant，≥4.5× 名义载荷压缩）、面向存储与传输的压缩（KVTC，8×–64× 压缩率、精度损失 ≤0.8 分）、以及无损熵编码（nvCOMP-ANS，TTFT 至多 +18.7% 收益）三条路径，把"显存容量墙"变成"编解码开销"问题。

**第 4 场前半（第 68–78 页，15:45–15:56；卢祥龙，NVIDIA 解决方案与工程）** 讲**缓存感知路由**：KV Router 用"前缀命中 + 实时负载"两类信号给 worker 打分，Flash Indexer 用"位置→worker"地图回答"哪里有缓存"，索引吞吐从 385K 提升到 4M、再到 170M block ops/s（累计 440×/42×），并把索引拆成独立服务。

**第 4 场后半（第 79–96 页，15:58–16:26）** 收在**智能体推理**：单会话 ISL 峰值 219K、请求 p99 近 900K、工具/用户间隔可达 60 s 以上，传统 TTFT/TPOT 已不足以描述体验，于是提出 trajectory 级 SLA 与四组目标（容量→命中率→并发→路由），并用会话网关 **ThunderAgent** 在 SWE-Bench Lite 上把端到端时延降低 **51%**、TTFT p50 降低 **87%**、吞吐提升 **1.6×**。

**一句话总结**：这份材料展示了一条清晰的"性能工程主线"——当模型结构（MoE + 混合注意力 + 超长上下文）已经把算力与显存压到极限时，剩下的提升空间来自**边界消除**（把通信塞进计算内核、把压缩塞进数据通路、把索引塞进路由、把会话调度塞进网关）与**度量重定义**（从单请求时延到会话轨迹级 SLA）。

# 一、资料与方法

## 1.1 资料的基本事实

| 维度 | 事实 |
| --- | --- |
| 图片总数 | 96 张（jpeg） |
| 时间跨度 | 13:32:19 — 16:26:59，约 175 分钟；拍摄节奏前后差异明显（前半约 40–90 s/张，后半约 10–20 s/张） |
| 拍摄方式 | 手持/连续翻拍投影画面，横向构图；个别页有屏幕反光、透视变形与相邻页内容入画 |
| 内容性质 | 同一场技术分享的投影片；中英混排（前三段几乎全英文，第四段为中文，第五段中英混排） |
| 主题数量 | 4 场分享（5 位讲者）：MegaMoE 内核 / 训练 playbook / KV Cache 压缩 / Dynamo 与智能体推理 |
| 配套录音 | 同现场 4 段录音（共约 4.9 小时）及讯飞转写结果，用于交叉验证讲者、场次与口头数据（见 1.4） |
| 重复拍摄 | 第 4–5 页、第 39–40 页、第 41–42 页为同一张投影片的重拍（后者更完整或更清晰） |

从拍摄时间与页首标题的对应关系可以看出，这 96 张照片基本按讲者的实际顺序记录，没有明显缺页：第 34 页（MegaMoE 的 Summary）之后紧接着第 35 页（"Two Challenges Define Training at Scale"），第 52 页 "Lessons for Scalable Training" 之后是第 53 页 "Motivation: KV Cache as a Serving Bottleneck"，段落切换处都有明显的过渡页。

## 1.2 处理流程

1. **图像规整**：读取 EXIF 方向、统一转为 RGB，按最长边 2600 px 生成识别副本（保留原图坐标，供文本层回贴）。
2. **文字识别**：RapidOCR（PP-OCRv4 检测 + 方向分类 + 识别模型，ONNX Runtime CPU 推理），中文 + 英文混合识别，按行聚类还原阅读顺序。
3. **质检与二次识别**：统计每页的文本量、平均置信度、低分框数量；对识别最弱的 8 页（18、26、31、81、84、86、87、92）用最长边 3800 px 重跑一遍，取更完整的一版。
4. **结构化整理**：按页生成"标题 + 要点 + 图表说明 + 不确定标记"，再按主题合并成章（见第六章逐页要点整理）。
5. **成品输出**：① 原图影印本 PDF（每页含隐形 OCR 文本层，可全文检索）；② 本总结分析 PDF。

## 1.3 录音转写与对齐（新增）

投影片只能看到"讲了什么"，看不到"谁讲的、为什么这么讲"。为此另做了录音侧的整理：

| 环节 | 做法 |
|---|---|
| 音频 | `录音机-20261016-1331.wav`（120:00）、`录音机-20261016-1531.wav`（64:00），48 kHz 双声道 24-bit |
| 转写 | 讯飞「录音文件转写大模型」（autodialect），得到 2827 句带时间戳的转写 |
| 对齐 | `音频偏移 = 照片拍摄时刻 − 录音起始时刻`；每页取 `[拍摄 −8 s, 下一张拍摄]` 的窗口 |
| 结果 | 96 页全部对齐成功；落入窗口的转写共 47,899 字；场次与讲者由交接语/自我介绍确认 |

由此确认：本场为 **NVIDIA 开发者日"大语言模型训练与推理"分会场**，共 **4 场分享、5 位讲者**（孙方宇、张毅 / 白洪潇 / 徐文杰 / 卢祥龙）。 逐页口头要点与现场问答见配套文件《大模型训练与推理_讲稿整理（录音对照）.pdf》；本章此后凡引用"口头"信息，均在该文件中有出处段落。

## 1.4 可靠性说明

- **识别质量总体良好**：96 页平均置信度 **0.947**，共识别 60,024 字；仅第 31 页平均置信度低于 0.80（0.76，经高分辨率二次识别后主要内容已恢复）。
- **典型错误形态**：① 英文词间空格丢失（如 `WhatisMegaMoE`、`Largetilesize`）；② 中英混排时行内串读（把两栏文字读成一行）；③ 图表内的细字与坐标轴刻度易混淆（如 1.59× 与 1.51× 的归属）；④ 中文投影片偶有别字（如"缓存"被识别为"媛存"）。
- **本文对数字的处理原则**：只引用幻灯片明写的数值；明显属于同一句、同一组的数值按上下文归类，并在无法确定归属时加 `(?)`。第 33 页 token sweep 柱状图中"哪一根柱子对应哪个倍数"存在归属不确定，因此只采用讲者文字结论（per-tensor geomean 1.16×、peaks 1.35×；blockwise geomean 1.30×、peak 1.72×）。
- **本文不做的推断**：不推断未写明的时间、未写明的硬件数量、未写明的模型细节；对"讲者/主办方/会议名称"只在有直接证据时提及（页脚 NVIDIA 标识、页面内出现的 `gtc26-*` 链接）。

# 二、全篇脉络：从"一个内核"到"一个系统"

## 2.1 时间轴与主题分布

| 时段 | 页码 | 场次 / 讲者 | 主题（主讲内容） | 层次 | 关键词 |
|---|---|---|---|---|---|
| 13:31–14:11 | 1–34 | 第 1 场 · 孙方宇（1–15）、张毅（16–34） | **MegaMoE：单内核融合的 MoE 推理/训练算子** | GPU 内核 / 指令级 | persistent kernel、dispatch/combine、swapAB、TMEM、WGMMA、FP8/FP4 |
| 14:12–15:00 | 35–52 | 第 2 场 · 白洪潇（NVIDIA Megatron） | **大规模训练优化 playbook**（DeepSeek-V4-Pro + Megatron Core + Blackwell） | 集群 / 并行策略 | PP4·EP64、混合注意力、FP8 Indexer、CP 负载均衡 |
| 15:00–15:44 | 53–67 | 第 3 场 · 徐文杰 | **KV Cache 压缩：TurboQuant / KVTC / nvCOMP-ANS** | 显存与数据通路 | 在线量化、PCA 压缩、无损熵编码、TTFT |
| 15:45–15:56 | 68–78 | 第 4 场前半 · 卢祥龙（NVIDIA 解决方案与工程） | **KV Router 与 Flash Indexer（Dynamo）** | 服务化 / 路由 | 前缀命中、位置→worker 地图、索引吞吐 385K→170M |
| 15:58–16:26 | 79–96 | 第 4 场后半（同上） | **智能体推理与 ThunderAgent** | 业务 SLA / 调度 | AgentX trace、ISL 219K、trajectory SLA、会话网关 |

（第 34 页"Summary"、第 52 页"Lessons"、第 67 页"Takeaways"分别是前三场的收束页；第 4 场内部则由第 79 页"SLAs of New Agentic Era"完成从"路由与索引"到"会话调度"的转折。录音确认：前三场的收束页之后都紧跟现场问答（第 52 页对应 7 分钟问答、第 67 页对应 13 分钟问答），问答要点见配套《讲稿整理》。）

## 2.2 五个部分之间的内在联系

把这五段串起来看，它其实回答的是同一个问题的五个层次——**"当模型规模和上下文长度都到了硬件极限附近，性能还能从哪里来？"**

1. **内核层（第 1–34 页）**：同一个 SM 内的通信与计算本是两件事，把二者放进一个 kernel，用波次（expert wave）和时间片交错，理论上可多拿约 35% 的时间轴（1.42× → 1.92×）。代价是 shared memory 被通信借用、TMA 主循环被干扰——于是又需要一整套寄存器/流水/同步的微调来把"理论加速"变成实测加速（几何平均 1.16×–1.30×）。
2. **集群层（第 35–52 页）**：单卡内核再好，装不下 1.6T 参数也不行。于是问题变成"并行布局 + 显存策略 + 逐模块替换低效实现"：先把 EP/TP/CP 这些通信密集的组关进 NVLink 域，再用 4K 稀疏代理模型验证特性，接着按 step 1→4 把运行时开销、混合注意力、专家执行、CP 不均衡逐个打掉。

3. **显存层（第 53–67 页）**：训练看算力，推理看显存。KV Cache 一旦成为瓶颈，解法不是"换更大的卡"，而是"用更少的字节存同样的信息"：在线量化（TurboQuant）、面向存储/传输的压缩（KVTC）、无损熵编码（nvCOMP-ANS），三者分别对着容量、带宽、以及"既不想掉精度又想要收益"的场景。
4. **路由层（第 68–78 页）**：压缩之后的 KV 依然散落在多个 worker 上，"谁手里有这段前缀的缓存"决定了要不要重算。KV Router 把"缓存命中"和"实时负载"合成评分，Flash Indexer 把索引成本压到微秒级——这是把"省下来的显存"真正兑换成"省下来的时间"的关键一环。
5. **业务层（第 79–96 页）**：到了智能体场景，单次请求变成几十上百步的会话轨迹，KV 会被反复驱逐、重算，于是评价指标从 TTFT/TPOT 升级为 trajectory 级 SLA；ThunderAgent 用一个 5 s 一跳的会话网关，在"容量—命中率—并发—路由"四个目标之间做取舍，最终换来端到端 −51%、TTFT −87%。

换句话说，全场技术叙事的重心在缓慢上移：**算子 → 并行 → 显存 → 路由 → 会话调度**，而每一层的核心动作都是同一个——**消除边界**：消除通信与计算的边界、消除"装不下"与"跑不快"的边界、消除存储与计算的边界、消除缓存与调度的边界、消除请求与服务会话的边界。

```
单卡内核          集群并行            显存/带宽          服务路由          会话语义
MegaMoE    →   训练 playbook  →   KV 压缩      →   KV Router    →   ThunderAgent
1.92× 理论上界    64K +59%           4.5×/8-64×/1.4×   440× 索引          e2e −51%
```

# 三、技术主线剖析

以下按五条主线展开，每条主线给出"问题—手段—数据—要点"四层，数据只引用幻灯片上的原始数值。

## 3.1 主线一：把 MoE 的通信与计算压进一个内核（第 1–34 页）

### 3.1.1 问题：MoE 层是"四段式"的，通信插在中间

一个 EP（专家并行）MoE 层天然分成四段：**dispatch（把 token 按专家路由 all-to-all 发出去）→ FC1 + SwiGLU 激活 → FC2 → combine（把结果收回来）**。四段串联执行时，通信期间计算单元空闲、计算期间网络空闲。第 1 页用三行时间轴把这件事画得很直白：

| 方案 | 排布方式 | 理论加速比 |
|---|---|---|
| (a) Naive Solution | `Dispatch → L1 → Act → L2 → Combine` 严格顺序 | 1.00×（基线） |
| (b) Comet | 通信（Dispatch/Combine）与计算（Act 等）拆成两条流水线并行推进 | **1.42×** |
| (c) Ours（MegaMoE） | `Dispatch All-to-All` 与计算重叠，计算侧按 `L1/L2` 交替重复，再逐段插入 `Act`、`Combine`；通信按 `Expert Wave 1/2/3` 分波，波间为 `Combine All-to-All` | **1.92×** |

关键差别在颗粒度：Comet 是"通信/计算两条大流水"，MegaMoE 是"按专家波次把通信切成多段、与 L1/L2 交替填满时间轴"。

### 3.1.2 代价：融合不是免费的（第 3 页）

讲者专门用一页讲"融合带来的挑战"，这是理解后续十一项优化的钥匙：

- **通信和计算跑在同一个 SM 上**，通信要**借用计算侧的 shared memory**；
- 通信流量会**干扰主循环的 TMA 效率**；
- `NVFP4` 把 L1/L2 的耗时减半，但 combine 数据"一直"是 bf16，于是 **NVLink 流量的相对代价被放大。**

三条叠加的效果就是：GEMM 效率与 NVLink 效率相互牵制（图中直接标为"compute 与 comm 之间的干扰"）。后面的优化几乎全部是在给这三条还债。

### 3.1.3 手段：三条优化清单（第 4–5 页大纲）

| 优化方向 | 具体条目 | 主要落地页 |
|---|---|---|
| **GEMM 性能** | 大 tile（256×256）、swapAB 动态 tileN、TMEM 内 epilogue 转置 | 6–8、19 |
| **降低通信/计算干扰** | MEMBAR 同步优化（`batch-commit`、`async UREDGR`）、只留一个活跃 dispatch warp | 13、27 |

| 优化方向 | 具体条目 | 主要落地页 |
|---|---|---|
| **combine 性能** | 三种流量模式、FC1/FC2 相位交织、自适应流量控制、低精度 combine 数据类型 | 14–16 |

**（1）GEMM 侧：256×256 与 "overlapping acc"** 把 tile 从 256×192 改成 **256×256**，理由是"更大的 tile 有更充裕的访存预算"；难点是寄存器/累加器放不下，于是用**累加器重叠（overlapping ACC）技巧**让 256×256 近似获得"两段式 acc"的效果（第 6 页）。

**（2）GEMM 侧：swapAB 动态 tileN（第 7、19 页）** **根因**：SM90 的 WGMMA 形状是 `m64nNk32`——M 每个 atom 固定为 64、N 可灵活（最小到 8）。于是非 swapAB 下 token 只能落在 M 维，token tile 最小就是 64：解码类负载常常只有 8 个 token，**64 行里有 56 行是 padding**（第 19 页原话：`56 of 64 token rows wasted`）。**做法**：把 token 维换到 N 维，`M = weight channels（128 / 256，恒密）`、`N = tokens（8 / 16 / 32 / 64 / 128）`，于是 `M128/256 × N8` 的 tile 几乎"全是有用的"，**tile 利用率最高提升 8×。代价与技巧**：swap 本身只是**纯 stride view、零拷贝**；expert 维变成 GEMM-A 的 `L` mode，**TMA descriptor 变成 tile-invariant**；转置被推到 epilogue（FC1 走 R2S + SMEM 寻址，FC2 干脆不落 SMEM、直接从寄存器发 BF16 的 `STG`）；而"8 行 gate/up 交错"恰好落在同一线程的寄存器对上，**folding 不需要任何 shuffle**；同时它与主 kernel 共用 MMA mainloop 与 warp topology，维护成本低。

**（3）GEMM 侧：TMEM 内 epilogue 转置（第 8 页）** 这一页的方法论比结论更有意思：起点只是一个 `16×8 → 32×4` 的 copy-pattern 小技巧（`2× LDTM.16dp128b + STTM.32dp32b`），讲者与**LLM 迭代讨论**后把"每条 LDTM、寄存器置换、STTM"统一建模为**作用在 thread 位 / register 位 / TMEM-DP 位 / column 位上的位级置换**，再用 **Cayley/BFS 搜索**找最优组合，最后用**模拟器回放验证**，得出可用的完整 TMEM 转置（32×32、16×32→32×16），并集成进 CuTe 内核。页面上那五个步骤 `Observe → Model with LLM → Search → Validate → Integrate` 值得单独记住。

**（4）精度策略：per-tensor 与 blockwise 是两种不同的工程取舍（第 20 页）**

| | per_tensor | blockwise（DeepGEMM 风格） |
|---|---|---|
| 权重缩放 | 每个专家一个标量 | 128×128 块 |
| 累加 | 1×acc / 2×acc，按 K tile 做 FP32 promote | promote 内建 |

这也解释了第 33 页为什么要把两种量化模式分开测：blockwise 的量化粒度更细、GEMM 更重，单内核融合的收益结构随之改变。

**（5）同步与调度：把"等待"变成"重叠"（第 27–32 页）** 这一段是整场最"硬核"的部分，全文都是微架构级的取舍：

| # | 优化（页） | 做法 | 效果 |
|---|---|---|---|
| 1 | Cooperative warp-group scheduling（21） | 协作式 warp-group 调度，单 task tile `M64×N256` | 多个启发式表 bucket 仍由协作模式取胜（如 blockwise ≥512 恒用 `(64,128)`） |
| 2 | Task-tile ping-pong（22） | 任务 tile 乒乓交替 | 保持流水不空转 |

| # | 优化（页） | 做法 | 效果 |
|---|---|---|---|
| 3 | Multi-CTA CGA + TMA multicast（23） | `2×2`（亦可 2×1/1×2）CGA 簇，按簇分组 task tile | TMA 多播降低重复搬运 |
| 4 | Mainloop & Store 流水（24） | WGMMA 软件流水（`k_pipe_mmas=1`）；FC2 R2S+TMA store 两段式，SMEM stage 4 KiB；228 KB SMEM 预算动态填满 | 延迟隐藏 |
| 5 | All-Register epilogue fusion chain（25） | R2S 单次过 SMEM → TMA 批量 store，并在链条里做 FP8 requant | 减少一次 SMEM 往返 |
| 6 | Gate/Up interleave = 8（26） | 依据 WGMMA 寄存器布局决定交错因子 | 让 FC1 的 gate/up 配对读取不额外搬数 |
| 7 | 单个活跃 dispatch warp（27） | 只保留 1 个 warp 做 dispatch（约 **132 行在飞**已超过 NVLink 带宽时延积） | 实测 **1 warp > 2 > 4**：更深的读队列会拉长每次往返并与 GEMM 争用 |
| 8 | 折叠 producer warp-group + 寄存器重排（28） | 让"双 WG 协作"变得可行 | blockwise 512–32768 全部切到 `M64N256`，**+11% ~ +30%，几何平均 +15.8%** |
| 9 | 提前发布 `fc1_done`（29） | 发布点上移，FC2 tile 更早通过就绪检查、更早发 TMA load | 缩短等待 |
| 10 | 调度器分组 + `fc1_done` 自旋（31） | 把小 bucket 下的调度组扩大到 **264（≈6 个专家）**，使 FC2 tile 到达时 `fc1_done` 已完成 | 消除 activation TMA producer **约 25%** 的自旋时间 |
| 11 | 16 字节 FC2 输出（32） | 4×4 butterfly transpose（4 次 shuffle）让每条 lane 持有某行连续的 8 列 | `8× STG.32`（swapAB 时 `8× STG.16`）→ **1× `STG.128`** |

**（6）架构：一个 kernel 做所有事（第 17–18 页）** `Overall Architecture: One Kernel Does Everything` 的四段数据通路环流是：**Dispatch（4 个 dispatch warp 通过 NVSHMEM symmetric heap 把各 rank 的 token 拉到本地 pool）→ FC1（pool 的 FP8 view 直接被当作 GEMM 的 activation，零拷贝、零中间 kernel）→ FC2（通过 GMEM counter 握手 `fc1_done_counter` 消费 FC1 输出）→ Combine（写回 source rank）**。Combine 的组合方式是 `3 种 executor × 2 种 reduce 落点 = 6 种模式`（这正好解释了第 34 页"six modes"的来历）。与常规基线相比，关键差别是**通信 warp 与 GEMM warp 同驻一个 SM 且真正并发**：token 一到就被计算。

第 18 页的 warp 分工表给出了这套"同驻"的具体编制（总量 **20 warps / 640 threads**，`setmaxnreg` 按角色重新分配，其中 epilogue 拉满到 **256**）：

| 角色 | Warps | setmaxnreg | 职责 |
|---|---|---|---|
| epi / WGMMA WG | 4×(1–2) | 200–256 | 发 WGMMA + SwiGLU / quant / store |
| TMA-A | (?) | — | 装 activation 与 `fc1_output`；FC2 的 `fc1_done` 自旋 |

| 角色 | Warps | setmaxnreg | 职责 |
|---|---|---|---|
| TMA-B | (?) | — | 装权重（与 FC1 解耦，不等 counter） |
| sched | (?) | — | 发布 work tile + 专家预取 |
| dispatch | (?) | — | 跨 rank 拉 token（MegaMoE 的核心动作） |
| token-back | 4（可选） | (?) | 把 FC2 结果推回 source rank |

（`Warps` 列的个别数字与 `setmaxnreg` 列在 OCR 中粘连，逐行取值不可靠，故标 `(?)`；总量 `20 warps / 640 threads` 为页面明写。）**最反直觉的一点**：这里**没有独立的 MMA warpgroup**——由 epilogue WG 自己发 WGMMA，accumulator 常驻 RMEM（`acc stage = 1`），省掉了一个 warp-group 的握手中转。

### 3.1.4 数据：token sweep（第 33 页）

| 项目 | 设定 |
|---|---|
| 硬件/并行 | 4× H200，EP4 |
| 模型形状 | DeepSeek-V4-Pro shape，完整 MoE 层 forward |
| 统计口径 | p50 over the slowest rank |
| token 档位 | 8 / 16 / 32 / 64 / 128 / 256 / 512 / 1024 / 2048 / 4096 / 8192 / 16384 / 32768（13 档） |
| 基线 | 常规四段式：NCCL all-gather / reduce-scatter + CUTLASS grouped GEMM |
| 对照 | MegaMoE：单内核（NVSHMEM dispatch/combine） |

**结果**：

- **Per-Tensor FP8**：13 个档位全部领先 —— 512 及以下 **1.02–1.11×**；1024 起 **1.24–1.35×**（峰值 **1.35×** @8192）；**几何平均 1.16×**。
- **Blockwise FP8**：512 以下差异很小；1024 起 **1.33–1.72×**（峰值 **1.72×** @32768）；**几何平均 1.30×**。
- **讲者的解释**：小 token 数时两者接近，因为此时 **launch 与通信开销主导**；只有当 GEMM 占比上升后，单内核"省掉 launch、省掉中间张量往返、通信与计算交织"的优势才显现。
- **口头补充（录音）**：讲者在总结时给出了档位级的说法——小 token 档位加速约 **3%**、大 token 档位约 **24%**、blockwise 在 **32768** 档可达 **72%**；并解释"小 token 时通信占比高、实际计算量小、launch 也不饱和，两种配置其实都是 **memory-bound**"，而增益来自"通信与计算融合、epilogue 与 SwiGLU 融合，减少了 HBM 读写"。

**要点**：这类优化的收益是**形状相关**的。它不是在所有场景都赢 1.9×，而是"理论 1.92× 的上界 + 实际 1.16–1.30× 的几何平均 + 大 token 时 1.35–1.72× 的峰值"。第 34 页的 Summary 也印证了这一点：`1 个 persistent kernel、6 种模式、2 个变体（non-swap / swapAB）、2 种量化模式、30+ 项优化`，外加 **token-bucket heuristics**——把 sweep 出来每个档位的最优解冻结成查表。

---

## 3.2 主线二：大规模训练的"装得下 / 跑得快"四步法（第 35–52 页）

### 3.2.1 两个挑战（第 35 页）

| Make it fit（装得下） | Make it fast（跑得快） |
|---|---|
| 超大参数量 | 内核与 launch 开销 |
| 模型状态与 HBM 容量 | 重算（recompute）与 offload 代价 |
| 长上下文窗口 | 通信与负载不均衡 |
| 并行的切分布局与拓扑 | （同上） |
| 特性兼容性 | |

取材来源写得很清楚：**公开的 DeepSeek-V4-Pro 架构 + Megatron Core 实现 + NVIDIA Blackwell GPU。**

### 3.2.2 模型为什么这样设计优化（第 37–38 页）

DeepSeek-V4-Pro 的规格是 **1.6T 总参数 / 49B 每 token 激活参数**；结构自下而上为 `Embedding → Pre-Block Mixing → Residual Mixing → Indexer → CSA/HCA（混合注意力）→ DeepSeekMoE → Post-Block Mixing → Transformer Block × L → Prediction Head → MTP Modules（MTP Loss）`；预训练配方从 `4K → 16K → 64K → 1M` 逐级拉长，数据形态从 **dense 过渡到 sparse。**

讲者由此把"模型特征"直接映射到"优化方向"（第 38 页四条）：

1. 模型状态（参数 + 梯度 + 优化器状态）挤占显存余量，限制"更快执行"的余地；
2. **混合注意力**：不同注意力路径（CSA/HCA）需要各自定制的内核；
3. **细粒度 MoE**：专家计算、通信与 buffer 决定性能；
4. **因果长上下文**：同样 token 数也可能对应不等的实际工作量（负载不均）。

### 3.2.3 先定起点：并行布局与显存策略（第 39–43 页）

- **布局三原则**：模型规模定初始布局（为 activation 与 buffer 预留 headroom；专家优先 `EP`、跨域用 `PP`、必要时加 `TP`）；上下文长度定 `CP`；**拓扑定放置**——通信密集的 `EP/TP/CP` 尽量收在 **NVLink 域**内。
- **实例**：256 GPU 的 DeepSeek-V4-Pro 训练，初始布局 `PP4` + `EP64`。
- **显存策略随上下文切换**：

| | 64K 上下文 | 1M 上下文 |
|---|---|---|
| 并行度 | `CP16` | `CP64` |
| 每 rank token | 4,096 | 16,384 |
| 策略 | 细粒度重算 + offload | 全量重算 |
| 好处/代价 | 给 CUDA Graphs 留出余量 | activation 压力更大 |

- **先证后调**：用 **4K 上下文的 sparse 代理模型**验证所需特性（kernel fusion、packed sequence、CUDA Graphs 等），再用 profiler 定位。

### 3.2.4 四步 playbook 与实测收益（第 36、44–52 页）

| 步骤 | 做什么 | 模块级收益 | 端到端收益 |
|---|---|---|---|
| **STEP 1** 运行开销与显存权衡 | 64K 场景的主要问题是"大量 GPU idle"；用 **kernel fusion** + **CUDA Graphs**（把 packed sequence 用 padding 变静态后捕获/重放） | — | 为后续提供基线 |
| **STEP 2** 混合注意力 | ① 稀疏注意力"左对齐有效索引、只扫前 L 个 slot"，`topk_length = L` 消除空洞；② Indexer **一个内核内完成打分+选择**（Q·K 量化为 FP8，只写回被选中的分数）；③ Group-O 投影 **8 个小 GEMM 合成一个 batched GEMM** 并启用 FP8 | 稀疏注意力 **25%–30%**、FP8 Indexer **25%–35%**、Group-O batched GEMM **50%–60%** | 64K **+4.6%**、1M **+9.0%** |
| **STEP 3** MoE | MoE 中约 **30%–50%** 是 EP 通信；两条路：**跨 batch EP 重叠**（适合跨 NVLink 域，block size 由 nsys 的 silicon time 决定，代价是显存开销）与**用 megakernel 做细粒度重叠**（NVLink 域内更易融合、显存更省） | 融合执行 + 流水化 token 传输；引用的 MoE 侧手法可带来 **30%–40%** 的 MoE 部分加速 | 64K **+14.0%**、1M **+4.6%** |
| **STEP 4** CP 负载均衡 | 根因：**QK score 计算量随序列长度平方增长**——512K / CP32 下同一组内各 rank 的 GPU time 从 **53.1 ms 一路降到 18.4 ms**（差近 3×），R3 成为掉队者、其余空等；手段：用 **zigzag 布局**把"最轻段 + 最重段"配到同一 rank（R0 同时承担 lightest 与 heaviest） | — | 1M **+12.2%**；**64K 无可测收益**（页面原话 `HOWEVER: the balance process also introduces extra overheads`） |

第 52 页"Lessons for Scalable Training"把结论收成一套可复用配方（针对 64K 与 1M 两种上下文），这与第 36 页的 `+59% @64K、+32% @1M` 相互印证——注意后者是**整条 playbook 叠加后的吞吐收益**，比单步收益之和高得多，说明各步之间存在正向协同（例如 CUDA Graphs 与 batched GEMM 都减少 launch 空隙）。

### 3.2.5 要点

- 这套 playbook 的**顺序很讲究**：先解决"GPU 空转"（STEP 1），再解决"注意力算不动"（STEP 2），然后解决"通信藏不住"（STEP 3），最后才谈负载均衡（STEP 4）——因为前一步的收益会改变下一步的瓶颈占比（第 47 页明说："1M 下 indexer 占 step time 的比例更大"）。
- **收益有适用范围，代价也写在页面上**：STEP 4 的 zigzag 均衡在 **1M 拿到 +12.2%**，而同一页紧接着写 `HOWEVER: the balance process also introduces extra overheads`——所以**在 64K 下没有可测收益**。这说明第 45 页那种"把 step time 按模块拆开"的度量是决策依据，而不是"做了就有效"。

---

## 3.3 主线三：KV Cache 是推理侧的第一性瓶颈（第 53–67 页）

### 3.3.1 定位：三重瓶颈、方法分类与集成判定规则（第 53–56 页）

第 54 页先立题——**KV 显存随上下文长度线性增长**，于是形成三重瓶颈：

| 瓶颈 | 页面对应的因果链 |
|---|---|
| **GPU 容量** | 能装下的活跃请求变少 → 有效 batch ↓、可用上下文 ↓ |
| **HBM 带宽** | 每生成一个 token 都要重读历史 → 每步读取字节数 ↑，直接变成带宽受限 |
| **系统 I/O** | 容量溢出时 KV 要在 GPU ↔ CPU / SSD / 远端之间搬运 → **TPOT** ↑，prefill / decode 被拖慢 |

而系统级权衡是三条互相牵制的目标：**维持并发**（固定 GPU 预算下要么加卡、要么减活跃请求）、**维持 TTFT/TPOT**（代价是 offload 与 re-prefill ↑）、以及**全局服务最优点**。

第 55 页把压缩方法分成两大类：**训练期靠架构设计**（`MQA/GQA`、`MLA/CLA`、`Kimi Linear / CSA·HCA / MSA`），**推理期靠运行时压缩**，后者又有三个子类：**量化**（每元素更少比特，配 scales + 保护离群值）、**存储稀疏**（保留更少 KV，剪枝/淘汰）、**计算稀疏**（只读更少的 KV 或页，查询感知 top-K）。

第 56 页回答"压缩放在哪儿"，给出**两条集成路径与一条判定规则**：

| 路径 | 形态 | 收益 | 代价 / 挑战 |
|---|---|---|---|
| **推理引擎内**（vLLM / SGLang / TRT-LLM） | 在线路径（如 INT8、TurboQuant），**压缩后的 KV 常驻 GPU** | 容量↑、每步 decode 读的字节↓、对引擎**透明** | 需与 attention 融合，且与引擎/模型强绑定 |
| **KV 缓存后端**（LMCache / FlexKV / Mooncake） | 卸载路径（如 KVTC、nvCOMP-ANS），**注意力前先还原成标准 KV** | 存储/传输字节↓、CPU/SSD/远端容量↑，并带来**前缀命中潜力** | **解压开销、不直接改善 TPOT** |

> **INTEGRATION RULE**：`KV-read savings > decompression overhead` → **放进推理引擎；否则退回 KV-cache 后端。** 这条规则比任何压缩比都重要——它把"要不要在线解压"变成一个可测量的不等式。

**口头补充（录音问答）**：讲者把"为什么没有 TPOT 收益"说透了——解压算子"绝对会占用 SM"，实测"对 SM 的压力还挺多"，"解压执行的时候推理会受到一定程度的阻碍"；好在"解压带宽基本在 **50 GB/s** 量级、解压比较快"，因此在真实场景里占用时间短；但干扰会让"整个推理系统的稳态变得特别复杂"，"并发高了以后 TTFT 就会下降"，性能波动约 **−5.6% ~ +5%**。

也就是说，KV 压缩不是"免费午餐"，它的收益主要体现在 **TTFT（跨层/跨节点取 KV 的等待）** 与 **容量（并发数）**，而 TPOT 未必改善甚至可能退化——这条判断在第 67 页 Takeaways 里被明确写成"要在编解码与集成成本之后重新度量质量、TTFT、TPOT、吞吐与缓存命中率"。

### 3.3.2 三条技术路线与各自数据

| 路线 | 机制 | 压缩率 / 收益 | 代价与适用 |
|---|---|---|---|
| **TurboQuant**（在线 KV 量化，57–60） | **随机旋转 → Lloyd-Max codebook 查表 → 逆旋转/反缩放 → 可选 QJL 内积修正**；每向量只存"质心索引 + 残差符号位 + 标量范数"；为解码把 KV 布局改为**页内字段平面化**（16 token 一页、**2,144 B/KV head**），使 32 B 扇区有效字节 **3.68 B → 29.75 B**；解码用 **tiled MMA**（64-token tile、双缓冲、FP32 running state） | Llama-3.1-8B 上 **≥4.5×** 名义载荷压缩；解码吞吐（Qwen3-8B、BS=32、仅 decode）短上下文约 **1.25–1.28×**、长上下文 **2.5–4.8×**（峰值 **4.84×**） | 需要定制解码内核与布局；收益随上下文变长放大，基线在长上下文多处 **FA OOM** |
| **KVTC**（存储与传输压缩，61–64） | **跨层/头拼接 + PCA + 动态量化**（重要主成分给更多比特，可选 nvCOMP DEFLATE），压缩单元为**一个 token**、输出无损变长字节；配套**流水化解压** | 压缩率 **8×–64×**；16× CR 时精度损失 **≤0.8 分**（3 模型 × 5 基准）；单请求 TTFT **−5.5% ~ −10.8%**；96 GiB 压力下吞吐 **2.49×**、TTFT **−64%**、命中率 **11.9% → 74.7%** | 粒度不匹配 + dequant/逆 PCA 太重，故定位在存储/卸载侧；容量充足时几无开销也几无收益 |
| **nvCOMP-ANS**（无损熵编码，65–66） | 用 ANS 熵编码做**无损** KV 传输压缩，解码后可**精确还原** | TTFT **≈1.4×（BF16）/ ≈1.1×（FP8）**；P90 TTFT 最高 **+18.7%**、均值 **+8.2%**，且**低并发收益最大** | 无损意味着压缩率有限，收益集中在"传输占比高"的低并发场景 |

第 67 页 Takeaways 三句话概括了这一段的工程哲学：**KVTC 在显存压力下保住吞吐；nvCOMP-ANS 在低并发下降低 TTFT 且能精确恢复；任何 codec 都要在"编解码 + 集成"成本之后重新度量质量、TTFT、TPOT、吞吐与命中率。**

三条路线的实现细节（决定它们能不能兑现压缩比的关键）：

- **TurboQuant（在线量化）**：解码路径是 `K/V 向量 → 随机旋转（高维下坐标更平滑、近似独立）→ 预计算 Lloyd-Max codebook 查表（码心 -1.2 / -0.4 / +0.4 / +1.2，只存最近质心的 ID）→ 逆旋转与反缩放 → 可选的 QJL 内积修正（残差投影取符号位）`；每个向量只存**质心索引 + 残差符号位 + 标量范数**，而旋转/投影矩阵与 codebook 是**全局共享、不逐向量存储**。为了让解码真的快，它把布局从"按 token 交错"（`D=128` 时 **134 B/token**，V codes 从第 66 字节开始）改为**页内字段平面化**（16 token 一页、每 KV head **2,144 B**，K/V codes 各 1,024 B、K norms 与 V scales/zeros 各 32 B），使 **NCU 测得的每 32 B 访存扇区有效字节从 3.68 B 提到 29.75 B**；解码内核按 **64-token KV tile** 流式加载、双缓冲异步、把码字直接映射成 MMA fragment，并用 **FP32 running state（max / exp sum / weighted output）跨 tile 保持 + 在线 softmax 重缩放**，最后对多个 KV split 做稳定归约输出 BF16。实测（Qwen3-8B、BS=32、仅 decode、128 个输出 token）相对 FP16 KV 的 FlashAttention 基线，短上下文约 **1.2–1.3×**、长上下文 **3.7–4.8×**（峰值 **4.84×**）；基线在长上下文多处 **FA OOM = capacity-skipped**，这也是"容量本身就是性能"的直接证据。
- **KVTC（存储/传输压缩）**：流水是 `跨层/跨头拼接 → PCA 变换 → 动态量化（重要主成分给更多比特）→ 可选 nvCOMP DEFLATE`，压缩单元是**一个 token**、输出**无损变长字节**；之所以定位在存储侧，是因为**粒度不匹配**（attention 按 per layer/head 取用，KVTC 按跨层/头组织）与**运行时开销**（dequant + 逆 PCA 对

在线解码太重）。集成走**可插拔 codec 后端**（引擎侧 vLLM / SGLang / TRT-LLM，管理层 LMCache / HiCache / Dynamo），并把解压与 TP-GEMM、AllGather 做**三段错位流水**来回收时间。

- **nvCOMP-ANS（无损熵编码）**：在 GPU / CPU 内存 / SSD 三级之间搬运 KV 时做 ANS 熵编码，解码可**精确还原**，因此不需要改内核——这也解释了它的收益为何"温和但稳定"。

**KVTC 的两组关键数据**（值得单独记）：① **单请求 TTFT**：GLM-5.1 FP8 / 8×Hopper / PCIe Gen5 / TP=8 / vLLM+LMCache、压缩率 4.7× 下，四组上下文分别 **843→773 ms（−8.4%）、591→558 ms（−5.5%）、407→377 ms（−7.4%）、204→182 ms（−10.8%）**；页面结论句写"降低 5.5%–10.6%"，与柱值最大降幅 **10.8%** 存在小幅不一致（已在明细页标注 `(?)`）。② **显存压力下**（96 GiB 主机内存、并发 16）：输出吞吐 **2.49×**、平均 TTFT **−64%（108.8 → 39.1 s）**、LMCache 命中率 **+62.8 个百分点（11.9% → 74.7%）**；而在容量充足（400 GiB）时开销可忽略（吞吐变化 **+3.6% ~ −2.1%**，主机内存占用约小 **4.7×**）。这一对数据说明：**压缩的价值不是"平均更快"，而是"避免性能断崖"**。

---

## 3.4 主线四：从"算得快"到"找得准"——KV Router 与 Flash Indexer（第 68–78 页）

这一段是中文投影片，讲的是 NVIDIA Dynamo 体系里的**缓存感知路由**，骨架可以概括为"一个成本函数 + 一张位置地图 + 六代索引"。

### 3.4.1 打分机制：一个成本函数（第 69–70 页）

请求路径是四跳：**请求 Token → Block Hash → KV Indexer（给出缓存前缀命中）→ Slot Tracker（给出活跃的 Prefill + Decode）→ Cost Function → 最低成本 worker**；另一条反向通路是 worker 主动推送 **KV Events（缓存新增/删除）** 来更新索引。成本函数写作：

```
Cost = Prefill 成本 + Decode 成本 +（可选）活跃请求成本
```

页面上的总结句把两类信号的分工说得很清楚：**"缓存告诉我们能省多少，负载告诉我们还能接多少。"** 这也是整个路由设计的核心张力——只按命中率路由会把热节点压垮，只按负载路由会把缓存白白丢掉。第 69 页另给出生产验证：Baseten 侧 **TTFT 快 2×、TPS 1.6×**；另一家（美团）**TTFT 快 2×、KV cache 多 20%**。

### 3.4.2 索引的键设计：为什么必须带位置（第 71–72 页）

Flash Indexer 不生成 token、也不转发请求，只回答一个问题——**"哪里有缓存？"**；四步链路是 `prompt blocks → block_hashes(h0 h1 h2 …) → 实时缓存地图（持续接收 Store/Remove 事件）→ 各 worker 的连续命中深度`（示例输出 `Worker C: 4`、`Worker A: 2`、`Worker B: 0`）。它同时受两个约束：**查询慢了会拖慢路由，状态旧了会错过可复用的 KV。**

索引的键不是 block 内容，而是 `(position, block_hash)`：

| key | value（示例） |
|---|---|
| `(0, h0)` | Worker A, C, D |

| key | value（示例） |
|---|---|
| `(1, h1)` | Worker A, C |
| `(2, h2)` | Worker C |
| `(3, h3)` | Worker C |

**为什么必须带 position**：同一个 `block_hash` 可能出现在不同位置，只有位置也一致，才说明这段 prefix 真的连续（示例中只有 Worker C 在 position 0–3 全部命中，因此是唯一能连续命中 4 块的 worker）。

### 3.4.3 六代索引演进：从 O(W×D) 到 O(D/J+W)（第 73–76 页）

| 代 | 实现 | 吞吐 | 关键突破 / 瓶颈 |
|---|---|---|---|
| GEN 1 | Naive Nested Dict | **385K** block ops/s | "按 worker 分册的账本"，外层扫 worker、内层扫深度，复杂度 **O(W×D)**；暴露了两个问题：访问模式不对、`local_hash` 需要在 prefix 上下文中消歧（引出 `local_hash` + `sequence_hash` 双哈希） |
| GEN 2 | Rust Actor | — | 单队列保证顺序，但读并发未释放 |
| GEN 3 | Inverted Index | — | 按 block 反置到 worker |
| GEN 4 | Radix Tree | **4M** | **用树表达前缀上下文，碰撞被上下文约束住**；双键设计：`local_hash` 负责遍历、`seq_hash` 负责事件定位；仍是单线程 actor 之后 |
| GEN 5 | Concurrent Radix | — | 读并行、写保序 |
| GEN 6 | **Positional Indexer + Jump Search** | **170M** | `index[position][local_hash]` 直接随机访问某深度，按 checkpoint（0/64/128/...）整段跳过，只需 `O(D/J+W)`；`SeqEntry::Single` 直接命中，只有 `Multi` 碰撞才 **lazy** 计算 `sequence_hash` |

页面标注的累计提升为 **440× / 42×**；基准配置写明 `24-core Arrow Lake 285K 桌面机 + Mooncake FAST'25 trace + mock engine + 16,384 GPU blocks + 24 个事件处理线程`。这条曲线的意义在于：**索引从"数据结构问题"变成了"系统吞吐问题"**——路由要想真正吃到缓存，索引的查询成本必须低到可以忽略。

### 3.4.4 从组件到服务：Standalone KV Indexer（第 77–78 页）

把索引能力从 router/frontend 里拆出来，做成**可独立查询的服务**。定位写得很克制——它**是**：独立维护 cached KV blocks 的索引、订阅 worker 的 KV cache 事件、返回 prefix overlap；它**不是**：不是 standalone router、不做最终 worker 选择、不维护请求生命周期；**适合**：调试 router 看到的 KV 状态、外部系统自建路由、观察不同 worker/tier 的缓存分布。架构上是"**ZMQ 事件面负责 ingest，HTTP 请求面负责注册、查询与观测**"：worker 用 ZMQ PUB 推事件 → indexer 侧 SUB 收事件 → 写入 `Worker Registry` 与 `(model, tenant) → Radix Tree` → 对外暴露 `GET /health`、`POST /register|/unregister`、`GET /workers`、`POST /query|/query_by_hash`、`GET /dump`、`POST /register_peer|/deregister_peer`、`GET /peers`，查询结果还可按 **gpu / cpu / disk 分层**返回。

## 3.5 主线五：智能体推理重写了性能度量（第 79–96 页）

### 3.5.1 度量先变（第 79–81 页）

智能体（Agent）场景把"一次请求"变成"一条会话轨迹"，于是 SLA 表被扩展为：**TTFT、TPOT、请求级 E2E、轨迹级 E2E、首个答案 token 时间、系统吞吐、满足 SLA 的并发 agent 数（AA-AgentPerf），以及全局 KV 缓存命中率 > 90%。**

第 80–81 页用一条真实 trace（Claude Code 轨迹）量化了这件事的规模：

- 单会话**原生上下文窗口 1,048,576**；请求 p99 接近 **900K**；ISL 分布为 32K–128K 占 21%、128K–256K 占 27%、256K–512K 占 30%；
- 单会话**累计输入 token 峰值 219K**；存在 **60 s 级的工具/用户间隔**，甚至出现"第 42 步间隔 46.7 小时后恢复"的极端情况；
- 单步时延分解显示 prefill（TTFT）与 tool/user gap 是主要构成项。

**含义**：单请求 TTFT/TPOT 已经描述不了体验；**"间隔可达数十分钟、KV 早已被逐出、恢复时要么重算、要么从存储取回"**才是常态。

### 3.5.2 四组目标：从容量到路由（第 82–92 页）

| 目标 | 页面 | 关键事实 |
|---|---|---|
| **容量**：1 GB 能装多少 token | 85 | 每 token KV **55.9 KB**（MLA 43.9 / DSA 9.8 量级拆分）；1 GB、单节点 8 GPU 时，TP8 复制 8 份仅约 **33 个会话**，而 **DEP 约 200 个会话**；同节点 token 容量 **1.26M → 7.6M（6×）** |
| **命中率**：实际命中率 ≈ 理论上界 | 86–87 | GPU 侧 G1（约 53 GB/rank）、主机侧 G2（512 GB/节点）、G3（溢出到 1 TiB 量级）；重算占 **8.2%**、与理想差距 **−3.9 pt**；会话并发 16/32 时的容量与 in-flight 余量被逐项列出 |
| **并发**：decode 并发必须跟得上 | 88 | TP8 的 ceiling 为 33 会话；DEP 下按 `1.26M vs 7.6M tokens/node` 逐档对比 |
| **路由**：智能体路由策略 | 89–90 | Hard affinity 在 DEP 下出现**负载不均**（32 个 DP rank 中大量时段占据 ≥90%）；给出 KV-aware / Hard / Soft affinity 的取舍与 NIXL 点对点拉取 |

第 89 页那张"每分钟一格的 KV 占用热力图"值得单独注意：它把"Hard affinity 会把某些 rank 长期顶到 90%+"这件事可视化成了一个失败模式，等价于给"缓存亲和"这条路画了一条边界——**缓存亲和必须与负载均衡联立求解。**

### 3.5.3 会话网关 ThunderAgent（第 91–96 页）

方案是一个**全局控制回路**：每 **Δt = 5 s** 一拍，按 `util = 活跃 program 的上下文量 ÷ GPU KV 池` 决定 `resume / pause`：

- 门限：**util < 0.80 恢复、0.80–0.85 观察、> 0.95 暂停**（页面写 `0.80 / 0.85 / 0.95`，并有"97% resume 上限"的字样）；

- 恢复顺序：优先选"上下文最小、重算最便宜"的 program，并落到**空闲 KV 最多**的 worker；等待超过 **1800 s** 则强制恢复（避免饿死）；
- 暂停时若命中冷区，则**把 KV offload 出去**（第 94 页：G1 95%、G2 60%，且多 program 共享同一份 KV 副本）。

**实测（第 93 页）**：SWE-Bench Lite、GLM-5.2、8× NVIDIA Hopper、会话并发 32 —— **E2E 均值 −51%、TTFT p50 −87%、TPOT 减半、生成吞吐均值 ×1.6。**

**延伸到 RL rollout（第 95 页）**：与 Dynamo + VeRL 集成，Uni-Agent SWE-Bench、Qwen3-Coder-30B-A3B-Instruct、8×Hopper、rollout-only 场景，页面标注 **+16.5% / +6.8%** 两类提升。

第 96 页"Assets & Future Work"收尾：文档、Flash Indexer 与 Dynamo 的集成 PR（页面上可见 `#11447 / #11787 / #11616` 等编号）、以及 ThunderAgent v2 等后续工作。

---

## 3.6 贯穿全场的方法论

五段技术内容风格迥异，但方法论高度一致，抽出四条：

1. **测量先行，把瓶颈拆到可决策的粒度。**第 45 页把 step time 按 `MoE experts / EP comms / Indexer + CP comm / Sparse Attn / Group-O Proj / Others` 分段堆叠，才好判断"下一步该打哪一块"；第 44 页直接指出"64K 场景的主要问题是大量 GPU idle"。
2. **先抬高理论上界，再用工程把它兑现。**1.42× → 1.92× 是上界推导，后面的 11 项优化是把 1.92× 兑现成实测 1.16–1.30×（几何平均）；对应地，第 34 页用 token-bucket 查表把"每个形状最优解"冻结下来。
3. **人类硬件直觉 + LLM 辅助建模。**第 7–8 页明确把这套组合写进了方法论页：人提出 `16×8→32×4` 的观察，LLM 把问题统一成位级置换模型，再用 BFS 搜索 + 模拟器验证 + CuTe 内核集成。
4. **如实记录负结果与代价。**CP 均衡在 64K 下"no measurable perf gain"（均衡开销抵消了收益）、KV 压缩"没有直接的 TPOT 收益"、combine 数据是 bf16 带来的 NVLink 相对代价——这些都留在幻灯片上，构成了这套材料可信度的重要来源。

# 四、总结

## 4.1 十二个关键结论

| # | 结论 | 出处 |
|---|---|---|
| 1 | MoE 层"四段式"串联执行时，通信与计算互不重叠；把 dispatch/FC1/FC2/combine 融进**一个 persistent kernel** 并按专家波次交织，理论上界从 1.42×（Comet 式）提升到 **1.92×** | 页 1 |
| 2 | 融合的代价是真实的：通信与计算共享 SM、借用 shared memory、干扰 TMA 主循环；`NVFP4` 让 L1/L2 提速后，combine 侧 bf16 的 NVLink 流量代价反而被放大 | 页 3 |
| 3 | 小 token（解码）场景的关键是 **swapAB 把 token 映射到 N 维**：WGMMA 的 N 可小到 8，tile 利用率最高提升 **8×**；转置代价被推到 epilogue | 页 7、19 |
| 4 | 微架构级取舍往往反直觉：dispatch 侧 **1 个活跃 warp 反而比 2/4 个更快**（约 132 行在飞已超过 NVLink 带宽时延积，更深的队列拉长每次往返并争用 GEMM） | 页 27 |
| 5 | 融合的收益随形状变化：4×H200 / EP4 / DeepSeek-V4-Pro 形状下，**per-tensor FP8 几何平均 1.16×（峰值 1.35× @8192）**，**blockwise FP8 几何平均 1.30×（峰值 1.72× @32768）**；小 token 档位由 launch 与通信主导，收益有限 | 页 33–34 |
| 6 | 大规模训练先分"装得下 / 跑得快"，再按 **布局 → 显存策略 → 四步 playbook** 推进：初始 `PP4 + EP64`；64K 用 `CP16` +细粒度重算+offload，1M 用 `CP64` +全量重算；整条 playbook 实测 **64K +59%、1M +32%** | 页 35–43、36 |
| 7 | 四步 playbook 的模块级收益：稀疏注意力 **25–30%**、FP8 Indexer **25–35%**、Group-O batched GEMM **50–60%**；端到端 64K **+4.6%**、1M **+9.0%**（MoE 一步为 64K **+14.0%**、1M **+4.6%**） | 页 47、49 |
| 8 | "每 rank token 数相等 ≠ 每 rank 工作量相等"：QK score 计算量随序列长度平方增长，512K/CP32 下同组 rank 的 GPU 工时可差近 3×（53.1 → 18.4 ms）；用 zigzag 对角配对重分配后 **1M +12.2%**，但均衡本身带来额外开销，**64K 下无可测收益** | 页 50–51 |
| 9 | KV Cache 是推理侧第一性瓶颈，三条压缩路线各有边界：**TurboQuant ≥4.5×**（在线量化，解码吞吐 1.25×–4.84× 量级）、**KVTC 8×–64×**（精度损失 ≤0.8 分，TTFT −5.5%–10.8%，显存压力下吞吐 2.49×、命中率 11.9%→74.7%）、**nvCOMP-ANS ≈1.4×/1.1×**（无损，P90 TTFT 最高 +18.7%）；压缩**没有直接的 TPOT 收益**，是否放进推理引擎取决于一条判定规则：`KV-read savings > decompression overhead` | 页 53–67 |
| 10 | 缓存感知路由的本质是"命中率 × 负载"的联立：KV Router 用前缀命中 + 实时负载打分，Flash Indexer 以"位置→worker"地图回答"哪里有缓存"，索引吞吐 **385K → 4M → 170M block ops/s（440×/42×）** | 页 68–78 |
| 11 | 智能体场景下度量必须升级：单会话 ISL 峰值 **219K**、请求 p99 近 **900K**、工具/用户间隔可达 **60 s+**（甚至 46.7 小时），于是出现 trajectory 级 E2E、首个答案 token 时间、AA-AgentPerf、全局命中率 >90% 等新指标 | 页 79–81 |

| # | 结论 | 出处 |
|---|---|---|
| 12 | 会话网关 ThunderAgent（5 s 一拍，util 门限 0.80/0.85/0.95，按上下文大小决定恢复顺序，>1800 s 强制恢复）在 SWE-Bench Lite / GLM-5.2 / 8×Hopper / 并发 32 上取得 **E2E −51%、TTFT p50 −87%、TPOT 减半、吞吐 ×1.6** | 页 91–93 |

## 4.2 三句话的电梯陈述

1. **算子层**：MoE 的最优解不是"更快地做通信"或"更快地做 GEMM"，而是**不要在两者之间划边界**——用一个持久内核把四段揉在一起，再用 30 余项微架构优化偿还"融合税"。
2. **系统层**：显存与带宽是这一代大模型的硬约束，因此**压缩（KV 量化/PCA/熵编码）+ 路由（缓存命中）+ 调度（会话网关）**成为新的性能来源，其收益结构高度依赖"你处在哪种并发与上下文分布下"。
3. **度量层**：当负载从"请求"变成"会话轨迹"，**SLA 必须同步重写**；否则优化的方向会系统性跑偏（例如把 TTFT 压到极致，却因 KV 反复驱逐导致轨迹级时延更差）。

# 五、分析与评论

> 本章为分析者观点，与前述事实陈述分开表述。凡引用幻灯片的数字均已在前文给出出处页。

## 5.1 这份材料的技术价值

**第一，它罕见地给出了"从理论上界到实测数字"的完整闭环。** 大多数技术分享停在"我们做了融合、我们更快"，而这份材料在页 1 给出 1.42×/1.92× 的**理论上界推导**，在页 21–32 逐条列出 11 项优化（几乎每一条都带具体百分比或小数），在页 33 用 13 档 token sweep 给出**几何平均与峰值**，在页 34 用一张 Summary 收敛。这套"上界—手段—数据—固化"的叙事结构，本身就是可复用的技术汇报模板。

**第二，它把"负结果"和"代价"放在了显眼位置。** （录音侧还有一条更硬的边界：讲者自己承认这些训练侧的优化**"没法直接迁移到推理"**，见配套《讲稿整理》第六章。） `Fusion is not free`、`KV 压缩没有直接的 TPOT 收益`、`CP 均衡在 64K 下 no measurable perf gain`、`Hard affinity 导致负载不均`——这些内容对读者的价值不低于正面结论，因为它标出了**方法的边界**。

**第三，它覆盖了训练与推理两条链路，并且两条链路的结论是互通的。** 训练侧的"通信占比 30–50%，要藏到计算后面"与推理侧的"把 dispatch/combine 融进内核"，本质是同一个命题在两种负载上的两次求解；训练侧的 CP 负载不均衡与推理侧的 Hard affinity 失衡，也是同一个"亲和性 vs 均衡"矛盾的两个投影。

**第四，方法论的透明度高。** 页 7–8 直接把"人类硬件直觉 × LLM 辅助数学建模"写进方法论页，并给出五步流程（观察 → 用 LLM 建模 → 搜索 → 验证 → 集成）。在 2026 年的语境下，这比任何单点优化都更有信息量：**AI 已经从"被调用的工具"进入"内核级设计协作"的位置。**

## 5.2 我的四条趋势判断

**判断一：性能工程的重心正在从"框架层"下移到"边界层"。** 过去十年的深度学习系统优化主要在框架层（图优化、算子库、并行策略）。这份材料里真正的大收益几乎都来自"边界"：通信与计算之间（单内核融合）、存储与计算之间（KV 压缩 + 解压流水）、索引与路由之间（把索引做成独立服务）、请求与会话之间（会话网关）。**共同模式是：找出两个原本串行的阶段，用一个新的抽象把它们并成一个可流水化的整体。** 这条经验可以直接迁移到任何"有明确阶段边界"的系统。

**判断二：低精度与压缩已成"一等公民"，但必须与内核同源设计。** NVFP4/MXFP8 让 GEMM 变快的同时，反而让 bf16 的 combine 成为相对瓶颈（页 3）；TurboQuant 的 4-bit KV 之所以能吃到 2.5×–4.8× 的解码吞吐，靠的是**为压缩布局专门写的 tiled MMA 解码内核**（页 58–59）。结论很清晰：**压缩比只是参数，能不能兑现取决于有没有配套内核。**这也解释了为什么"无损压缩"（nvCOMP-ANS）的收益看起来温和（≈1.4×/1.1×、均值 +8.2%）——它不需要改内核，所以天花板也低。

**判断三：推理系统的核心问题已经从"调度请求"变成"管理系统状态"。** （录音带来的限定：第 2 场讲者在问答中说“前面讲的这些其实跟训练更相关，推理方向可能完全不一样，**这些东西没法直接迁移过去**”——因此此处所说的“互通”仅限于**思路层面**，实现与参数不可平移。） 当单会话上下文达到 200K–900K，"状态"（KV）的体积和位置就成了系统的主要自由度：**1 GB 只装得下约 33 个会话（TP8 8× 复制）而 DEP 能装约 200 个**（页 85），这句话几乎决定了所有后续设计。缓存感知路由（页 68–78）解决"状态在哪"，会

话网关（页 91–94）解决"状态何时该在"，KV 压缩（页 53–67）解决"状态多大"。三者是同一件事的三个侧面。

**判断四：SLA 会跟着负载形态一起演进，指标设计本身就是产品设计。** 页 79 那段 SLA 表（TTFT / TPOT / 请求 E2E / 轨迹 E2E / 首个答案 token / 系统吞吐 / 并发 agent 数 / 全局命中率）看起来只是"加了几个指标"，实质是把**"用户感知"从一次请求扩展成一段会话。**一旦采用轨迹级 SLA，很多"局部最优"（例如一味压低 TTFT 而频繁驱逐 KV）会暴露为整体劣化。这一点对所有做推理服务的人都有直接启发。

## 5.3 局限与待核验之处

**材料层面的局限：**

1. **只有投影片，没有讲解。**所有"为什么这样选"的口头论证无法获得；页面上很多判断（例如"1 warp 比 2/4 快"）只给了结论与一句解释，缺少完整实验条件。
2. **OCR 的不确定性。**第 33 页柱状图"哪根柱子对应哪个倍数"存在归属不确定；第 26 页（WGMMA 寄存器布局）与第 18 页（warp 编制表）的表格串行严重；第 84 页被界面元素分割。这些位置已在正文与附录中标注 `(?)`，引用时请以原图为准（配套影印本可逐页对照）。
3. **重复拍摄。**第 4–5、39–40、41–42 页为同页重拍，个别数字在两版之间略有出入（已取更完整的一版并在逐页整理中注明）。
4. **演讲者与场合。**材料只显示 NVIDIA 标识、`gtc26-*` 链接、以及"引用 DeepSeek-V4 技术报告 / Megatron Core / Dynamo"等线索，不足以确认讲者身份与会议全称，本文因此不作断言。

**技术层面的待核验点：**

1. **收益的条件性。**1.92× 是理论上界，实测几何平均 1.16–1.30×，峰值只出现在特定 token 档位；换硬件（如从 H200 到 Blackwell）、换 EP 度、换专家数，收益结构都会变。把它当作"普遍 1.9×"是误读。
2. **"没有直接的 TPOT 收益"的含义。**KV 压缩在页 56 被标注为对 TPOT 无直接收益，但 TurboQuant 页 60 又给出解码吞吐 2.5×–4.8× 的提升。二者并不矛盾（一个是"压缩本身"，一个是"压缩 + 专用内核"），但需要读者自己拼接——**这也是本材料最容易被误读的一处。**
3. **端到端成本模型缺失。**压缩率、命中率、并发之间的定量关系只在页 85–88 有片段数据；缺少一个把"压缩比—命中率—TTFT—吞吐"串起来的统一模型，因此"该选哪条压缩路线"仍要靠场景实测。
4. **训练侧的收益归因。**STEP 1–4 的端到端收益（+4.6%、+9.0%、+14.0%、+4.6%）与该步的模块级收益（25–60%）差距很大，说明瓶颈在步骤间迁移；要复现"整条 playbook +59%"必须按同样顺序执行，不能只抄其中一步。
5. **单点实验。**token sweep 是 4×H200/EP4；训练是 256 GPU；智能体基准是 8×Hopper + 并发 32。这些规模下成立的比例关系，在千卡/万卡与更高并发下未必线性外推。

## 5.4 对实践的启示（按角色）

**做内核 / 算子的人：** - 先算**理论上界**（把串行时间轴拆开、算出可重叠比例），它决定了值不值得投入；页 1 那种三行对比图是最便宜的投资判断工具。 - 融合之后要立刻列一份**"融合税清单"**（smem 借用、TMA 干扰、精度不对称带来的流量放大），后续优化按这份清单还债。 - 对"小 token / 小 batch"这类极端形状，优

先考虑**维度映射**（swapAB 把 token 放到 N 维）而不是继续调 tile 大小。 - 每个优化都要**按形状分桶验证**，最后固化为查表（token-bucket heuristics），否则会在形状切换时回退。

**做训练系统的人：** - 把"装得下"与"跑得快"分开立项，先定**并行布局与显存策略**（布局三原则：规模定初始布局、上下文定 CP、拓扑定放置），再动内核。 - 优化顺序比优化本身重要：**先消 launch/GPU 空转（fusion + CUDA Graphs），再算注意力，再藏通信，最后谈负载均衡。** - 用**代理模型**（本例是 4K 稀疏代理）先验证特性可用性，可以大幅降低在 1M 上下文上试错的成本。 - 记住"等 token 数 ≠ 等工作量"：因果长上下文的负载均衡要按真实工作量度量，否则会出现"看起来均衡、实际不均衡"。

**做推理服务 / 平台的人：** - 先回答"我的负载在哪一段"：低并发看 TTFT（无损压缩收益最大）、高并发看容量与命中率（有损压缩 + 缓存路由收益最大）、长上下文看解码内核（在线量化 + 专用 MMA 收益最大）。 - 把 KV 当作**有位置的资源**来调度：路由要考虑"命中 + 负载"两个信号，纯亲和会造成第 89 页那种"长期 90%+"的失衡。 - 为智能体负载补上**轨迹级指标**，并配一个低成本的控制回路（本例 5 s 一拍、三个 util 门限），其收益（E2E −51%）远高于多数单点内核优化。 - 任何 codec / 路由 / 网关上线后，都要**重新度量质量、TTFT、TPOT、吞吐与命中率**（页 67），因为它改变的是系统平衡点，而不是某一个数。

## 5.5 如果继续深挖，值得补的八个问题

1. MegaMoE 在 Blackwell（SM100）与 Hopper（SM90）上的收益差多少？`swapAB` 在 SM100 的 WGMMA/FMA 结构下是否仍成立？
2. 1.92× 上界推导里对"通信与计算可完全重叠"的假设有多强？实际能达到重叠率多少？
3. per_tensor 与 blockwise 的选择准则是什么？是否与专家数、token 分布、以及 combine 精度绑定？
4. 训练侧"EP 通信占 MoE 30–50%"在 PP4/EP64 下随规模如何变化？到万卡是否仍成立？
5. TurboQuant 的精度损失在大 batch 长上下文下如何累积？与 KVTC 的 PCA 路线是否存在叠加收益？
6. KV Router 的两类信号（命中/负载）权重如何标定？在负载突增时如何避免"缓存热节点雪崩"？
7. ThunderAgent 的 5 s 控制周期与真实会话间隔（60 s–46.7 h）如何匹配？更长的暂停是否该直接 offload 而非保留在 G2？
8. 轨迹级 SLA 如何在多租户下做定价与配额？"满足 SLA 的并发 agent 数"是否会成为新的集群容量单位？

# 六、逐页要点整理

本章按五个部分逐页给出整理稿。每一页给出“推测标题 + 该页要点 + 图表说明”，数字与英文术语均按幻灯片原样保留；`(?)` 表示识别或归属不确定，`识别内容不足` 表示该页文字过少或版面被截断。原图见配套影印本的同编号页。

## 6.1 第一部分 • MegaMoE：单内核融合的 MoE 算子（第 1–34 页）

这一段是全场的“内核篇”。先看融合能带来多少理论上界（第 1–3 页），再看优化清单如何逐条兑现（第 4–32 页），最后用 token sweep 给出实测加速比并收束（第 33–34 页）。阅读时可把第 4–5 页的大纲当成目录：后面每一页都在回答这张清单上的一项。

### 第 1 页 | MegaMoE recap — What is MegaMoE

- 一句话定义（OCR 首行要点）：**Fine-grained pipeline to better overlap communication and compute** —— 细粒度流水线，以更好地重叠通信与计算。
- 该页用三条泳道时间轴对比三种方案：
- **(a) Naive Solution**：`Dispatch → L1 → Act → L2 → Combine` 顺序执行。
- **(b) Comet**：把 `Communication`（`Dispatch`、`Combine`）与 `Computation`（`Act` 等）拆成两条流水线；标注 **Theoretical speedup: 1.42x**。
- **(c) Ours**：`Disp(atch) All-to-All` 与计算重叠，计算侧为 `L1 / L2` 交替重复（`L1 L2 L1 L2 L1 L2`），并逐段与 `Act`、`Combine` 交织；标注 **Theoretical speedup: 1.92x**。
- 图上出现的计算/通信单元标签：`Linear 1 GEMM`、`SWiGLU + FP8 Cast`、`Linear 2 GEMM`；通信按 **Expert Wave 1 / Expert Wave 2 / Expert Wave 3** 分波次，波次之间为 `Combine All-to-All`。
- 图描述：整页是一张三行对照的时序图，横轴为时间。越往下（Naive → Comet → Ours）通信块与计算块的重叠程度越高，最下方一行把通信切成与 Expert Wave 对齐的多段，从而与 `L1/L2` 计算段交错填满时间轴；右上角给出 1.42x 与 1.92x 两处理论加速比。
- 页脚残留文字 `rom Deeps` → 推测为出处标注 `from DeepSeek…` **(?)**，OCR 不清。

### 第 2 页 | MegaMoE support matrix

- **Inference and training are both supported** —— 推理与训练均支持。
- **Compute data types**（计算数据类型）：
- `W4A4`
- `W8A8`
- `W16A16`
- `Mixed precisions (W4A8, W8A16 and more in progress)` —— 混合精度，W4A8、W8A16 及更多仍在开发中。
- **Architectures**（支持架构）：
- `Hopper`

- `Blackwell`
- `Blackwell GeForce`
- `Rubin`
- 通信/框架相关（OCR 残缺较多）：
- `72andIBRDMAwithNCCLGIN(WIP)` → 疑似与 **IB RDMA + NCCL GIN** 相关，且标注 WIP（开发中）；前半段数字与缩写不可辨 **(?)**。
- `eworkintegration` → 应为 `Framework integration`（框架集成）。
- 集成对象：`LLM` (?)、`FlashInfer`、`cuDNN FE (WIP)`（cuDNN Frontend）。
- 该页为矩阵/清单式投影片，OCR 只保留了条目文字，无图表趋势信息。

### 第 3 页 | MegaMoE on Blackwell — Challenges from fusion

- 副标题为 **Challenges from fusion**（融合带来的挑战），核心论点：
- **Fusion is not 'free'** —— 融合不是"免费"的。
- **Communication and compute in the same SM** —— 通信与计算跑在同一个 SM 上。
- **Comm borrows smem from compute** —— 通信要借用计算侧的 shared memory（SMEM）。
- **Comm traffic can interfere mainloop TMA efficiency** —— 通信流量会干扰主循环的 TMA 效率。
- **NVFP4 halves L1/L2 time while combine data is always bf16, NVLink traffic will pay more cost** —— NVFP4 把 L1/L2 的耗时减半，但 combine 数据始终是 bf16，因此 NVLink 通信的代价占比更高。
- 图描述：该页为一张分区时序图，上方是 `Dispatch / L1 / L2 / Act / Combine` 的分块时间轴（`L2 L1 L2 L2 / Act Combine` 交替），右侧有两个纵向指标标注 **Interference between compute and comm**（计算与通信的相互干扰）、**GEMM efficiency**（GEMM 效率）与 **NVLink efficiency**（NVLink 效率）；图中无具体数值曲线，只以标签说明干扰会同时压低 GEMM 与 NVLink 两类效率。

### 第 4 页 | MegaMoE on Blackwell — Optimizations（与页 5 同一张投影片，右侧内容被裁切）

- 标题：**Optimizations** 大纲页，三个大项： 1. **GEMM Perf optimizations**
  - **Large tile size (256x256)**
  - **Dynamic tileN with swapA/B**
  - **In-TMEM epilogue transpose** 2. **Reduce interference between communication and compute**
  - **MEMBAR synchronization optimization**
  - `Batch-commit`
  - `Async UREDGR` 3. **Combine perf issues and optimization(s)**
  - `Three modes for combine traffic`
  - `Phase interleaving between FC1 and FC2`（OCR 作 `hase interleavingbetweenFCl andFC2`）
  - `Adaptive traffic control`（OCR 作 `tive traffic control`）

  - `Support for low-precision combine types`（OCR 作 `rt for low-precision co`）

- 该页右侧文字被画面边缘截断，条目只剩词尾片段；**页 5 是同一张投影片的另一次翻拍，内容更完整**，两页可互相校对。

### 第 5 页 | MegaMoE on Blackwell — Optimizations（页 4 的补拍版本）

- 与页 4 为同一张投影片，标题与结构一致，右侧裁切更少，因此完整列出优化清单：
- **GEMM Perf optimizations**：`Large tile size (256x256)`；`Dynamic tileN with swapA/B`；`In-TMEM epilogue transpose`。
- **Reduce interference between communication and compute**：`MEMBAR synchronization optimization` → `Batch-commit`、`Async UREDGR`。
- **Combine perf issues and optimizations**：`Three modes for combine traffic`、`Phase interleaving between FC1 and FC2`、`Adaptive traffic control`、`Support for low-precision combine types`。
- 该页为纯文字大纲，后续各页（6–16）正是按这份大纲逐条展开。

### 第 6 页 | MegaMoE on Blackwell — Large tile size (256x256)

- 论点：**Larger tile has more memory access budget** —— 更大的 tile 意味着更充裕的访存预算。
- 具体做法：**Using 256x256 instead of 256x192** —— 由 256×192 改为 256×256。
- 关键技巧：**Overlapping acc trick to achieve nearly 2-stage acc for 256x256** —— 用累加器（ACC）重叠的技巧，让 256×256 也近似获得"双缓冲/两段式 acc"的效果。
- 图描述：示意图标注 `512 columns`、`48 48 for SF`(?)（疑为 scale factor 的 48×48 分区）以及两块累加器 `ACC0`、`ACC1`；图上以颜色区分两块 ACC 在列方向上的错位搭接，说明"重叠"如何覆盖 512 列宽度。

### 第 7 页 | MegaMoE on Blackwell — Dynamic tileN with swapA/B

- 主旨：**Dynamic tileN with swapA/B**，即配合 swapAB 做动态 tile N。
- 要点：
- **Use swapAB to map N onto the token dimension to reduce tile quant(ization)** —— 用 swapAB 把 N 维映射到 token 维，以减少 tile 量化浪费（OCR 行尾为 `tile quant`，宾语缺失）。
- **Dynamically tile N to skip unnecessary loads & computation** —— 动态切分 N，跳过不必要的加载与计算。
- **Epilogue transpose** 子项：
- **Fuse TMEM 16dp and 32dp ops to transpose 16x32 → 32x16** —— 融合 TMEM 的 16dp 与 32dp 操作完成 16×32 → 32×16 的转置。
- **Human & LLM co-work to find the optimal TMEM solution** —— 人类与 LLM 协作寻找最优 TMEM 方案。
- 图描述：右侧为「**View of one CTA**」的 tile 布局示意，纵轴标 `Token` 与 `Weight`、横轴标 `tileN` 与 `tileM`，另有 `Valid tokens` 区与 `ACC` 块；图中用 `Valid tokens` 之外的空洞区域表示被动态 tileN 跳

过的无效计算。

### 第 8 页 | From a 32x4 Trick to a Full TMEM Transpose

- 标题即该页主张：从一个 **32×4** 的小技巧，扩展到完整的 **TMEM transpose**。
- 方法论副标：**Human hardware intuition × LLM-assisted mathematical modeling** —— 人的硬件直觉 与 LLM 辅助的数学建模 相乘。
- 左侧「**The observable starting point**」（可观测的起点）：
- **2x LDTM.16dp128b + STTM.32dp32b**
- **16×8 → 32×4** 的 copy-pattern trick（OCR 作 `16×8→32×4`；该数字组合与标题的 `32x4` 呼应，具体映射关系可能被 OCR 简化 **(?)**）。
- 右侧「**The modeling leap** — developed through iterative discussion with an LLM」：
- **Treat each LDTM / register permutation / STTM as a bit permutation over thread bits, register bits, TMEM-DP bits, and column bits** —— 把每条 LDTM、寄存器置换、STTM 都视为作用在 thread 位、register 位、TMEM-DP 位与 column 位上的**位级置换**。
- 顶部给出五步流程图：**Observe → Model with LLM → Search → Validate → Integrate**，各步下方标注：
- `Observe`： `16×8→32×4 copy-pattern trick`
- `Model with LLM`： `bit permutations + generator constraints`
- `Search`： `Cayley/BFS`
- `Validate`： `simulator replay` + `32×32 and 16x32→32×16`
- `Integrate`： `+ CuTe silicon kernels` + `+ FC1 epilogue`；并有 `overlap-safe in-place … component` (?) 的批注。
- **贡献切分**（页面下半部分两栏对照）：
- **What the LLM contributed**： `mathematical abstraction`，`search-state design`，`permutation generators`，`systematic exploration beyond what one developer could enumerate` —— 数学抽象、搜索状态设计、置换生成器，以及单个开发者无法穷举的系统性探索。
- **What the developer contributed**： `Hardware intuition`，`useful constraints`，`simulator/silicon validation`，`and the final overlap-aware kernel integration` —— 硬件直觉、有效约束、模拟器/硅验证，以及最终的"重叠安全"内核集成。
- 结论行：**Result: 16x32 → 32x16 in 4 rounds / 10 TMEM instructions within the searched generator model** —— 在搜索得到的生成器模型内，用 4 轮 / 10 条 TMEM 指令实现 16×32 → 32×16（OCR 作 `Rec t:`）。

### 第 9 页 | CuteDSL MegaMoE on Blackwell — MEMBAR Synchronization optimization: batch-commit

- 标题：**MEMBAR Synchronization optimization - batch-commit**。
- 机制：**L2 flag is used for pipeline synchronization** —— 用 L2 上的 flag 做流水线同步。

- 两步写 flag 的协议： 1. **MEMBAR.GPU**：确保此前所有 in-flight 的 GPU-scope 访存流量 **write flag** 都已到达 **PoC**（point of coherence，本例中即 **L2**）。 2. **RED.ADD flag**：对该 flag 执行 **atomic add 1**。
- 图描述：三行泳道时间轴 —— `Dispatch` 行（`Load token & store local` → `write flag`，重复两次）、`Computation` 行（`Act` → `STG` → `write flag`）、`Pipeline` 行（`wait flag` 之后进入 `Combine`），中间竖排标 `L2`，表示 flag 与数据都落在 L2 上，消费方靠 `wait flag` 等待。

### 第 10 页 | CuteDSL MegaMoE on Blackwell — MEMBAR batch-commit 的权衡

- 核心做法：**The batch-commit: merging multiple MEMBARs into one** —— 把多条 MEMBAR 合并成一条。
- 动机：**MEMBAR can stall requests** —— MEMBAR 会阻塞（stall）请求。
- 代价：**A tunable trade-off between fewer MEMBARs and higher latency** —— 在"更少 MEMBAR"与"更高延迟"之间做可调折衷。
- 两处具体合并粒度：
- **Dispatch to FC1: batch of 2 flag update** —— 每次批处理 2 个 flag 更新。
- **FC2 to Combine: batch of 4 flag update** —— 每次批处理 4 个 flag 更新。
- 图描述：两条时间轴分别对应上述两段。`Dispatch to FC1` 行是两次 `Load token & store local` 之后接一条 `MEMBAR`，再用两条 `REDG` 完成 2 次 flag 更新；`FC2 to Combine` 行是 4 组 `Act → STG` 之后接一条 `MEMBAR`，再由 `REDG ×4` 完成 4 次更新，消费侧 `wait flag` 后进入 `Combine`。可见 MEMBAR 数量被压到每批一条，而 flag 更新次数不变。

### 第 11 页 | CuteDSL MegaMoE on Blackwell — MEMBAR Synchronization optimization: async UREDGR

- 标题：**async UREDGR**（OCR 作 `aysnc UREDGR`）。
- 定义：**Async atomic add instruction with built-in MEMBAR** —— 内置 MEMBAR 的异步原子加指令。
- 对比：**MEMBAR is blocking**，而 **UREDGR is non-blocking**。
- 指令原文：`red.async.release.gpu.global.add.u32 [addr], b;`
- **Before**（改造前）：`Load token & store local` ×2 → `MEMBAR` → `REDG` ×2，计算侧 `wait flag`，中间标 `L2`。MEMBAR 显式阻塞。
- **After**（改造后）：`Load token & store local` 连续三次流水推进，计算侧 `wait flag`，用 **Async UREDGR** 替代显式 MEMBAR，屏障语义并入指令内部，不再单独阻塞。
- 图描述：Before/After 两条时间轴上下对照，After 一侧去掉了独立的 `MEMBAR` 块，`Dispatch` 行的加载动作因此可以更早连续发射。

### 第 12 页 | MegaMoE on Blackwell — Three modes for combine traffic（模式一）

- 标题：**Three modes for combine traffic** —— combine 流量的三种模式。本页讲第一种（OCR 作 `Model:`，按上下文与后两页的 `Mode 2 / Mode 3` 判断应为 **Mode 1**）。

- 方案：**direct store over NVLink in L2 epilogue（DeepGEMM solution, long time ago）** —— 在 L2 的 epilogue 里直接通过 NVLink 远端写，这是 DeepGEMM 很早的做法。
- 优点：**good for low latency combine; epilogue warps can send out tokens directly** —— 适合低延迟 combine，epilogue warp 可以直接把 token 发出去。
- 缺点：
- **Burst NVLink traffic makes fc2 epilogue longer** —— 突发式 NVLink 流量使 FC2 的 epilogue 变长。
- **epilogue becomes longer, blocking L1/L2** —— epilogue 变长后会阻塞 L1/L2。
- **the issue makes nvfp4 worse since it halves mainloop** —— 该问题在 NVFP4 下更严重，因为 NVFP4 使 mainloop 减半，epilogue 占比进一步升高。
- 图描述：时间轴分三行 `Dispatch / Computation / Epilogue`。Computation 行是 `L1 L2 L2 L2 L2` 与 `Act Act` 交替；Epilogue 行在每个 `Act Act` 之后出现成对的 `Remote Store`，右侧批注指明这是 NVLink 突发流量导致 epilogue 变长。

### 第 13 页 | MegaMoE on Blackwell — Three modes for combine traffic（模式二、模式三）

- 过渡页，先点出目标与障碍：
- **Decouple combine from epilogue warps** —— 把 combine 从 epilogue warp 中解耦。
- **Backpressure from NVLink still blocks the TMA LDG issue** —— 来自 NVLink 的反压仍会阻塞 TMA 的 LDG 发射。
- **Mode 2: reuse dispatch warp group (to smooth the combine traffic over kernel lifetime)** —— 复用 dispatch 的 warp group，把 combine 流量在整个 kernel 生命周期内摊平。
- **Mode 3(?)：standalone combine warp group (further overlap dispatch with combine)** —— 独立的 combine warp group，进一步让 dispatch 与 combine 重叠（OCR 该行行首被裁，按语义应为 Mode 3）。
- 图描述：两条时间轴。Mode 2 中 `Dispatch` 行先出现 `Dispatch`、后段出现 `Combine`（同一 warp group 前后复用），Computation 行为 `L2` 与 `Act Act STG STG` 交替，Epilogue 行不再有 Remote Store。Mode 3 在此基础上把 `Combine` 抬为独立的第三行，与 `Dispatch`、Computation 并行，形成三层重叠。

### 第 14 页 | MegaMoE on Blackwell — Phase interleaving between FC1 and FC2

- 标题：**Phase interleaving between FC1 and FC2**（OCR 作 `FCland FC2`）。
- 要点：
- **FC1/FC2 tile schedule matters** —— FC1 与 FC2 的 tile 调度顺序很关键。
- **Perfect overlap with L1 mainloop** —— 目标是让远端写与 L1 mainloop 完全重叠。
- **Prologue state: consume some L1 tiles to cutoff the L1/L2 dependency in steady state** —— prologue 阶段先消化掉若干 L1 tile，从而在 steady state 切断 L1 与 L2 之间的依赖（OCR 作 `cutoff theL1/L2 dependencyinsteadystate`）。

- 图描述：单条流水时间轴，`Dispatch` 行给出 `L1 L2 L2 L2`，其下 `Act Act Act` 之后出现 `Remote Store`（缩写 `RS`）与 `Act` 交替，右侧为 `RS RS`；图意是让 `RS` 落在 L1 计算之下被完全遮盖，从而不延长 epilogue。

### 第 15 页 | CuteDSL MegaMoE on Blackwell — Adaptive NVLink traffic control

- 标题：**Adaptive NVLink traffic control**（OCR 作 `Adaptive NvLiNK`）。
- 机制：**Measure each chunk's NVLink round trip, EMA (Exponential Moving Average) it, and nanosleep the deviation** —— 测量每个 chunk 的 NVLink 往返时间，用指数滑动平均（EMA）跟踪，并对偏差做 `nanosleep`（让出时间片）。
- 目的：**Reduce remote traffic head-of-line blocks** —— 减少远端流量的队头阻塞。
- 适用范围：**Used in mode 2 and 3 with combine that's not in epilogue warps** —— 用于 Mode 2 与 Mode 3，即 combine 不在 epilogue warp 里的情形。
- 图描述：上方是 `Combine` 的 chunk 序列 `chunk0, chunk1, chunk2, …, the last chunk`；下方标注两个时刻 `t0`、`t1` 及 `w = t1 - t0`(?)（即测得的往返/等待窗口），并给出 `TMA load: local → SMEM` 与 `bulk store peer` 两个动作，中间插入 `nanosleep`。图意是按实测往返偏差主动插入短暂休眠，避免远端流量扎堆造成队头阻塞。

### 第 16 页 | MegaMoE on Blackwell — Support for low-precision combine data types

- 标题：**Support for low-precision combine data types.**
- 主张：**MXFP8 and NVFP4 to reduce NVLink traffic** —— 用 MXFP8 与 NVFP4 降低 NVLink 流量。
- 精度格式对照表（表头为 `data format / scale / block`）：

| data format | scale | block | 说明 |
|---|---|---|---|
| `bf16` | `bf16` | （未标） | baseline；**the only format the in-kernel REDG path can atomic-add** —— 是内核内 REDG 路径唯一能直接原子加的格式 |
| `FP8 E4M3` | `E8M0` | `32` | `32 e4m3 × e8m0`，即 standard MXFP8 power-of-two scale（2 的幂次 scale） |
| `NVFP4 E2M1` | `bf16 amax` | `16` | `16 e2m1 × bf16`，full-precision amax per 16 elements —— 每 16 个元素一个全精度 amax，**recovers accuracy the E8M0 rounding loses**（挽回 E8M0 舍入造成的精度损失） |

- 前提条件：**Precision doesn't regress on some models** —— 在部分模型上精度不退。
- 工程代价：因为 bf16 是唯一可被内核内 REDG 原子加的格式，低精度 combine 需要 **A separate topK reduction kernel after megakernel** —— 在 megakernel 之后加一个独立的 topK 归约 kernel。
- 图描述：底部为两张并排的方块图，`Mega Kernel` → `TopK Reduce Kernel`，表示低精度 combine 被拆成"主内核 + 后置 topK 归约内核"两段流水。

- 页脚（OCR 残留 `ALDONOTDISTRIBUTE`）应为 `NVIDIA CONFIDENTIAL. DO NOT DISTRIBUTE` 一类保密声明。

### 第 17 页 | Overall Architecture: One Kernel Does Everything

- **总体架构：一个 kernel 包揽全部（One Kernel Does Everything）**，四个阶段在同一条链上：
- **Dispatch**：`4 dispatch warps` 通过 **NVSHMEM symmetric heap**，把 token 从 every rank 拉到本地 pool。
- **FC1**：pool 的 **FP8 view** 被直接当作 GEMM 的 activation 消费 —— **zero copy, zero intermediate kernels。**
- **FC2**：通过一个 **GMEM counter handshake**（即 `fc1_done_counter`）消费 FC1 的输出。
- **Combine**：FC2 结果写回 source rank；组合方式为 **three executors × two reduce placements = six modes。**
- **vs. baseline**：communication warps 与 GEMM warps **co-resident 且并发**；token 一到就算（tokens are computed as soon as they arrive）。
- 页脚总结句（OCR 残缺，`...es live in one persistent kernel`）：dispatch / GEMM / token-back warps 全部 co-resident 于**一个 persistent kernel** 内，token 到达即被计算。
- 图为一条从左到右的数据通路，OCR 中可见的标签依次为：
- 左端 `Peer ranks` → **NVSHMEM symmetric heap** / `NVLink`；
- 中间 `local FP8 pool (GMEM)`，旁注 `4 dispatch warps pull`、`zero-copy FP8 view`、`token_buffer`（OCR 作 `11_token_buffer`，前缀不清）**(?)**；
- 计算段 `FC1 WGMMA + SWiGLU + FP8 requant` → `fc1_output workspace (GMEM)` → `FC2 WGMMA`，段间以 `TMA store` 与 `fc1_done_counter handshake` 连接；
- 右端 `BF16 output`，回写段标 `Combine — writeback to source rank`，执行方式为 `epi-warp STG/REDG or bulk push`。
- 图意：通信侧（NVLink/NVSHMEM pull）与计算侧（FC1/FC2 WGMMA）在同一 kernel 内左右咬合，形成 拉取 → FC1 → 握手 → FC2 → 回写 的环流，中间无独立 kernel 边界。

### 第 18 页 | Warp Specialization

- 该页是一张 **warp 角色分工表**，表头为 `Role | Warps | setmaxnreg | Duty`（OCR 列名残缺，第 3 列数字与第 4 列职责有串行，以下按可辨读重建，(?) 标注不确定项）：

| Role | Warps | setmaxnreg | Duty |
|---|---|---|---|
| epi / WGMMA WG | 4×(1–2) | 200–256 | Issue WGMMA + SwiGLU / quant / store |
| TMA-A | 32**(?)** | —**(?)** | Load activation / `fc1_output`；FC2 的 `fc1_done` spin |
| TMA-B | 32**(?)** | —**(?)** | Load weights（independent of FC1，no wait on the counter） |
| sched | 40**(?)** | —**(?)** | Publish work tiles + expert prefetch |
| dispatch | 48**(?)** | —**(?)** | Pull tokens across ranks（**MegaMoE**） |

| Role | Warps | setmaxnreg | Duty |
|---|---|---|---|
| token-back | 4 (opt.) | 32**(?)** | Dedicated push of FC2 results back to the source rank |

> 注：`Warps` 列的 OCR 数字多与 `setmaxnreg` 列粘连（如 `32↓`、`40`、`48`），无法确认逐行取值；**总量以页面明写的 `20 warps / 640 threads` 为准**，各角色具体 warp 数仅作参考 **(?)**。 - **总量与配额**：`20 warps / 640 threads`；**setmaxnreg 按角色重新分配**（rebalanced per role），其中 **epi 拉满到 256**。 - **关键设计**：没有独立的 MMA warpgroup —— **由 epilogue WG 自己发 WGMMA**，accumulator 常驻 **RMEM**（`acc stage = 1`）。

## 第 19 页 | The swapAB Variant: the Cure for Small Token Counts

- **Problem**：`SM90 WGMMA` 是 **m64nNk32**，非 swapAB 下 token tile 最小就是 **64**；decode 类负载大部分 tile 都浪费在 padding 上。
- **swapAB 的做法**：`M = weight channels (128 / 256)`，`N = tokens (8 / 16 / 32 / 64 / 128)`。
- WGMMA 的 **N 维最小可到 8** → token tile 可以小到 **8**，**up to 8× better utilization**。
- **M 轴变成恒密的 channel 维**，因此即使 token 数极小也能填满 SM。
- **Implementation highlights**:
- swap 只是**纯 stride view，zero copy**；expert 维变成 GEMM-A 的 **L mode**，于是 **TMA descriptor 是 tile-invariant 的**。
- transpose 挪进 epilogue：FC1 走 **R2S SMEM addressing**；FC2 **丢掉 TMA store**，直接从寄存器发出 BF16（**STG**）。
- **8 行 gate/up interleave 正好落在同一线程的寄存器对上 → folding needs zero shuffles**。
- **与主 kernel 共用 MMA mainloop 与 warp topology → low maintenance cost**。
- 图为左右对照:
- 左 `non-swap: tokens on M-tile, M64×N128` —— `8 real tokens` 落在 M 方向，标注 **padding — 56 of 64 token rows wasted**。
- 右 `swapAB: tokens on N-tile, M128/256 × N8` —— `same 8 tokens` 落在 N 方向，标注 **N=8 tokens, all useful**。
- 中轴说明：`WGMMA m64nNk32: M is fixed at 64 per atom`，`N is flexible down to 8 — token tile 8/16/32/64/128`；`M = weight channels (always dense)`。
- 结论标签：**up to 8× better tile utilization**；OCR 残句 `...tches leave most of the tile empty`（指非 swapAB 的 tile 大量空置）**(?)**。

## 第 20 页 | FP8 Quantization: per_tensor vs. blockwise

- 该页是一张 **per_tensor 与 blockwise（DeepGEMM style）逐项对照表**:

| 项目 | per_tensor | blockwise (DeepGEMM style) |
|---|---|---|
| activation | scalar FP32 | **per-token × K128 block** |

| 项目 | per_tensor | blockwise (DeepGEMM style) |
|---|---|---|
| weight | per-expert scalar | **128×128 block** |
| FC2 activation | scalar（two-pass precompute） | **per-token × K64**（online in FC1 epilogue） |
| dequant | scalar multiply in the epilogue | **per-K-tile promote in the mainloop** |
| accumulation | 1×acc / 2×acc（FP32 promote per K tile） | **promotion built in** |

- **blockwise 的 online requant 链路**：register **absmax** → `shuffle_sync_bfly` **quad reduction** → **single-lane STG** 写出 scale → 再乘（乘回，OCR 行尾断在 `multiply by ...`，下一页补全为 reciprocal）。
- **blockwise half-K split**：scale load 与另一半的 WGMMA 相互重叠（scalar loads overlap the other half's WGMMA）。
- **零额外同步**：`SF shares the TMA stage barrier with A/B (merged tx_count)` —— 量化 scale 因子复用 A/B 的 TMA stage barrier，**zero extra synchronization**。

## 第 21 页 | Optimization 1: Cooperative Warp-group Scheduling

- **cooperative 模式下 warpgroup 数量随 tile 形状走**:
- `N=256`（swapAB `M=256`）：**两个 warpgroup 协作切同一 task tile** —— 各管一个 `N=128`（`M=128`）slice，各自发自己的 WGMMA、各自跑自己 slice 的 epilogue。
- `N=128`（swapAB `M=128`）：**单 warpgroup 精简配置（8 warps）**；epi 寄存器可拉满到 **256**。
- **Task boundary**：用 **CTA-level NamedBarrier**；所有 epi warps 一起进入下一个 task。
- **Characteristics & trade-offs**:
- WGMMA → epilogue 在单个 WG 内**串行**，会暴露 epilogue latency；但 **TMA warps 是独立的**，下一个 task 的 load 能继续 prefetch。
- **大 tile（N=256）摊销 A/B load、减少 task 数** → 更适合 GEMM-bound 的大 batch。
- 启发式表里仍有大量 bucket 由 cooperative 胜出（例如 **blockwise ≥512 一律用 (64,128)**；**per_tensor 16384 用 (64,256)**）。
- 图为 `One task tile: M64×N256 (swapAB: M256 × token N)` 的切分与流水:
- `WG0 owns N=0..127`、`WG1 owns N=128..255`，各自 `WGMMA + epilogue`。
- 时间轴：`WG0: WGMMA task0 (own half) → epilogue task0 → WGMMA task1 → epilogue task1`；`WG1` 同构并行。
- 每一 task 边界标 `CTA NamedBarrier` —— **all epi warps advance**。图意：两个 WG 在 N 方向对半分工，靠 CTA 级 barrier 对齐推进，没有跨 WG 的语义依赖。

## 第 22 页 | Optimization 2: Task-tile Ping-pong

- **两个物理 warpgroup 交替消费 task stream 中的 task**（ping-pong）。
- **两套 PipelineOrder barrier**:
- `math_wg_order` 串行化 **WGMMA issue 权**;

- `epi_wg_order` 串行化 **epilogue 所有权**。
- **重叠方式**：当 WG-A 占着 tensor core 时，WG-B 跑上一个 task 的 `SwiGLU / quant / store` —— **epilogue latency 被完全隐藏**。
- **被跳过的 peer task** 通过 `advance_pipeline_state` 推进，是**纯状态记账，不碰 mbarrier**。
- **Retire-handoff 沿用 CUTLASS 顺序**：tail async stores → publish done flag → hand over epilogue ownership → 然后 wait for the descriptor **two steps ahead**。
- 图为两条泳道的时间轴（`WG-A` / `WG-B`），横轴 `time →`：
- `WG-A: WGMMA task0 → epilogue task0 → WGMMA task2 → (wait) → epilogue task2`
- `WG-B: wait(math order) → WGMMA task1 → epilogue task1 → wait → WGMMA task3`
- 泳道之间标 `math order` / `epi order` 两列 barrier。
- 图注：**WGMMA never overlaps across the two WGs（tensor core 是独占的）**；**one WG's epilogue overlaps the other WG's WGMMA → epilogue latency is hidden**；补充适用配置 `(swapAB M=128)`。

### 第 23 页 | Optimization 3: Multi-CTA CGA + TMA Multicast

- **Cluster 形状**：`(2,1)` / `(1,2)` / `(2,2)`；cluster 内**每个 CTA 各自跑一个独立的 1-CTA WGMMA**。
- **多播方向**：**A 沿 cluster-N 多播，B 沿 cluster-M 多播** → 共享 operand 的 **L2/DRAM traffic 减半**（is halved）。
- **调度粒度**：scheduler 以 cluster 为粒度把 task tiles 成组（groups task tiles at cluster granularity）。
- **计数器隔离**：`fc1_done_counter` **每个 CTA 独立 slot** —— cluster peer 之间**没有 false-sharing 等待**。
- 图为 `CGA cluster 2×2 (also 2×1 / 1×2)` 的四象限布局：
- 上方源数据：`A tile (GMEM/L2)`、`B tile (GMEM/L2)`。
- 四角各一个 `1-CTA WGMMA`：`CTA(0,0)`、`CTA(0,1)`、`CTA(1,0)`、`CTA(1,1)`。
- 箭头分别为 `A multicast (cluster-N)` 与 `B multicast (cluster-M)`。
- 图注：一次 A load 服务沿 cluster-N 的 CTAs，一次 B load 服务沿 cluster-M 的 CTAs → 共享 operand 的 L2/DRAM 流量（减半）；而 `one_counter stays per-CTA (no false sharing)`。

### 第 24 页 | Optimization 4: Mainloop & Store Pipelines

**Mainloop** - **WGMMA software pipelining（`k_pipe_mmas=1`）**：稳态下**保持 1 个 WGMMA group in flight**，推迟 `consumer_release` —— **TMA refill 与 WGMMA 完全重叠**；`producer_acquire` 只回收上一组。 - **TMA producer `try_acquire` peek**：探测下一 stage 的可用性，**跳过大部分 mbarrier wait**。 - **A/B pipeline stage 数动态填满 228 KB SMEM**（`misc budget` 只占 **1 KB**）。 - **SMEM 复用**：FC1 的 **FP8 64×64** 与 FC2 的 **BF16 64×32** 每 stage **都正好 4 KiB** → 覆盖在同一份 **WG-private ring** 上。

**Store path** - **FC2 R2S + TMA store 两级流水**：每 WG 两个 **4 KiB SMEM buffer**。 - 当前 TMA S2G in flight 时，**下一个 subtile 的 R2S 写另一个 stage**。 - **Deferred drain**：保持一个 store group in flight，**只在 phase 边界 / kernel 尾部 / 需要可见性时**才 drain。 - **TMA store 只由 warp0 通过 `elect_one` 发**

**出**；**空 tile 直接跳过**（empty tiles are skipped outright）。 - 图为 epi WG 的 store 时间轴： - `R2S subtile0 → stage0` 、 `subtile1 → stage1` 、 `subtile2 → stage0` 、 `subtile3 → stage1` （ring depth = 2）。 - 下方 TMA 泳道： `TMA S2G stage0 (inflight)` 、 `stage1` 、 `stage0` 交替； `SMEM stage0 (4 KiB)` / `SMEM stage1 (4 KiB)` 。 - `producer_acquire` 只回收上一组的 store（ `ring depth 2` ），于是 **subtile i+1 的 R2S 与 subtile i 的 in-flight S2G 重叠**。 - 图注强调 **FC1/FC2 share the same WG-private ring (both 4 KiB)** ，以及 deferred drain 的三个 drain 时机。

## 第 25 页 | Optimization 5: All-Register Epilogue Fusion Chain

- 该页把 epilogue 做成一条**全寄存器融合链**，OCR 中可见的链路顺序为：

`WGMMA accumulator (FP32)` → `gate/up register deinterleave (interleave=8 → acc layout, zero shuffle)` → `dequant (scalar multiply, read once per task tile)` → `SwiGLU (exp2 fastmath + rcp_approx)` → `[optional: absorb top-k weights]` → `requant (precomputed reciprocal, multiply instead of divide)` → `R2S (single pass through SMEM)` → `TMA bulk store` → `fc1_output`

- 逐环节要点：
- **deinterleave**： `interleave=8` 与 acc layout 对齐 → **zero shuffle**（展开见下一页）；dequant 是 scalar multiply，**每 tile 只读一次**。
- **SwiGLU**：用 `exp2 fastmath + rcp_approx` —— **no slow exp/division**。
- **optional**：吸收 top-k 权重（**unlocks the atomic-add combine**）。
- **requant**：**提前算好的 reciprocal，用乘法代替除法**，再 requant 到 FP8。
- **同时挂在这条链上的 blockwise 处理**：per-token **amax** 用 **register accumulation + `shuffle_sync_bfly` quad reduction**，**不做 SMEM reduction**。
- **register pressure 的代价**： `M=128` **tile 因 spilling 被禁用**；swapAB blockwise 改为 **chunked**； `per-t_shape` 折叠成编译期常量。
- 图为 accumulator（ `RMEM (registers)` ）向下流经 `dequant → SwiGLU → requant to FP8 → SMEM → R2S → GMEM → TMA bulk store` 的竖向管线，侧栏标 `exp2 fastmath + rcp_approx` 、 `multiply by a precomputed reciprocal` 、 `optional: absorb top-k weight` 、 `unlocks the atomic-add combine` 。

## 第 26 页 | Why Gate/Up Interleave = 8: the WGMMA Register Layout

- 该页解释**为什么 gate/up interleave 必须取 8**，从 **Accumulator D fragment（m64nNk32）** 的寄存器排布出发：
- **M 方向**：每个 warp owns **16 rows**；每个 thread 持有 **row r 与 row r+8**。
- **N 方向**：寄存器按 **8 列一组**推进（**每 group 4 个 reg**）。
- **gate/up 在离线阶段按 8 交错**，两种布局都能**零 shuffle 折叠**：
- **non-swapAB（沿 N）**：gate 在 `regs 8k..8k+3` ，与之配对的 up 在 `8k+4..8k+7` 。
- **swapAB（沿 M）**： `row r = gate` ， `row r+8 = up` → 对应 `regs 0/1 vs 2/3` 。
- **为什么恰好是 8**：M 方向的行对间距恰好是 **8**（row-pair spacing）；**更小的粒度需要 shuffle，更大的粒度就要走 cross-warp SMEM**；因为 N 是 8 的倍数 → **8 是唯一可行的粒度**（8 is the only granularity），两种布局都能做到 **zero-SMEM folding**。

- 图为一张大表：行方向是 tile 行号（OCR 可辨 `2 4 6 7 8 10 11 12 13 14 15 ... N-8 N-7 N-6 N-5 N-4 N-3 N-2 N-1`，另有块头 `0 / 8 / 15 / 16 / 31 / 32 / 47 / 48 / 63`），单元格内是该行的 thread fragment 标签与它持有的元素对：
- 前半段 block 的 fragment 为 `T0..T3`、`T4..T7`、`T28..T31` 等，元素对写作 `(d0,d1)`、`(d2,d3)`、`(d4,d5)`、`(d6,d7)`，N 方向更远的 group 写作 `(dx,dY)`（列索引符号 OCR 不清）**(?)**；
- 后续 block 出现 `T32..T35`、`T60..T63`、`T64..T67`、`T92..T95`、`T124..T127` 等，行号跨度到 `63`，右下角标 `(tid % 128): fragments`。
- 图意：同一 tile 内 thread 与寄存器的映射随 N 方向每 8 列移交给另一组 4 个线程，**行 r / r+8 落在同一线程**，因此 gate（r）与 up（r+8）天然成对，这是 interleave=8 能零 shuffle、零 SMEM 的直接原因。
- ⚠️ 该页 OCR 表格**严重残缺与错乱**（数字与 `T*` 标签大量串行、空单元格丢字），上表结构为按可辨读片段重建，**具体 fragment 编号不保证逐格正确 (?)**。

## 第 27 页 | Optimization 6: A Single Active Dispatch Warp

- **active dispatch warp 默认为 1**（可配 `1 / 2 / 4`）：**4 个物理 warp 仍全部保留**，多余的跳过 dispatch，**只在 kernel tail 重新汇合**。
- **为什么更快**：`1 warp/SM`（约 **132 rows in flight**）**已经超过 NVLink bandwidth-delay product**；实测 **1 > 2 > 4 warps** —— 更深的 read queue 只会**拉长每一次 round trip**，并且**与 GEMM 抢 L2**。
- **Compact pull buffer**：SMEM slot 只按 active warp 分配，**每 CTA 省出 21 KiB** 还给 GEMM 的 A/B stages。
- 图为 `dispatch warpgroup (4 physical warps)` 的对比：
- **after**：`warp0 — ACTIVE`，流程 `prep · barrier · pull-token-back`（`NVSHMEM, over NVLink`）→ `SMEM pull slot`（**1 row in flight**）→ `local token pool (GMEM)`；`warp1 / warp2 / warp3 — idle: skips dispatch, rejoins at kernel tail`。
- **before**：`4 slots = 4 × hidden_bytes`；**after**：`1 slot`。
- 回收量：`bulk load 3 × 7168 B = 21 KiB per CTA`，**returned to the GEMM AB stages**；另有 `bulk store` 与 `peer rank rows` 两条通路。
- 图注（`4/EP8` 场景）：`1 > 2 > 4 warps` —— 每 GPU 约 **132 rows in flight** 已超过 NVLink BDP，更深的 queue 只是拉长每个 request 的 RTT，并和 GEMM 抢 L2 / issue slots。

## 第 28 页 | Optimization 7: Folding the Producer Warpgroup + Register Refit

- **整个 warpgroup（128 threads）消失**（folded away）。
- **Register refit**：一个 warpgroup 的 **4 个 warp 共享同一个** `setmaxnreg` **值** —— **producer 侧统一为 48，剩下的全给 epilogue：双 WG 时 224，单 WG 时 256。**
- **Effect**：
- **two-WG cooperative 从此可行**；
- **blockwise 512–32768 全部切换到** `M64N256`，收益 **+11% … +30%，geomean +15.8%。**

- **对照**：在旧布局下同一个 tile **慢 14%** —— 结论是 **"the layout, not register tuning, makes the two-WG mode usable"**（是布局而非寄存器调参让双 WG 模式变得可用）。
- 图为 before / after 两条 warp 槽位条：
- **before**：`epilogue/WGMMA WG ×1-2 | TMA-A | TMA-B | sched | idle | disp0 | disp1 | disp2 | disp3`，左侧另标 `producer warpgroup — 128 threads + registers`。
- **after**：`epilogue/WGMMA WG ×1-2 | disp0 | TMA-A | TMA-B | sched`，标 **128 threads per CTA**，且 `dispatch warpgroup now hosts the producers` —— 原本占位的 producer WG 被折叠进 dispatch warpgroup 的空闲 warp。
- 中间说明：`register cap = 65536 / threads rises` → **epilogue 拿到 224（two WGs）/ 256（one WG）**。
- 图注重申：4 个 warp 共享一个 `setmaxnreg`，producer 侧统一 48、余额给 epilogue —— **这才是 `two-WG cooperative M64N256` 可行的原因。**

## 第 29 页 | Optimization 8: Early fc1_done Publish

- **做法**：FC1 tile 的 **TMA store 一落地**，就由**一个 epilogue 线程立刻用 release atomic add 发布 `fc1_done`**，而不是等到 **task boundary 再批量发布。**
- **two-WG cooperative 下的阈值**：每个 WG 各 **+1**，FC2 的等待阈值因此是 **×2** —— **不需要 cross-WG barrier。**
- **Why it is faster**：发布点前移 → FC2 tile 更早通过 readiness 检查、更早发出 TMA load → **FC1→FC2 依赖链变短。**
- **代价**：**no extra warp / registers** —— 这是 **Mega default path**，同时作为下一页 store offload 的 **fallback。**
- 图为 epilogue 的 per-tile 时间轴（`before` / `after` 两行）：
- **before**：`R2S → TMA store → wait for next task → task-boundary barrier → batched publish fc1_done`。
- **after**：`R2S → TMA store lands → release-publish fc1_done → wait for next task → task-boundary barrier`。
- 图上标出两个不同的 `A FC2 tile unblocked here` 位置：after 行的解除阻塞点明显前移（早于 task-boundary barrier）。
- 图注：一个 epilogue 线程在 store 之后立即做 **GPU-scope release atomic add**；双 WG cooperative 时每个 WG 各 +1、FC2 等待阈值翻倍，**no cross-WG barrier needed。**

## 第 30 页 | Optimization 9: Blockwise FC2 Dual-Accumulator Overlap

- **背景**：blockwise FC2 会把**每个 K tile 按 activation scale 切成两半**；**旧代码严格串行**，中间要 `fence → wait → FP32 promote → re-issue`，**tensor core 在这段时间空转。**
- **新代码**：额外分配**第二个 accumulator register fragment**，把两半的 WGMMA **背靠背发出**；读完 weight scale 后**只等第一半**，在**第二半仍在 tensor core 上跑的时候**完成第一半的 promote，然后再等第二半。

- **Gating 条件**：**accumulator 尺寸 ≤ 64 registers**，且满足 **（single-WG standalone kernel）** 或 **（folded layout 且 epilogue registers ≥ 224）** —— 依赖上一页的 **register refit。**
- **关闭时退化**：禁用后指令序列**与旧版本完全一致**（pure refactor）；**per-tensor 路径与 FC1 mainloop 不变。**
- 图为同一个 K tile 的 before / after 时间轴：
- **before**：`WGMMA half A → wait → promote A (FP32 FMA) → WGMMA half B → wait → promote B (FP32 FMA)`，标注 **tensor cores idle during every wait + promote。**
- **after**：`WGMMA half A → acc1`、`WGMMA half B → acc2`、`wait A only · promote A`、`wait B · promote B`，标注 **half B 仍在 tensor core 上跑时 A 已 promote。**
- 图注：second accumulator register fragment；gated on `accumulator ≤ 64 regs` 且（single WG，或 folded layout 且 `≥ 224` epilogue regs）；禁用后指令序列与旧代码相同。

## 第 31 页 | Optimization 10: Scheduler Groups and the fc1_done Spin

- **调度方式**：scheduler **按 group 派发** —— 先发**一组 experts 的全部 FC1 tile**，再发它们的 FC2 tile；**默认 group = one wave。**
- **小 bucket 的问题**：此时每个 expert 只有 **1 个 token tile，但有 48 个 FC1 tile**；一个 group 只装得下 **1–3 个 expert**，于是 **FC2 tile 紧跟自己的 FC1 tile，而那些 FC1 还在跑。**
- **后果**：activation TMA producer **约 25% 的时间在 `fc1_done` 上空转 spin**（OCR 此处带一处不可辨标注 `(IKET)`）**(?)。**
- **解法**：**把 group 放大到 264 ≈ 6 experts** —— FC2 tile 到达时 `fc1_done` **已经完成，spin 消失。**
- ⚠️ **该页识别内容严重不足**：除上述要点外，图部分几乎全为乱码（OCR 原文如 `Gepechoner(onegrowp=allengensFCities,then hsinFC2 liles)-wall ocket...`，以及 `FCi mlee envetO(e) / FOM.e91 / F02.801 / FOA/EPO / FO2/S01` 等碎片），可推测是对比「one group = 所有 experts 的 FC1 tile，再是其 FC2 tile」前后 wall clock 的条形图，**具体轴标签与数值不可辨，不予臆测。**

## 第 32 页 | Optimization 11: 16-byte FC2 Output Stores

- **改造前**：FC2 的 **BF16 输出按 accumulator layout 逐对写出** —— 每个 lane 每个 subtile **8 ×** `STG.32`（swapAB 为 **8 ×** `STG.16`），**一次 warp-level store 只填半个 sector。**
- **non-swap 的改造**：在 quad 内做 **4×4 butterfly transpose（4 次 shuffle）** → 每个 lane 持有**某一行的 8 个连续列 → 一条** `STG.128`；**cross-rank 直写的目的地查找从 8 次降到 2 次。**
- **swapAB 的改造**：accumulator 是 **channel-major**，改用**三级 lane-group transpose（6 次 shuffle + 6 次** `prmt`**）**替代 7 条 STG；**仅 local-pool 路径**适用。
- **净效果**：**one warp-level store：8 half sectors → 16 full sectors。**
- 图为寄存器布局到 store 形态的变换（左右两栏）：
- 左（before，`WGMMA accumulator layout, one row r`，**每 8 列组每 lane 持 2 个相邻 BF16**）：`lane0: c0c1 | c8c9 | c16c17 | c24c25`、`lane1: c2c3 | c10c11 | c18c19 | c26c27`、`lane2: c4c5 | c12c13`

`| c20c21 | c28c29`、`lane3: c6c7 | c14c15 | c22c23 | c30c31`，标注 **8 × `STG.32` per lane per subtile / half-sector writes**。

- 中：`4×4 butterfly transpose — 4× shuffle (xor1, xor2)`。
- 右（after）：`lane0: cols c0..c7 (16B)`、`lane1: cols c8..c15`、`lane2: cols c16..c23`、`lane3: cols c24..c31`，标注 **1 × `STG.128` per lane per row**。
- 结论行：**after 每 lane 拿 8 个连续列 → 一条 16 字节 store；一个 warp 填满 16 个完整 32B sector，而不再是 8 个半 sector**；同时把 cross-rank 目的地查找从 **8 次降到 2 次/subtile**。
- 页脚：swapAB（channel-major accumulator）走三级 lane-group transpose，`6 shuffle + 6 prmt → 1 STG.128`，**local-pool 路径专用**；store 原语**复用自 training-output（`generate_c`）路径，no new PTX**；**对齐依赖 hidden 是 8 的倍数**。

## 第 33 页 | Token Sweep Results

**图表**：4 组/两簇柱状图为 token sweep 的耗时对比，横轴 `tokens per rank`（8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768，共 13 个 bucket），纵轴为耗时 `ms (slowest rank, useful FLOPs)`(?)（刻度 200/400/600/800/1000/1200）。两组配色分别对应 `Per-Tensor FP8` 与 `Blockwise FP8 (DeepGEMM-style)`；每组柱子上方标注 `MegaMoE ÷ baseline` 的加速倍数。图中可见两组柱子在小 token 数时高度接近，随 token 数增大差距拉开。

- 实验设定：`4x H200`、`EP4`、`DeepSeek-V4-Pro shape`、`full MoE-layer forward`，统计口径为 `p50 over the slowest rank`。
- 图注定义：每组上方数字 = speedup（`MegaMoE ÷ baseline`）。
- Baseline = 常规四段式流程：`NCCL all-gather / reduce-scatter` + `CUTLASS grouped GEMM`（fused finalize）。
- 对照实现：`MegaMoE` = fused single kernel（`NVSHMEM` dispatch/combine）。
- 关键结论（讲者原话）：Per-Tensor 配置下 `MegaMoE` 在全部 13 个 bucket 上领先 —— 512 tokens 及以下为 **1.02–1.11x**，1024 起为 **1.24–1.35x**（峰值 **1.35x** 出现在 8192），geomean **1.16x**。
- 解释：小 token 数时两者接近 —— launch / communication 主导；一旦 GEMM 占比上升，单内核方案的优势才显现出来。
- 图中标注的其余倍数（OCR 可读）：1.33x、1.24x、1.72x、1.35x、1.59x、1.51x、1.29x、1.27x、1.45x、1.37x、1.11x、1.31x、1.09x、1.22x、1.06x、1.05x、1.03x、1.02x、1.18x、1.16x。其中大于 1.35x 的数值（如 1.72x、1.59x、1.51x）应属于 Blockwise FP8 一组（由 Per-Tensor 组 ≤1.35x 的结论反推）(?)。
- 页脚可见 `NVIDIA` logo。

## 第 34 页 | Summary

**What was built**（这一页是 MegaMoE 部分的总结清单）：

- **One persistent kernel**，融合 `dispatch + FC1/SwiGLU + FC2 + combine`（共 **six modes**）。
- **Two complementary kernel variants**：`non-swap` / `swapAB`；并配 **two quantization modes**：`per_tensor` / `blockwise`。

- **30+ kernel / communication / scheduling optimizations**，并用 **token-bucket heuristics** 把 sweep 出来的 winner 冻结下来（freezing the sweep winners）。
- 页面上重复出现多枚 NVIDIA logo（版式元素）。

## 6.2 第二部分 • 大规模训练优化 playbook（第 35–52 页）

视角从单个内核抬到集群。第 35–38 页先立问题（装得下 / 跑得快）并说明模型特征如何驱动优化，第 39–43 页确定起点（并行布局、显存策略、代理模型），第 44–51 页按 STEP 1→4 逐步落地，第 52 页把经验收成可复用配方。

### 第 35 页 | Two Challenges Define Training at Scale

**结构**：左右两栏并置 —— 左侧「Make it fit」，右侧「Make it fast」，中间标注本讲的取材来源。

- 左侧 **Make it fit**（装得下）：
- `Massive parameter counts`（超大参数量）
- `Model states and HBM capacity`（模型状态与 HBM 容量）
- `Long context windows`（长上下文窗口）
- `Parallel layout and topology`（并行切分布局与拓扑）
- `Feature compatibility`（特性兼容性）
- 右侧 **Make it fast**（跑得快）：
- `Kernel and launch overhead`（内核与 launch 开销）
- `Recompute and offload costs`（重算与 offload 代价）
- `Communication and load imbalance`（通信与负载不均衡）
- 取材说明（This talk is based on）：
- `Public DeepSeek-V4-Pro model architecture`（公开的 DeepSeek-V4-Pro 模型架构）
- `Implementations in Megatron Core`
- `NVIDIA Blackwell GPUs`

### 第 36 页 | One Playbook, Two Workloads

**结构**：左栏列出统一的优化 playbook 四步，右栏给出这套 playbook 实测的吞吐收益。

- **One Optimization Playbook**（四步）： 1. `Runtime Overhead and Memory Trade-offs` 2. `Attention Compute Efficiency` 3. `Expert Execution and Memory Footprint` 4. `Context-Parallel Load Balance`
- **Measured throughput gains from the playbook**:
- **+59%** at **64k** context length
- **+32%** at **1M** context length
- 标题点题：同一套 playbook，作用于两种不同 workload（64K 与 1M 上下文）。

### 第 37 页 | DeepSeek-V4-Pro Overview

**图**：DeepSeek-V4-Pro 的整体模型结构图（左侧为从 `Input Tokens` 向上的堆叠，右侧为两个关键规格数字与预训练 recipe）。图中元素与标签：

- 自下而上的层级标签：`Input Tokens` → `Embedding` → `Pre-Block Mixing` → `Residual Mixing` → `CSA/HCA` → `DeepSeekMoE` → `Post-Block Mixing` → `Transformer Block x L` → `Prediction Head` → `MTP Modules`（并标注 `MTP Loss`）。
- 右侧规格：`1.6T` **Total parameters**；`49B` **Activated parameters per token**。
- 右侧 **Pre-training recipe**：横轴为上下文长度 `4K → 16K → 64K → 1M`，对应数据形态 `dense → dense → dense/sparse → sparse`。
- 出处注：`The model graph is from DeepSeek-V4 tech report.`

### 第 38 页 | Model Features Drive Optimizations

**图**：与第 37 页同一张模型结构图，但本页在右侧用四条编号说明「哪些模型特征驱动了哪些优化」。图内标签同上（`Input Tokens`、`Embedding`、`Pre-Block Mixing`、`Residual Mixing`、`Indexer`、`CSA/HCA`、`DeepSeekMoE`、`Post-Block Mixing`、`Transformer Block x L`、`Prediction Head`、`MTP Modules` / `MTP Loss`）。注意本页 OCR 中比第 37 页多出 `Indexer` 一项（第 37 页 OCR 未识别到）。

- 规格：`1.6T` **total /** `49B` **active**。
- 四条特征 → 优化映射： 1. **Model states limit memory headroom for faster execution.**（模型状态挤占显存余量，限制了更快的执行方式） 2. **Hybrid attention**：different attention paths require tailored kernel optimizations.（不同注意力路径需要定制内核优化） 3. **Fine-grained MoE**：expert compute, communication, and buffers drive performance.（专家计算、通信与 buffer 决定性能） 4. **Causal long context**：equal token counts can still m[ean] unequal workloads.（同样 token 数也可能意味着不等的实际工作量）
- 出处注同上：`The model graph is from DeepSeek-V4 tech report.`

### 第 39 页 | Choose the Parallel Layout（General Guidance on Parallel Tuning）

- **1. Model size guides the initial layout.**
- 可以估算 static part budget，据此选一个初始 layout。
- 为 activations 与 buffers **预留 headroom**。
- 专家优先用 `EP`，跨 domain 用 `PP`，必要时再加 `TP`。
- **2. Context length guides CP.**（本页 OCR 为 `Cp`）
- 上下文长度影响 activation 与 buffer 的大小。
- **3. Topology guides placement.**
- 通信密集的组（如 `EP`、`TP`、`CP`）尽量收在 **NVLink domain** 内，以获得更好的吞吐。
- 实例：**256-GPU 的 DeepSeek-V4-Pro run，初始布局用** `PP4` **与** `EP64`。

### 第 40 页 | Choose the Parallel Layout（同页重拍）

与第 39 页为**同一张投影片**（本条 OCR 更完整，`CP` 未误识为 `Cp`），要点完全一致，此处仅列差异与完整句：

- **1. Model size guides the initial layout.** We can estimate the static part budget and choose an initial layout. Reserve headroom for activations and buffers. Prefer EP for experts, use PP across domains, and add TP if necessary.
- **2. Context length guides CP.** Context length affects the activation and buffer size.
- **3. Topology guides placement.** Keep communication-heavy groups (such as EP, TP, and CP) within NVLink domains where practical for better throughput.
- **For our 256-GPU DeepSeek-V4-Pro runs, we use `PP4` and `EP64` as the initial layout.**

### 第 41 页 | Context Length Drives Memory Policy

**结构**：左右对照两种上下文长度的显存策略。核心命题：`More tokens per rank increase activation and buffer memory.`

| | 64K Context | 1M Context |
|---|---|---|
| 并行度 | `CP16` | `CP64` |
| 每 rank token 数 | `4,096 tokens/rank` | `16,384 tokens/rank` |
| 策略 | Fine-grained recompute + offload | Full recompute |
| 效果 | More headroom for CUDA Graphs | Higher activation pressure |

- 结论句：**1M 时每个 rank 的 token 数是 4 倍，因此切换到更激进的显存策略（more aggressive memory policy）。**

### 第 42 页 | Context Length Drives Memory Policy（同页重拍）

与第 41 页为**同一张投影片**，本条 OCR 更完整（`1M Context`、`16,384 tokens/rank` 数字正确）；原文要点：

- `More tokens per rank increase activation and buffer memory.`
- `64K Context` / `CP16` / `4,096 tokens / rank` → **Fine-grained recompute + offload**；**More headroom for CUDA Graphs**
- `1M Context` / `CP64` / `16,384 tokens / rank` → **Full recompute**；**Higher activation pressure**
- `4x more tokens per rank at 1M, so we switch to a more aggressive memory policy.`

### 第 43 页 | The Starting Points

- 用 **4K-context 的 sparse proxy model** 来基本验证所需特性，包括：
- `Kernel fusions`、`packed sequence support`、`CUDA Graphs support`, and others
- 用 profile 来指导后续优化；两段 profile **都在 150ms 窗口内**（OCR 为 `15oms`，应为 150ms）(?)
- 图：上下两张 Nsight 式 timeline 截图，分别标注 `64K` 与 `1M`；行标签为 `CUDA HW (0008:06:00.0 - N…)`、`Kernel`、`Memory`、`[AllStreams]`；可见内核名形如 `nvjet_sm103_tst_…`、`nvjet_sm103_tss_…`（`sm103` 指向 Blackwell 一代的 CUTLASS 内核命名）。
- 该页文字偏少，主要是两张 timeline 截图作对照（未给出具体数值）。

## 第 44 页 | STEP 1: Runtime Overhead and Memory Trade-offs

- 判断：**64K 场景下的主要问题是大量 GPU idle（Lots of GPU idles）**。两种缓解手段：
- **Fusion kernels**
- 有些算子容易融合，例如 simple mathematics 与 memory movements。
- **CUDA Graphs**
- 把 static modules 捕获后 replay；**用 padding 把 packed sequences 变成 static**（`we use padding to make packed sequences static!`）。
- 图：上下两条 kernel timeline 对比 —— `w/o CUDA Graphs` 与 `w/ CUDA Graphs`；图中为密集的小 kernel 块（OCR 只见 `kern…`、`void`、`kernel`、`kerne…` 等碎片标签），示意启用 CUDA Graphs 后 kernel 发射的空隙被压缩。
- 该页数值信息极少（无具体百分比）。

## 第 45 页 | Where the Step Time Goes（Layer time decomposition）

**图**：堆叠柱状图，横轴 `Context Length` = `4K` / `64K` / `1M`，纵轴 `Percent %`（OCR 刻度数字残缺，仅见 `100`、`200`(?) 等碎片），每根柱子按模块分段。

- 图例（分段含义）：`MoE experts`、`EP comms`、`Indexer + CP comm`、`Sparse Attn`、`Group-O Proj`、`Others`；柱内另有 `Attn` / `MoE` 大类标注。
- 注：`4K is a sparse proxy case for comparison`（4K 仅作 sparse proxy 对照）。
- 讲者意图：拆解 layer 时间构成，为后面 STEP 2（注意力）与 STEP 3（MoE）的优化定位瓶颈。
- 该页 OCR 未给出各分段的具体百分比数值。

## 第 46 页 | STEP 2: Optimize the Hybrid Attention

**Optimize Three Important Modules in the DeepSeek-V4 Hybrid Attention** —— 三个模块各一组 BEFORE / AFTER 对照：

### 1. Sparse Attention

- 手法：`Left-align the valid indices`；kernel 跳过 invalid slots。
- **BEFORE**：`SCAN EVERY SLOT`（逐 slot 扫描）；图注 `One query row of KV indices — -1 is invalid`。
- **AFTER**：`SCAN THE FIRST L ONLY`；`valid prefix L`、其余 `ignored`；`topk_length = L` **— no holes left to walk over**。

### 2. Indexer

- 手法：`Skip the invalid part from causal mask`；对重头的 QK matmul **用低精度**（`Use low-precision on the heavy QK matmul`）。
- **BEFORE**：`MATERIALIZE, THEN SELECT` —— 先物化 `Full score matrix S`（图中标注 `(S/4)` mostly unused(?)），`second pass`，QK 用 **BF16**。
- **AFTER**：`SCORE AND SELECT IN ONE KERNEL` —— `Q·K quantized to FP8`，`Candidates + top-k` 一步出，`Only the selected scores are ever written out`。

- 图中另有两处 `4Q` / `40` (?) 的残缺标注（疑为 `4Q` 或 mask 比例），无法确认。

### 3. Group-O Projection

- 手法：`Fuse into a batched GEMM`； `Enable FP8 computation`。
- **BEFORE**：`SEPARATE LAUNCHES` —— 输入 `Attention output [S, B, heads × dim]`， `split into G groups`（图中画出 `G0, G1 … G7`，即 `G=8`）； **`G=8 small GEMMs, one kernel launch each`**。
- **AFTER**：`ONE BATCHED GEMM` —— `All G groups multiplied together — one kernel`；再 `Project back to hidden`。

## 第 47 页 | STEP 2: Optimize the Hybrid Attention（效果页）

- **Speedup (module-level)**（模块级加速，相对 STEP 1 之后的 config）：
- `Compact sparse attention`：**25%–30%**
- `Compact FP8 Indexer`：**25%–35%**
- `Group-O batched GEMM`：**50%–60%**
- **Throughput uplift from STEP 2**（端到端吞吐提升）：
- **+4.6%** at **64K** context length
- **+9.0%** at **1M** context length
- 解释：`These gains come from faster execution.`（这些收益来自更快的执行）
- 附加观察：`The indexer accounts for a larger share of step time at 1M.`（1M 下 indexer 占 step time 的比例更大）——这与 1M 的端到端收益（+9.0%）高于 64K（+4.6%）相呼应。

## 第 48 页 | STEP 3: Optimize the MoE

- 观察：**MoE 部分中约 30%–50% 是 EP communication**；目标是 `hide the communication behind computation`（把通信藏到计算后面）。
- **Two solutions now**： 1. **Cross-batch EP overlapping**
  - 更适合 **cross-NVLink domain** 场景
  - block size 由从 **nsys report** 中提取的 silicon time(?) 决定（`The block sizes are determined by the silicon time extracted from nsys report.`）
  - 代价：**Increase memory overhead**（增加显存开销） 2. **Fine-grained overlapping with Megakernel**
  - 在 NVLink domain 内相对更容易融合
  - 更好地利用显存（`Better utilize the memory`）
- 图（右下）：overlap 示意 —— `Overlapped` 区间内 `Computation` 为 `Expert FFNs1` / `Expert FFNs2`，`Communication` 为 `Dispatch1` / `Dispatch2` / `Combine1` / `Combine2`，横轴 `Time`；左上另有 `layer1_f` / `layer0_b` 之类的层命名碎片（`Ifib` (?)、 `Ifib with loss` (?) 等 OCR 残字，含义无法确认）。
- 出处注：`The graph below is from the MoK(?) blog from Cursor.`（`MoK` 疑为 `MoE`，待核）

## 第 49 页 | STEP 3: Optimize the MoE

- 标题：**STEP 3: Optimize the MoE。**涉及对象：**Mixture-of-Kittens (MoK) from Cursor**（OCR 原文如此，`Kittens` 一词存疑 **(?)**）。
- 三条优化手段，逐条列出： 1. **Fused execution** —— *Combine expert computation and communication in one kernel.*（把专家计算与通信合并进同一个 kernel。） 2. **Communication overlap** —— *Pipeline token transfers with expert computation.*（token 传输与专家计算流水重叠。）→ **+14.0% at 64K context length** 3. **Buffer reuse** —— *Reuse fixed-size ring buffers to reduce memory usage and avoid CPU synchronization for buffer allocation.*（复用固定大小的 ring buffer，降低显存占用并避免缓冲区分配带来的 CPU 同步。）→ **+4.6% at 1M context length**
- 结论句：*MoK can bring 30%–40% speedup on the MoE part in the DeepSeek-V4-Pro setting.*
- 图元素：右侧为「Throughput uplift from STEP 3（compared with the config after STEP 2）」的柱状/条形标注，两项数值即 **+14.0%（64K）** 与 **+4.6%（1M）**；趋势是长上下文（1M）下收益明显小于 64K。
- 该页是「优化清单 + 收益数字」型投影片，无曲线图。

### 第 50 页 | STEP 4: Balance CP Workloads

- 标题：**STEP 4: Balance CP Workloads。**
- 核心论断：**Equal tokens per rank does not mean equal work on each rank.**（每个 rank 分到等量 token，并不等于每个 rank 承担等量工作。）
- 图元素：柱状图，图注为 **CSA and Indexer time on 512K context size with CP32**；纵轴 **GPU time (ms)**，刻度 10 / 20 / 30 / 40 / 50 / 60；横轴为 **CP rank**；每根柱按两个分量堆叠，图例为 **Indexer score forward** 与 **CSA forward。**
- 柱顶数据（OCR 保留的 9 个数值）：**53.1、49.9、45.3、40.9、36.6、31.9、27.3、23.1、18.4。**
- 图描述：柱高自 53.1 单调递减到 18.4，说明同一 CP32 组内各 rank 的总 GPU 时间相差近 3 倍，负载极不均衡；堆叠关系表明 `Indexer score forward` 与 `CSA forward` 两部分共同构成该时间。
- 不确定项：横轴 CP rank 只留下 `12 / 16 / 20 / 24 / 28 / 31` 等刻度，与 9 个柱值数量不匹配，疑有刻度识别丢失 **(?)。**

### 第 51 页 | STEP 4 II：用 zigzag 布局均衡 CP 负载

- 标题（OCR 作 `STEP4I`）：**STEP 4 II: Balance CP Workloads。**
- 复述论断：*Equal tokens per rank does not mean equal work on each rank.*
- **Why?** —— *QK score computation scales quadratically with sequence length.*（QK score 的计算量随序列长度二次增长。）
- **How?** —— *Balance the workloads with zigzag layout.*（用 zigzag 布局均衡负载。）
- 收益：**12.2% at 1M context length**（图注为「Throughput uplift from STEP 4，compared with the config after STEP 3」）。
- 图元素（负载重分配示意，两行 rank 块）：
- 布局前：`R0 R1 R2 R3` 对应 `lightest / hea(v)ier / heavier / heaviest`，即 **R3 becomes the straggler and the others wait**（R3 成为掉队者，其余 rank 空等）。

- 布局后（标注 `AFTER 2xCP SEGMENTS, PAIRED DIAGONALLY`）：上一行 `R3 R2 R1 R0`、下一行 `R0 R1 R2 R3`，即 **R0 takes the lightest and the heaviest**（R0 同时承担最轻与最重的段），以此拉平各 rank 总工时。
- 代价：*HOWEVER: the balance process also introduces extra overheads.* → **NO measurable perf gain at 64K context length**（64K 下无可测量收益）。
- 页脚残留 `DIA` 疑为 `DIAGONAL` 的截断 **(?)**。

### 第 52 页 | Lessons for Scalable Training（KEY TAKEAWAY）

- 标题：**Lessons for Scalable Training**，标签 **KEY TAKEAWAY**。
- 三条结论：
- **Parallelism and memory policy jointly determine feasibility.**（并行策略与显存策略共同决定可行性。）
- **Use profiling to guide the optimization.**（用 profiling 指导优化。）
- **Optimization priorities depend on the target workload.**（优化优先级取决于目标负载。）
- 右半区：**Get Started with Megatron Core** —— **Reference recipes for DeepSeek-V4-Pro**；三条卖点：
- *Composable parallelism for scalable dense and MoE training.*
- *FP8, fused kernels, and communication overlap optimized for NVIDIA GPUs.*
- *Modular building blocks for custom models and training workflows.*
- 链接：`https://github.com/NVIDIA/Megatron-LM`；配方路径 OCR 作 `Megatron-LM/examples/moe recipes/deepseek_pro/gb300/`（`examples` 与 `moe_recipes` 的目录名拼接不清）**(?)**。
- 配置文件名（OCR 严重残缺，仅能读出片段，标注不确定）：
- `DmSL4K266OPU_TP1PP4EAyam` → 疑似 **…64K…256GPU_TP1_PP4….yaml** 一类的 64K 配方 **(?)**
- `mTHO1_256OPUTPIPPP64CP84yal` → 疑似含 **256GPU / TP1 / PP? / CP64** 的 1M 配方 **(?)**
- `mTH045CCPU_TP1PECP16ym` → 疑似含 **TP1 / PP? / CP16** 的配方 **(?)**
- 另有 `[Dev] Updte c(NVI335)`、`Tools bench 1379…` 等行基本不可辨 **(?)**。
- 该页为「结论 + 资源入口」型投影片，无图表。

## 6.3 第三部分 • KV Cache 压缩：TurboQuant / KVTC / nvCOMP-ANS（第 53–67 页）

推理侧的“显存篇”。第 53–56 页说明为什么 KV Cache 是服务瓶颈、压缩在系统中的位置与代价，第 57–60 页是在线量化（TurboQuant），第 61–64 页是存储/传输压缩（KVTC），第 65–66 页是无损熵编码（nvCOMP-ANS），第 67 页给出三条路线的取舍要点。

### 第 53 页 | Motivation: KV Cache as a Serving Bottleneck（议程页）

- 标题：**Motivation: KV Cache as a Serving Bottleneck**。

- 本页是后续整段的 **Agenda**，四节： 1. **Motivation: KV Cache as a Serving Bottleneck** 2. **Taxonomy: Inference-time KV Cache Compression Patterns** 3. **Performance Analysis: TurboQuant & KVTC & nvCOMP** 4. **Takeaways**
- 该页为目录页，仅有章节标题，无数据；文本中未出现图表元素。

### 第 54 页 | KV Cache Is a Serving Bottleneck

- 标题：**KV Cache Is a Serving Bottleneck**；首句 **KV memory grows linearly with context length**（KV 显存随上下文长度线性增长）。
- **THREE BOTTLENECKS**（三重瓶颈）：
- **GPU capacity**：*Fewer active requests fit* → **active batch ↓**、**max context ↓**
- **HBM bandwidth**：*Every token rereads history* → **more HBM bytes, bandwidth-bound**
- **System I/O**：*When capacity spills, KV moves GPU ↔ CPU / SSD / remote* → **TPOT ↑**、**prefill / decode not work**(?)
- **SYSTEM TRADE-OFF**（系统级权衡），三条互相牵制的目标：
- **Maintain concurrency** —— 固定 GPU budget 下，要维持并发就得 **more GPUs • cost ↑**，否则 **active requests ↓**
- **Maintain TTFT / TPOT** —— 代价是 **offload / re-prefill ↑**
- **Optimize the global serving trade-off**
- 图描述：整页为「三列瓶颈 → 一栏 trade-off」的矩阵式版面，三列各自向下给出因果链，底部汇到全局权衡；OCR 未含数字，只有方向性符号（↑ / ↓）。

### 第 55 页 | KV Cache Compression Methods: training-time & inference-time

- 标题：**KV Cache Compression Methods**，副题 **Training-time & inference-time**。
- 分成两大类：
- **TRAINING-TIME — Architecture design**（架构设计，示例为模型结构本身）：
  - `MQA · GQA`
  - `MLA · CLA`（`CLA` 拼写存疑 **(?)**）
  - `Kimi Linear · CSA/HCA · MSA`
- **INFERENCE-TIME — Runtime compression**（运行时压缩），三个子类：
  - **Quantization**：*Fewer bits per KV element*（FP16 → low-bit），手段为 **Scales + protected outliers**（缩放因子 + 保护离群值）
  - **Storage sparsity**：*Retain fewer KV entries*，手段为 **Prune / evict KVs**（剪枝 / 淘汰 KV）
  - **Compute sparsity**：*Read fewer KV entries or pages*，手段为 **Query-aware top-K reads for attention**（查询感知的 top-K 读取）
- 版面内含 `FP16 | Low-bit`、`full cache`、`retained`、`full cache stays resident` 等对比标签，说明低比特方案保持全量 cache 常驻，只是每元素位宽下降。
- **SERVING IMPACT**（指标定义，页脚）：

- **MEM = footprint**（显存占用）
- **MEM READ = KV-cache bytes / decode**（每步 decode 读取的 KV 字节数）
- **ATTN = attention**(?)（OCR 截断）
- 三类压缩均标 **MEM↓**，注意力开销行标 **ATTN↓**；某行 OCR 作 `ATTN=(+DQ)`，疑为 `ATTN = f(DQ)` 一类函数写法 **(?)**。
- 残缺项：表内 `161616`、`MEMI / MEMI / MEM=` 等字符为表格分隔线与角标的误识 **(?)**。

### 第 56 页 | Where KV Compression Fits（两种集成路径）

- 标题：**Where KV Compression Fits**，副题 **Online attention path or KV-cache backend**。
- **REUSABLE MODULE**：`KV-cache compression` —— 三个实现：**TurboQuant**、**KVTC**、**nvCOMP-ANS**。
- **TWO INTEGRATION PATHS**（两条集成路径）：
- 路径一：**Inference engine（vLLM / SGLang / TRT-LLM） → Online path, e.g. INT8 • TurboQuant**
  - *Compressed KV remains GPU-resident.*（压缩后的 KV 常驻 GPU）
  - 收益：**Capacity ↑**、**KV bytes per decode step ↓**、对推理引擎**透明（Transparent）**，等效于 **more effective KV batch / context / throughput**
  - **CHALLENGE**：**attention fusion • engine & model-specific integration**（需与 attention 融合，且与引擎、模型强绑定）
- 路径二：**KV-cache backend（LMCache / FlexKV / Mooncake） → Offload path, e.g. KVTC • nvCOMP-ANS**
  - *Restore to standard KV before attention.*（注意力前先还原成标准 KV）
  - 收益：**Storage / Transfer ↓**、**Capacity ↑**（CPU / SSD / remote），并带来 **prefix-hit potential**
  - **CONS**：**decompression overhead • no direct TPOT gain**（解压开销，且不直接改善 TPOT）
- **INTEGRATION RULE**（选择准则）：**KV-read savings > decompression overhead → Inference engine；Otherwise → KV-cache backend**（只有省下的 KV 读取量大于解压开销，才值得放进推理引擎，否则退回 KV-cache backend）。
- 图描述：两列对照版面，左列标「在线路径」、右列标「卸载路径」，每列自上而下为「压缩后形态 → 收益行 → 代价行」，底部一条横向判定规则把两条路径连起来。

### 第 57 页 | TurboQuant: Online KV Quantization

- 标题：**TurboQuant: Online KV Quantization**，副题 **Low-bit vectors for the attention path.**。
- **概念解码路径（Conceptual decode path）**，按流水顺序： 1. **Input K/V vector → Random rotation**（随机旋转）→ 理由：*In high dimensions: smoother, nearly independent coordinates*（高维下坐标更平滑、近似独立） 2. **Codebook lookup → low-bit indices**：用**预计算的 Lloyd-Max codebook**（Precomputed Lloyd-Max codebook），码字中心为 **−1.2 / −0.4 / +0.4 / +1.2**，编码示意 **00 | 10 | 01 | 11**；*Nearest centroid → store its ID*（存最近质心的 ID） 3. **Inverse rotate +**

**rescale**（逆旋转 + 反缩放） 4. **Optional inner-product correction**：*residual → QJL projection → sign bits*（残差经 QJL 投影取符号位）

- **每个向量存储三类信息**：
- **Centroid indices** —— main low-bit payload（主体低比特载荷）
- **Residual sign bits** —— optional QJL correction（可选修正）
- **Scalar norms** —— small per-vector metadata（少量逐向量元数据）
- **不逐向量存储、而是共享**：`rotation / projection matrices + codebook`。
- 效果与出处：**4.5X Compression ratio**；**Llama-3.1-8B • ≥4.5x nominal payload**；**50.06 LongBench-V1 aggregate = FP16**(?)（该行 OCR 作 `50.06 LongBench-V1 aggregate =FP16`，含义应为聚合分与 FP16 持平，`50.06` 数值存疑 **(?)**）。
- 论文链接：`https://arxiv.org/abs/2504.19874`
- 图描述：中部为「向量 → 旋转 → 量化到 4 个码字之一 → 反变换」的流程示意，底部一行色块图例把「主载荷 / 可选修正 / 元数据」对应到三类存储字段。

## 第 58 页 | TurboQuant: Layout Compressed KV for Decode

- 标题：**TurboQuant: Layout Compressed KV for Decode**；要点：**Quantize + pack directly into aligned page fields for MMA decode**（量化后直接打包进对齐的 page 字段，供 MMA decode 使用）。
- 对比两种布局：
- **TOKEN-INTERLEAVED**（按 token 交错）：**D = 128，134B per token**；每个 token 的字段依次为
  - `K codes` **64B** · `K norm` **2B** · `V codes` **64B** · `V scale` **2B** · `V zero` **2B**
  - **134B stride**，且 **V codes begin at byte 66**（V codes 从第 66 字节开始）
- **PAGE-LOCAL FIELD PLANES**（页内字段平面化）：**16 tokens • 2,144B per page / KV head**，字段平面宽度（schematic）：
  - `K codes` **16 × 64B = 1,024B**
  - `V codes` **16 × 64B = 1,024B**
  - `K norms` **16 × 2B = 32B**
  - `V scales` **16 × 2B = 32B**
  - `V zeros` **16 × 2B = 32B**
- 收益指标：**NCU • Useful bytes per 32B load sector：3.68B → 29.75B**（每个 32B 访存扇区里的有效字节数由 3.68B 提升到 29.75B，即访存利用率大幅提高）。
- 实现链接：`https://github.com/vllm-project/vllm/pull/53060`（OCR 作 `vm-project/vilm`、`vlm-pro`，疑为 vLLM **(?)**）。
- 图描述：上行为 token 交错排布（每 134B 一段，K/V 字段交替），下行为同页 16 个 token 重新按字段聚成 5 个连续平面，从而让一次 32B 扇区读取几乎全是有效数据。

## 第 59 页 | TurboQuant: Tiled MMA Decode Attention on 4-bit KV

- 标题：**TurboQuant: Tiled MMA Decode Attention on 4-bit KV**；要点：**Stream compressed tiles, build MMA operands on chip, carry softmax state**（流式加载压缩 tile、片上构造 MMA 操作数、携带 softmax 状态）。
- 循环结构：**REPEAT FOR EACH 64-TOKEN KV TILE**；**one CTA handles one KV split**；**Double-buffered async load**（`tile i: compute` / `tile i+1: load`）。
- 每个 tile 的计算链：
- **GQA Q pack • up to 16 heads → FWHT → FP16 Q tile**
- **4-bit KV page fields（codes + metadata） → K codes → centroid direct MMA fragment**（码字直接映射为 MMA fragment），再做 **Knorm correction**
- **$QK^T$ Tensor Core MMA**
- **V codes + scale/zero → FP16 shared-memory tile → P × V Tensor Core MMA**
- **FP32 RUNNING STATE**（跨 tile 携带的 FP32 运行状态）：**max score**、**exp sum**、**weighted output**；**Online softmax • rescale across tiles**，状态 **carried into the next tile**。
- 归约：**KV split 0 … KV split 1 … KV split N → Stable merge → BF16 output**。
- 图描述：整页是一张流水/数据流图，左侧为 Q 侧准备（FWHT 到 FP16 tile），中部为 K 侧（码字→fragment→$QK^T$）与 V 侧（反量化到 shared memory→P×V）两段 Tensor Core MMA，右侧是 FP32 运行状态的闭环回路与最终 BF16 输出。
- 链接同上：`vllm/pull/53060` **(?)**。

### 第 60 页 | TurboQuant: Decode Performance

- 标题：**TurboQuant: Decode Performance**，副题 **Higher throughput at long context lengths**。
- 对比设置：**vLLM TurboQuant（Ours）** vs **FlashAttention (BF16 KV)**，后者按硬件分档：**Ampere • FA2**、**Hopper • FA3**、**Blackwell • FA4**。
- 图元素：分组柱状图，**Shared y-axis • Decode throughput (tokens/s)**，纵轴刻度 **1,000 / 2,000 / 3,000 / 4,000 / 5,000 / 6,000**；横轴为 **Cached context length**（`128`、`4K`、`16K`、`32K` … `128K` 一类刻度，OCR 排序错乱）。
- 柱上标注的加速比（OCR 保留）：**1.28x、3.27x、1.25x、1.27x、2.52x、4.54x、2.74x、4.84x、3.71x、4.03x、3.99x、4.42x**；峰值 **4.84x**。
- 多个柱位标 **FA OOM**，页脚说明 **FA OOM = capacity-skipped**（FP16/BF16 KV 的 FA 在长上下文下显存不足被跳过，故该点无对应基线）。
- 实验条件：**Qwen3-8B, BS=32, cuda graph, decode only, 128 output tokens**，**excludes prefill, TTFT and first token**。
- 严谨性声明：*independent GPU/software runs, cross-GPU throughput directional*（各 GPU/软件版本为独立运行，跨 GPU 的吞吐对比仅作方向性参考）。
- 图描述：随 cached context length 增大，TurboQuant 相对各代 FA 基线的倍数整体上行（128 附近约 1.2–1.3x，长上下文达 3.7x–4.8x），且基线在长上下文多处因容量不足（OOM）缺席。

### 第 61 页 | KVTC: Compression for Storage and Transfer

- 标题：**KVTC: Compression for Storage and Transfer**，副题 **PCA and dynamic quantization across layers and heads。**
- 压缩流水：**Cross-layer/head concat → PCA transform → Dynamic quantization → OPTIONAL nvCOMP DEFLATE**
- **Compression unit: one token**（压缩单元为一个 token）
- **Rank principal components (PCs)**，按重要性分配精度：**Important PCs → more bits**
- 末级为 **LOSSLESS • VARIABLE BYTES**（无损、变长字节），**Higher ratio • variable bytes**
- **Runtime Attention KV Path**（`Pure extra overhead vs. standard KV already in HBM`）：`Load Compressed KV from CPU → KVTC decompress on SM → Store standard KV in HBM → Stream HBM→SM → Attention`
- **Why storage-oriented?**（为何面向存储而非在线）：
- **Granularity mismatch**：Attention 按 **per layer/head** 取用，KVTC 按 **cross layer/head** 组织
- **Runtime overhead**：**dequant + inverse PCA** 太重，**too heavy for online decode**
- 效果：**8x–64x Compression ratio**；**Max accuracy drop ≤ 0.8 pt with 16x CR**，覆盖 **3 models × 5 benchmarks。**
- 论文链接：`https://arxiv.org/abs/2511.01815`
- 图描述：上部为四段式压缩流水（concat → PCA → 动态量化 → 可选 DEFLATE），中部为「CPU 压缩态 → SM 解压 → HBM 标准态 → 注意力」的运行时路径，下部为「面向存储」的两条理由对照。

## 第 62 页 | KVTC: Integration and Pipelined Decompression

- 标题：**KVTC: Integration and Pipelined Decompression**；副题 **A pluggable codec backend for inference engines and KV cache management frameworks**（可作为推理引擎与 KV cache 管理框架的可插拔 codec 后端）。
- **Pipelined decompression**（流水化解压）对比：
- **Before**：`PCIe H2D | H2D | H2D | H2D` 与 GPU compute `DQ0 → Full GEMM×8 · C0`、`DQ1 → Full GEMM×8 · C1`、`DQ2 → Full GEMM×8 · C2` 串行交替；每段解压后都要做**全量 GEMM×8。**
- **After**（含 warm-up）：解压 `DQ0 / DQ1 / DQ2` 与 **M-sharded TP-GEMM • M/8 per GPU**、**AllGather(M)**（`AGC0 / AGC1 / AGC2`，产物 **[M,D]/GPU**）重叠流水。
- 收益标注：**Time saved**（节省的时间，因前后两排时间轴长度差）。
- **Pluggable integration architecture**（可插拔集成架构）分层：
- **Inference Engines**：`vLLM`、`SGLang`、`TensorRT-LLM`
- **KV Cache (management)**：`LMCache`、`HiCache`、`Dynamo`（另有 OCR 作 `aement` 的截断词，疑为 `Management` **(?)**）
- **Codec API / GPU Connector**
- **Algorithm**：`KVTC STORE/COMPRESS` —— **KV blocks**、**Layout & chunking**、**PCA/LRPCA**、**DP-planned mixed precision**、**Packed chunks**
- 图例：**Current integration** vs **Candidate adapter**（已集成 / 候选适配）。

- 图描述：下半部分为四层堆叠的架构框图（引擎 → KV cache 管理层 → Codec API → 算法内部步骤），上半部分为两条时间轴，下面一条把解压、TP-GEMM、AllGather 三段错位排布以显示并行度提升。

### 第 63 页 | KVTC: Single-Request TTFT

- 标题：**KVTC: Single-Request TTFT**；结论句：**Smaller transfers reduce TTFT by 5.5% to 10.6%** (?)（更小的传输量把 TTFT 降低 5.5%–10.6%；正文里出现的最大降幅为 **−10.8%**，与标题的 10.6% 不一致，以 OCR 为准存疑 **(?)**）。
- **SETUP**：**GLM-5.1** FP8 · **8× Hopper GPU** · **PCIe Gen5** · **TP = 8** · **vLLM + LMCache** · **Compression ratio: 4.7x**（`GLM-5.1` 版本号 OCR 如此，存疑 **(?)**）。
- 图元素：堆叠柱状图，纵轴为 **Prefill（ms）**（OCR 作 `(sW)`，疑为 ms **(?)**），刻度 200 / 400 / 600 / 800 / 1000；横轴按上下文 **32K / 64K / 128K / 192K** 分成四组，每组两根柱：**uncompressed baseline** 与 **KVTC**；堆叠分量为 **Prefill** + **Vanilla PCIe transfer / KVTC transfer + decompression**，**red = wall-clock TTFT**。
- 四组主数值与降幅（OCR 依序对应，配对关系为推断 **(?)**）：
- **843ms → 773ms**，**−8.4%**
- **591ms → 558ms**，**−5.5%**
- **407ms → 377ms**，**−7.4%**
- **204ms → 182ms**，**−10.8%**
- 其余残留数值（分量为传输/解压段等）：**582 / 514**、**390 / 355**、**207 / 180**、`200 197 201 203 261 259`、**113 / 98**、**91 / 84**。
- 页脚定义：**Prefill = TTFT − LMCache load latency**，堆叠分量分别呈现，红色为墙钟 TTFT。
- 图描述：上下文越长，未压缩基线越高（843ms → 204ms 一线对应最长上下文组的最低值），KVTC 柱始终略低于基线，降幅随上下文增长而扩大（5.5% → 10.8%）。

### 第 64 页 | KVTC: Throughput Under Memory Pressure

- 标题：**KVTC: Throughput Under Memory Pressure**；结论句：**Compression prevents the cache-capacity cliff**（压缩避免了「缓存容量悬崖」）。
- **Setup**：**GLM-5.1-FP8, TP=8 on 1 node with 8× Hopper GPU SXM 141GB**（NVLink **900 GiB/s**，PCIe Gen5 **465 GiB/s per GPU**(?)），**vLLM + LMCache**，**128 GiB CPU-DRAM**，**18K-token chunks**(?)（OCR 作 `L18K-tokenchunks`，疑为 `1K/18K` 之类 **(?)**），**NUMA interleave = all**，**prefix caching off**，**KVTC CR 8x**。
- **Problem — Vanilla degrades as memory shrinks**；**Resilience — KVTC holds performance under pressure**。
- 左图 **Capacity sweep • concurrency = 16 • Higher is better**：横轴 **Host memory capacity (GiB) — decreasing**（**400 / 300 / 200 / 96 / 80 / 60** 一类刻度，OCR 排序倒置），纵轴 **Output tok/s**，两条曲线 **KVTC** vs **Vanilla**；纵轴刻度含 **140 / 120 / 80 / 60** 与 **50.0** 等。

- 右图：**Concurrency = 16**，**Output tok/s** 与 **LMCache hit rate** 两条对照；标注 **Eviction pressure**、**Host memory constrained**、**124.7**、**11.9%**、**2.49x**、**−64%**、**+62.8pp**、hit rate 轴刻度 **0 / 20% / 40% / 60% / 80%**。
- **Takeaway 文字**：*Vanilla hits a capacity cliff as memory shrinks: cache eviction lowers hit rate and throughput.*
- **Low overhead — Capacity sufficient (400 GiB)**：**Throughput delta: +3.6% to −2.1%**，**~4.7x smaller host-memory footprint**。
- 表格（**Mean TTFT V/K**、**Output tok/s V/K**、**Δthroughput**，多组 concurrency）：
- **38.3 / 38.1**，**−0.5%**，**0.58 / 0.53 s**
- **72.8 / 71.8**，**−1.4%**，**0.57 / 0.56 s**
- **129.4 / 126.7**，**−2.1%**，**0.71 / 0.67 s**
- **127.1 / 131.0**，**+3.1%**，**13.1 / 12.9 s**
- **125.8 / 130.3**，**+3.6%**，**36.54 / 36.54 s**
- 关键点（96 GiB / concurrency 16）：**Output throughput 2.49x**、**Mean TTFT −64%**（`108.8 → 39.1 s`）、**LMCache hit rate +62.8pp**（**11.9% → 74.7%**）。
- 收尾句：*...KVTC reduces memory footprint by ~4.7x with negligible performance overhead.*（KVTC 把内存占用降低约 4.7x，性能开销可忽略。）
- 图描述：随 host memory 从 400 GiB 压到 96 GiB，Vanilla 的吞吐与 hit rate 双双断崖式下跌（hit rate 跌到 11.9%），KVTC 曲线基本走平并保持约 2.49x 的吞吐优势。

### 第 65 页 | nvCOMP-ANS: Lossless KV Cache Transfer

- 主标题 **nvCOMP-ANS: Lossless KV Cache Transfer**；副标题：**GPU compression reduces data movement during offload and reload**（GPU 压缩减少 offload / reload 过程中的数据搬运）。
- 该页是一条三级存储搬运链的示意图，泳道标签自上而下为：
- **GPU → CPU memory → SSD**
- 搬运路径标注：**D2H/H2D**、**Write/read**
- 搬运内容标注：**Raw KV**（**BF16/FP16/FP8**）、**Compressed KV Blocks**、**Compressed payload**
- 中间环节标注压缩组件：**nvCOMP ANS Compress/decompress**；元信息 **Per-chunk size metadata**；主机侧动作 **Host gather/scatter**；收益 **Fewer bytes per read/write**。
- 底部三栏为三项设计要点：
- **Exact KV recovery** —— **Lossless entropy coding restores the original values**（无损熵编码还原原始值）。
- **Overlapped execution** —— **Batched GPU codec with double-buffered transfers**（批处理 GPU 编解码 + 双缓冲传输）。
- **FlexKV integration** —— 支持 **MHA/GQA/MLA/DSA**，以及 **TP/DP** 并行方式。
- 一句话总结：**Fewer bytes move between tiers. CPU and SSD retain fixed-size cache slots.**（层级之间搬运的字节更少；CPU 与 SSD 侧仍保留定长 cache slot。）

- 图描述：横向为 GPU→CPU→SSD 三级，纵向对比"未压缩原始 KV 字节流"与"压缩后的定长块"两条路径；GPU 侧只做无损熵编解码，CPU/SSD 侧看到的是定长的压缩块加每块 size metadata，因此读取写入的字节数下降，而 cache slot 尺寸不变。链接：`https://developer.nvidia.com/nvcom`。

## 第 66 页 | nvCOMP-ANS: Compression and Serving Results

- 主标题 **nvCOMP-ANS: Compression and Serving Results**；结论句：**TTFT gains are strongest at low concurrency in this benchmark**（在本 benchmark 中低并发时 TTFT 收益最明显）。
- **LOSSLESS COMPRESSION**（无损压缩率，原文字节 / 压缩后字节）：
- **~1.4x**（对应 **BF16 KV cache**）
- **~1.1x**（对应 **FP8 KV cache**）
- **TTFT speedup by concurrency (c)** 柱状图数值（各组从左到右为 **Mean TTFT** 与 **P90 TTFT**）：
- **+18.7%**：柱高 **20.0%**、**16.9%**、**18.7%**
- **+8.2%** mean TTFT speedup：柱高 **5.7%**、**8.2%**
- **P90 TTFT speedup at c = 2**：柱高 **1.8%**、**4.2%**、**3.2%**、**3.0%**
- 负向：**2.1%**、**-10.0%**、**-5.6%**（图上有负值柱，说明部分点变慢）
- 明确标注：**Throughput unchanged**（吞吐不变）。
- 图注/公式：**TTFT speedup = baseline latency / nvCOMP latency - 1**；**Bars show means**；**P50 (?) varies widely at c = 3 and 8**；**-1.9% to +2.4% across c = 1-8**（OCR 中的 `Pg0` 疑为分位数标记，标 **(?)**）。
- **E2E 配置**：**SGLang + FlexKV (CPU only) / GLM-5.1-FP8 / BF16 KV / 128K input / 8x Hopper GPU**。
- 图描述：横轴为并发度（OCR 只保留到 `2` 与 `8` 两个刻度标签），纵轴为百分比变化；柱组按 Mean TTFT 与 P90 TTFT 分开，低并发（c = 2）处的正向柱最高（+18.7% / +8.2%），高并发出现负值，与"低并发收益最强"的结论一致。链接：`github.com/taco-project/FlexKV/pull/18a` (?)。

## 第 67 页 | Takeaways

- 该页为整段的收束页，三条要点：
- **Where compression runs determines what improves** —— 压缩发生在哪一层，决定了什么指标能改善。
- **Fused attention can reduce KV reads during decode.** —— 融合 attention 可以在 decode 阶段减少 KV 读取。
- **Cache backends can extend cache residency and reduce transfers.** —— cache backend 可以延长缓存驻留时间并减少传输。
- **The methods delivered different gains in these benchmarks**（各方法在这些 benchmark 中带来的收益各不相同）：
- **Optimized TurboQuant improved long-context decode**（优化后的 TurboQuant 改善长上下文 decode）；
- **KVTC preserved throughput under memory pressure**（KVTC 在内存压力下保持吞吐）；

- **nvCOMP-ANS reduced TTFT at low concurrency with exact KV recovery**（nvCOMP-ANS 在低并发下降低 TTFT，且保持 KV 精确还原）。
- **Choose by end-to-end results**（按端到端结果选型）：**Measure quality, TTFT, TPOT, throughput, and cache hit rate after codec and integration costs** —— 要把 codec 与集成成本算进去之后，再衡量质量、TTFT、TPOT、吞吐与 cache hit rate。
- 页脚残留 `nVID` / `env` 为 NVIDIA 标识残影，非正文。

## 6.4 第四部分 • KV Router 与 Flash Indexer（第 68–78 页）

这一段为中文投影片，讲 Dynamo 体系里的缓存感知路由：第 68–69 页是能力与生产验证，第 70 页是两类打分信号，第 71–72 页解释索引在链路中的位置，第 73–76 页给出六代索引的吞吐演进，第 77–78 页把它拆成独立服务。

### 第 68 页 | What Dynamo can do today

- 主标题 **What Dynamo can do today**；两列能力清单 + 一组社区数据：
- **All kinds of disaggregation**（各类分离式部署）：
- **Prefill / decode disaggregation**；**Frontend / KV router** 与 **Agg / Disagg FT**
- **Attention / FFN disaggregation (GPU/LPU)**（`LPU` 按 OCR 原样保留，**(?)**）
- **Multimodal E/P/D pipeline**；**Wide-EP FT-NIXL elastic EP**
- **KV-aware router and Flash Indexer**
- **Fault tolerance**（容错）：**Model Express and Snapshot**；**Planner**；**Online active scaling of P/D workers**
- **Global KV management**（全局 KV 管理）：**FlexKV / KVCR (?)**；**Multi-node distributed P2P KV reuse with router hints**
- **Grove – Find Most Ideal Worker**；**Digital twin of large-scale MaaS – high-fidelity projection**；**ai simulate**(?)
- **RL rollout acceleration** 与 **harness-based agentic inference**；**ThunderAgent、nvext agent hints, etc.**
- 底部社区数据：**~8.3K GitHub stars**、**244K+ pip installs**、**9,000+ pull requests**（OCR 只截到 `pull ro` / `pull r…`，按上下文复原，**(?)**）。
- 图描述：该页为"能力矩阵"式版面，左侧按 disaggregation 与容错/规划分类罗列，右侧是 KV 管理与社区代理类能力，底部三个数字为开源项目的采用度指标；OCR 未保留分区框线。

### 第 69 页 | KVRouter 在生产环境中被广泛验证 / FlashIndexer 是支撑准确路由的基石

- 主标题：**KVRouter 在生产环境中被广泛验证**；副标题：**FlashIndexer 是支撑准确路由的基石。**
- 两个客户案例卡片：
- **Baseten**：**2X faster TTFT & 1.6X TPS with KV aware routing**（KV 感知路由带来 2 倍 TTFT 提速与 1.6 倍 TPS）。

- **Meituan**：**2x faster TTFT & +20% more KV cache hit with KV routing**（OCR 作 `+20%`，应为 **+20%**）。
- 两条参考链接：
- `https://www.baseten.co/blog/how-baseten-achieved-2x-faster-inference-with-nvidia-dynamo/`（OCR 中 `b ow-baseten` 为粘连断行）
- `https://www.nvidia.com/en-us/on-demand/session/gtc26-s81943/`
- 该页为两张案例卡并排的版面，无图表数据；正文文字量少但语义完整。

### 第 70 页｜请求进入后，KV Router 用两类信号为 Worker 打分

- 主标题：**请求进入后，KV Router 用两类信号为 Worker 打分**；副标题：**缓存命中减少重复计算；实时负载防止缓存丰富的 Worker 过载。**
- 两条数据通路：
- **请求路径**：**请求 Token → Block Hash → KV Indexer**（输出**缓存前缀命中**）→ **Slot Tracker**（输出**活跃的 Prefill + Decode**）→ **Cost Function**（**综合成本**）→ **最低成本 Worker。**
- **缓存事件路径**：**推理 Worker → KV Events**（**缓存新增/删除**）→ 索引侧更新。
- 成本公式：**Cost = Prefill 成本 + Decode 成本 +（可选）活跃请求成本。**
- "为什么要两个信号？"一栏的结论：**缓存告诉我们"能省多少"，负载告诉我们"还能接多少"。**
- 图描述：左半为请求流（Token → Hash → Indexer → Tracker → Cost → 选中 Worker），右半为 Worker 反向推送 KV Events 的事件流；两条信号在 Cost Function 处汇合，一张是"命中深度"，一张是"当前排队/负载"，共同决定 worker 打分。OCR 中 `Slot T-acker`、`Prefll`、`Wor` 均为断行/错字，已复原。

### 第 71 页｜Flash Indexer 在链路里做什么

- 主标题：**Flash Indexer 在链路里做什么**；定位句：**它不生成 token，也不转发请求；它回答一个路由问题：哪里已经有缓存?**
- 四步链路： 1. **请求输入**：**prompt blocks**，**block_hashes: h0 h1 h2 h3 ...** 2. **实时缓存地图**：**Store/Remove events** 持续更新每个 worker 的 **cached prefix** 3. **查询输出**：**Worker C: 4**、**Worker A: 2**、**Worker B: 0**（即各 worker 上可连续命中的 block 数） 4. **Router 决策**：**优先选择缓存最长、负载合适的 worker**
- **为什么这很关键**（两栏）：
- **查询慢，会拖慢路由**（OCR 断作"查询慢，会按..."，**(?)**）；
- **状态旧，会错过可复用的 KV cache。**
- **Flash Indexer 的目标**：**在高并发读写下，仍然快速返回"谁命中最多"，让 prefix cache 真正参与生产路由。**
- 图描述：中间是"block_hash 列表 → 缓存地图 → 命中计数"的查询示意，输出侧给出三个 worker 的命中深度（4 / 2 / 0），Router 据此排序；图中不含时间或吞吐数值。

### 第 72 页｜索引里存的是一张"位置到 worker"的地图

- 主标题：**索引里存的是一张"位置到 worker"的地图**；副标题：**为了判断 prefix 是否连续，Flash Indexer 不能只看 block 内容，还要看它在第几个位置。**
- **请求被切成 blocks**，示意块序列为 **h5 h6 h7**。
- **每个 block 有两个信息**：
- **position**：第几块
- **block_hash**：内容指纹
- **key → value** 映射（图右侧）：
- **(0, h0) → Worker A, C, D**
- **(1, h1) → Worker A, C**
- **(2, h2) → Worker C**
- **(3, h3) → Worker C**
- **为什么要有 position**：**同一个 block_hash 可能在不同位置出现；只有位置也一致，才说明这段 prefix 真的连续。**
- 图描述：左侧是请求的 block 切分，右侧是 `(position, block_hash) → worker 集合` 的哈希表；示例中只有 **Worker C** 在 position 0–3 全部命中，因此是唯一能连续命中 4 块的 worker，A 只到 position 1，D 只到 position 0。

## 第 73 页 | GEN01/06 THROUGHPUT — 第一代：Naive Nested Dict

- 该页左侧是一条**版本演进阶梯**（Gen 1 → 6），右侧是每一代的定位/瓶颈说明；本页高亮第 1 代。
- **第一代性能**：**Naive Nested Dict — 385K block ops/s。**
- 演进阶梯上的节点（OCR 保留的名单）：**Nested Dict → RustActor（单队列保证顺序） → Inverted Index（按 block 反置到 worker） → Radix Tree（用树表达前缀） → Concurrent Radix（读并行，写保序） → Flash Indexer（按位置跳转 + Jump Search）。**
- **第一版的比喻与问题**：像一本按 worker 分册的账本——要找某个 prefix，只能先翻 W1、再翻 W2、再翻 W3；**每个请求都要逐个 worker 试一遍，看它的缓存能连续命中多深。**worker 越多、prompt 越长，查询越像"在全仓库逐柜翻找"。
- **为什么慢**：外层扫 worker，内层扫 prefix depth；**复杂度 O(W×D)**（页脚标注）。
- **为什么还不够准**：`local_hash` 只代表 block 内容；相同内容出现在不同位置时，可能被误认为 prefix 连续命中。因此引入 **Two hashes**：`local_hash`（内容）+ `sequence_hash`（上下文位置）。
- **这一代的价值**：它暴露了两个核心问题——**访问模式不对**（`access pattern`，OCR 作"访问椅式"，**(?)**），以及 **local_hash 碰撞需要 prefix context。**
- 图描述：右下为三本并列的"worker 缓存账本"（W1 / W2 / W3），每条账本按 `pos0 / pos1 / pos2 / pos3` 逐行对齐，标注 **match depth = 2**、**match depth = 1**、**match depth = 0**，说明必须逐个 worker 逐深度比对才能算出命中长度；`h3*` 标记提醒该位置存在内容相同但前缀位置不同的 block。

## 第 74 页 | GEN04/06 THROUGHPUT — 第四代：Radix Tree

- **第四代性能**：**Radix Tree — 4M block ops/s。**

- 核心思想：**把 prefix 变成树。**相同 block 内容只有在相同父前缀下才算同一路，**碰撞问题被上下文约束住**；**树的父子关系就是 prefix context**——同样的 `local_hash` 在不同父节点下是不同路径。
- 英文要点：**Radix Tree: prefix context scopes collisions。**
- **解决了什么**：碰撞不再只是 hash 层面的问题，而被放进"这段内容出现在什么前缀之后"的上下文里判断。
- **事件怎么处理**：**Store/Remove 事件用 `sequence_hash` 快速定位节点；query 则沿 `local_hash` 子节点往下走** —— 页脚英文总结为 **dual-key design: local_hash for traversal, seq_hash for events。**
- **还没解决什么**：树本身更正确，但**仍然在单线程 actor 之后**，读写并发能力还没释放出来。
- 图描述：一棵 `root → h1 → h2 → h3…` 的 radix 树，左侧标注 **event lookup（seq_hash → node）** 与 **prefix traversal（沿 local_hash 下行）** 两条不同路径；图下方标出 **W1, W2 / W3 / W4** 与 `seq_hash`，表示多个 worker 的同一前缀在树中共享节点。页脚有"新版节点更慢：先讲 3 个背景，再讲 6 代优化。空格播放/暂停，←/→切换"——这是视频播放器 UI 残留，非投影片内容（页 75 同）。

### 第 75 页 | GEN06/06 THROUGHPUT — 第六代：Positional Indexer + Jump Search

- **第六代性能**：**Positional Indexer + Jump Search — 170M block ops/s。**
- 核心思想：**不再沿树一层层走，而是按 position 随机访问，并用 checkpoint 跳过大片共享前缀。**
- **跳过的判据**：**如果第 0、64、128 个 block 都还匹配，就不必检查中间每一个 block** —— 英文标注 **all active workers still match → skip the whole interval**，遇到不匹配则 **lookback** 回扫那一小段找掉队位置。
- **为什么更快**：`index[position][local_hash]` **可以直接查某个深度，不需要从 root 一路走到那个深度**；查询成本 **O(D/J + W)**（D 为深度、J 为 jump size、W 为 worker 数）。
- **碰撞怎么处理**：**大多数 slot 是 Single，直接命中；只有出现 Multi 碰撞时，才 lazy 计算 `sequence_hash` 消歧。**
- 关键标签：**position -> local_hash -> SeqEntry**；数据结构与技巧：**Vec、DashMap、SeqEntry::Single、SeqEntry::Multi、lazy seq_hash、jump search。**
- 图描述：底部是一条按 checkpoint 刻度排布的横向轴，刻度为 **64、128、192、256、320、384、448、512、576、640**；命中段的整体向前跳跃，落单处标注 **lookback**。页脚英文：**checkpoints instead of pointer hops; collision path stays lazy。**
- 页面底部仍有播放器 UI 残留文字（"暂停 / 空格播放/暂停"），非投影片内容。

### 第 76 页 | 性能数据（Achieved vs. Offered Throughput）

- 主标题：**性能数据**；图标题 **ACHIEVED VS. OFFERED THROUGHPUT (HIGHER IS BETTER)**（实测吞吐 vs 提供负载，越高越好）。
- 图中出现的系列与数值点：**Naive（385K）**、**v0.1 Radix(?)**、**Radix Tree（4M）**、**Concurrent Radix Tree**、**Nested Map**；**Flash Indexer / 提升** 标注 **170M**，并给出 **440x / 42x** 两个倍数。
- 图上方另见 **810K**、**5.0M** 等刻度/点位数值（与图例对应关系 OCR 不清，**(?)**）。
- **Benchmark setup**（基准配置）：
- **24-core Arrow Lake 285K desktop**

- **Mooncake FAST'25 trace + mock engine**（OCR 作 `MooncakeFAsT'25`）
- **16,384 GPU blocks, prefix caching enabled**
- **24 concurrent event-processing threads**
- 横轴为 **Offered Throughput (block ops/s)**，纵轴上界标为 **50M / 200M**，x 轴刻度 OCR 部分残损（可见 `100K / 200K / 500K / 1M / 2M / 5M / 20M / 100M / 200M / 500M / 1.0G / 2.0G` 一类量级标签，个别字符不可辨，**(?)**）。
- 结论句：**Flash Indexer 的价值不只在"知道缓存在哪里"，还在"这个查询足够快"，足以支撑大规模请求。**
- 图描述：多条曲线在"提供吞吐"升高后先后出现拐点并掉头向下（达到饱和），曲线越靠右说明能承受的负载越大；OCR 可辨认的定量信息是末端点 **170M** 与相对提升 **440x / 42x**，其余曲线形状的细节数字未识别出来。

## 第 77 页 | Standalone KV Indexer: 定位

- 主标题：**Standalone KV Indexer: 定位**；一句话定义：**把 KV cache 的索引能力从 router / frontend 里拆出来，变成可独立查询的服务。**
- 三栏对照（**它是什么 / 它不是什么 / 适合什么时候用**）：
- **它是什么**
  - 独立维护 **cached KV blocks** 的索引
  - 订阅 worker 的 **KV cache 变化事件**
  - 通过查询接口返回 **prefix overlap**
- **它不是什么**
  - 不是 **standalone router**
  - 不做最终 **worker selection**
  - 不维护完整 **request lifecycle**
- **适合什么时候用**
  - 调试 router 看到的 **KV state**
  - 外部系统做 **custom routing**
  - 观察不同 **worker / tier** 的 **cache 分布**
- 该页为三列文字卡版式，无图表与数值；内容完整可读。

## 第 78 页 | Standalone KV Indexer: 架构

- 主标题：**Standalone KV Indexer: 架构**；总纲：**ZMQ 事件面负责 ingest，HTTP 请求面负责注册、查询和观测。**
- 三类 API（图中右侧清单）：
- **管理类 API**：`GET /health`、`POST /register` 与 `POST /unregister`、`GET /workers`
- **查询类 API**：`POST /query`、`POST /query_by_hash`

- **恢复类 API**：`GET /dump`、`POST /register_peer` / `POST /deregister_peer`、`GET /peers`
- 数据流：**Worker 1 / Worker 2** 各自通过 **ZMQ PUB** 推送 **KV events** → Indexer 侧 **ZMQ SUB Listeners** 接收 → 写入 **Worker Registry** 与索引 `(model, tenant) -> Radix Tree` → 对外由 **HTTP API** 提供查询。
- 查询返回：**查询结果按 map 返回，并可拆分 gpu / cpu / disk tier**（OCR 作"按 mato 返回"，**(?)**）。
- 图描述：左侧是两个 Worker 的 ZMQ PUB 出口，中间是 Standalone Indexer 内部的两层（事件面 ZMQ SUB Listeners、索引面按 `(model, tenant)` 分树的 Radix Tree 与 Worker Registry），右侧是 HTTP API 暴露的管理/查询/恢复端点；图中无性能数值。

## 6.5 第五部分 • 智能体推理与 ThunderAgent（第 79–96 页）

全场收尾，也是最新的一段：第 79–81 页重新定义 SLA 并用真实 trace 量化智能体负载，第 82–84 页给出三层软件栈与可观测性，第 85–90 页按“容量→命中率→并发→路由”四组目标实测，第 91–95 页是会话网关 ThunderAgent 及其基准与 RL rollout 集成，第 96 页列资产与后续工作。

### 第 79 页 | SLAs of New Agentic Era

- 主标题 **SLAs of New Agentic Era**；整页是 **Traditional Inference → Agentic Inference** 的指标对照表： | Traditional Inference | Agentic Inference | |---|---| | **TTFT – Time to First Token** | **Trajectory E2E latency** | | **TPOT – Time per Output Token** | **Time to First Answer Token** | | **Request E2E latency** | **System throughput** | | **满足 SLA 的千卡 QPS** | **AA-AgentPerf: SLO 下最大并发 agent 数** | | — | **Global KV cache hit rate > 90%** | | **以单次请求为中心** | **HiCache / FlexKV、P2P KV reuse** | | **API price** | **SLA 下并发 session 成本** |
- 核心结论：**单次请求指标 → trajectory / session 级系统 SLA。**
- 该页为双列对照表（无柱线图），OCR 完整；表中"—"为 OCR 未给出对应项的格子。

### 第 80 页 | AgentX Trace

- 主标题 **AgentX Trace**；顶部三个关键数字：**393**、**98,827**、**1M。**
- 其中：**393 真实会话（Claude Code 轨迹）**、**98,827 请求数**、**原生上下文窗口 1,048,576**、**trace 请求 p99 近 900K。**
- **主线程请求输入长度分布（按请求数占比）**：
- **< 32K：2%**
- **32K–128K：21%**
- **128K–256K：27%**
- **256K–512K：30%**
- **> 512K：20%**
- **pareto curve**（帕累托曲线）：纵/横为 **每 GPU tok/s** 与 **TTFT p90**，即"吞吐 vs 交互性"的权衡曲线。
- 实验方法标签：**真实会话**、**固定 seed**、**官方 harness 回放**、**每 lane 10 条**、**393 任务**、**25–75% 处切入**、**N lane**、**-1 小时 warmup 预热。**

- 图描述：上方是输入长度分布的堆叠条（`< 32K` 仅占 2%，主体集中在 32K–512K，`> 512K` 仍占 20%），下方/右侧是吞吐与 TTFT p90 的帕累托曲线，用于在固定真实 trace 上比较"要吞吐还是要交互性"；OCR 未给出曲线上的具体坐标数值。

## 第 81 页 | Single AgentX Session Characteristics

- 页面分左右两张图，主题是**单个 AgentX 会话的特征**：左图"单步时延分解"，右图"ISL 增长曲线"。
- 左图（单步时延分解，堆叠柱状图）：
- 纵轴为时延刻度：`100s`、`75s`、`50s`、`25s`；横轴为 `agent step →`，刻度 `20 30 40 50`。
- 图注：**59 steps，主线程请求，无 subagent。**
- 图例/分项文字：`tool / user gap（>60s）`(?)（OCR 作 `t项60s`）、`prefill(TTFT)`、`decode`；柱内散落数值 `70g`、`120g`、`1808`、`1328`、`47h`、`61a`、`66a` 等，OCR 严重失真，**具体数值不可靠**(?)。
- 图描述：每步时延被拆成 tool/用户思考间隔、prefill（TTFT）、decode 三段，其中 `tool / user gap（>60s）` 一段占比明显最大。
- 右图（ISL 增长曲线，累积输入 token，逐 query）：
- 纵轴刻度 `50K / 100K / 150K / 200K`，**峰值 219K**；横轴为 query 序号，刻度 `10 20 30 40 50`。
- 结论行：**ISL 从 448 单调增长到 219K，每步新增 p50 约 2K token。**
- 关键事件标注：**第 42 步：间隔 46.7 小时后恢复**，并注明**代码上下文 KV 早已被逐出，187K 全量重算。**
- 图描述：曲线严格单调上升，在第 42 步处因 46.7 小时的空档出现代价巨大的全量重算，是"长会话 + KV 逐出"问题的直接证据。

## 第 82 页 | 3 Layer Software Stack of Agentic Inference

- 副标题：**Dynamo Agentic Inference-System Architecture** —— 三层软件栈总览图。
- **HARNESS LAYER / Layer 0**（接入层）：
- 五个入口并排：`Claude Code`、`Codex`、`OpenCode`、`OpenCode`、`Your Agent`。
- 对应说明：`Coding Agent (this tool)`、`OpenAI Coding Agent`、`Open Source Coding Agent`、`Open Source Coding Agent`、`Any agent that calls LLM API`。
- 与下层接口：**HTTP API**、`next.agent_hints`、`cache_control`。
- **ORCHESTRATOR LAYER (Dynamo) / Layer 1**（编排层），三个组件：
- **FRONTEND**：`Multi-protocol support`；`v1/chat`、`v1/responses`、`v1/messages`；`Block boundary awareness`(?)（OCR 作 `Blockboundary ereness`）。
- **ROUTER**：`KV-Aware Placement`；`FlashIndexer` **170M ops/s**；`BinaryHeap priority queue`；`ThompsonSampling routing`。
- **KV CACHE MGR**：`4-tier memory hierarchy`；`Selective evict priority/TTL`；`Agent lifecycle awareness`；`<think>` **block auto-ephemeral。**
- **RUNTIME / Layer 2**（运行时层）：
- 两个引擎：`vLLM`（`High throughput`）与 `NVIDIA TRT-LLM`（`Max performance`）。

- 共性机制：`Priority eviction heap`、`KV Store (HiCache/KVBM)`、`TokenRangeRetentionConfig`、`GPU-CPU-Disk-RDMA SharedBlocks`。
- `Priority max` (?)（OCR 作 `Priorityraax`，疑为 `Priority max`）。
- 图描述：自上而下的分层架构图，Layer 0 是各类 coding agent，经 `agent_hints` / `cache_control` 进入 Dynamo 的 Frontend→Router→KV Cache Mgr，最终落到 vLLM / TRT-LLM 两个 runtime；页脚为 `nVIDIA` logo。

## 第 83 页 | AgentX Dashboard

- 该页是 **AgentX 基准测试 Dashboard** 的界面截图（配置面板 + 散点/曲线图 + 长图例）。
- 配置区：
- `BENCHMARK CONFIG` / `CHART CONFIG`；`MODE`、`X METRIC`、`Y METRIC`，带 `log x`、`log y` 开关。
- 指标选项：`Output throughput (tok/s)`、`agent perf` (?)、`accuracy`、`Per-user speed p50 (tok/s/user)`。
- 操作：`COMPARE HISTORY`、`Reset`、`Copy link`；筛选：`CONFIGURATION`、`RUN DATE`、`VALIDITY`，选项 `All` / `Latest per configuration and concurrency` / `Everything`。
- 页签：`Performance`、`Trends`、`Capacity`、`Runs`。
- 坐标轴：横轴 **Per-user speed p50 (tok/s/user)**，刻度 `20 30 40 50 60 70 80` (?)（OCR 作 `BO`）；纵轴 **Output throughput (tok/s)**，刻度 `200 / 400 / 600 / 800`；每条曲线上的并发标号 `c22a80c4` (?) 中的 `c48`、`c64`、`c96` 等为并发档位。
- 图例中的被测配置（含 run id 与副本数），按 OCR 可辨识部分：
- `unknown · mvp · aff (1)`、`unknown · official · aff (2)`
- `SGLang direct · agg · 1xTP8 · radix` … `44c7ebac (2)`
- `c96` / `c64` / `c48` 与 `c22a80c4 (2)`
- **`Dynamo · agg · SGLang · 4xTP8 · HiCache L2 + Mooncake L3 · 1Mctx · bigKV pool`** … `3c53ab66 (2)`
- `3c53ab66 · mvp · noaff (2)`、`3c53ab66 · official · aff (4)`
- **`Dynamo · agg · SGLang · 4xDEP8 · HiCache L2 + Mooncake L3 · 1Mctx · bigKV pool`** … `d22a02Ba · official · aff (7)` (?)（run id OCR 不清）
- **`Dynamo · disagg 2P2D · SGLang · 4xDEP8 · HiCache L2 + Mooncake L3 + rdma-rails · 1Mctx · bigKV pool`** … `213a5444 · official · aff (4)`、`092c10c0 · official · aff (2)`
- 图描述：多条"吞吐 vs 单用户速度"权衡曲线叠在同一张图上，从 `SGLang direct`、`Dynamo agg`（TP8/DEP8）到 `Dynamo disagg 2P2D` 逐步演进；图例长列表说明不同配置（`agg` / `disagg`、`aff` / `noaff`、HiCache L2 + Mooncake L3、`1Mctx`、`bigKV pool`）各跑一次并保留 run 副本。
- 图例文字渲染较小，部分 run id 与后缀数字 OCR 失真，标 `(?)`。

## 第 84 页 | Observability – Dynamo Provider Plugin

- 该页是 **Dynamo Provider Plugin 的可观测性（tracing）界面**截图，主体为多泳道的 trace 时间线。
- 顶部时间轴刻度：`00:02:30`、`00:03:00`、`00:03:10`、`00:04:00`（OCR 重复出现一轮，疑为标尺上下两行）。

- 点击/悬浮信息：`Workspace`(?)、`8.3`(?) 等字段值，OCR 不可辨。
- 泳道（每组一个 batch，括号内注明 `main thread`）：
- `swebench_p18taskpl-batch-1780049… traj-astropy_astropy-693 (main thread)`
- `swebench_p18taskpl-batch-1780049… traj-django_django-109 (main thread)`
- `swebench_p18taskpl-batch-1780049… traj-astropy_astropy-774 (main thread)`
- 另有多条同前缀泳道（`swebench_p18taskpl-batch-1780049…`），OCR 只保留了重复前缀。
- 前缀 `swebench_p18taskpl` 疑为 `swebench_pl8task…` / `swebench_p8task…` 之类批次名，`p18` 具体写法不确定 **(?)**。
- 子 span 标签：大量 `tool:bash` 短块在同一泳道内首尾相接；`Category` 列显示 `tool`。
- 模型与 token 数：`Qwen3.5-27B (2149 tok)`(?)（OCR 作 `Qwan3.5-27B(21491ok)`）；一条 span 显示 `58s 344ms` 与 `299`、`10`(?) 等数值。
- 图描述：横轴为真实时钟的甘特式 trace，每条泳道对应一个 SWE-bench 任务的 trajectory 主线程，泳道内是一串以 `tool:bash` 为主的工具调用块，直观呈现"多 agent/多任务并行、工具调用密集"的形态，用于定位单次请求内部的耗时构成。
- 该页文字被界面元素的截图严重割裂，多数 span 名称与数值残缺，**该页识别内容不足**，以上仅为可辨识标签。

### 第 85 页 | How many tokens fit in 1 GB

- 主题：**1 GB 显存能装多少 KV token**，以 GLM-5.2 的实际配置为例。
- 配置行：`GLM-5.2 · MLA + DSA · FP8 KV · 78 layers · 4 nodes / 32 GPUs`。
- 单 token KV 占用：`55.9 KB /token`，分项为 `MLA 43.9` KB + `DSA 9.8` KB + `2.3` KB（第三项名称 OCR 未捕获；43.9+9.8+2.3≈56.0，与 55.9 吻合，推测为索引器/额外开销 **(?)**）。
- 容量表（列：`1GB` / `Per node · 8 GPUs` / `4 nodes`）：
- **TP8 行**：`2.2K` / `1.26M` / `5.0M` token；标注 `8x replicated`、`~33 sessions`；旁注总量 `577GB`。
- **DEP 行**：`17.9K` / `7.6M` / `30.6M` token；标注 `no replication`、`~200 sessions`（OCR 作 `~2o0`）；旁注总量 `427GB`。
- 图描述：以 55.9 KB/token 为单价，把 1GB → 单机 8 卡 → 4 节点 32 卡逐级放大成 token 容量；对比两条路线——**TP8 全卡 8 份复制**只有 ~33 会话容量，**DEP（无复制）**把容量提到 ~200 会话，token 容量差约 6 倍（1.26M vs 7.6M per node），这是后续所有 DEP 结论的前提。

### 第 86 页 | Goal 1: actual KV hit rate vs theoretical max

- 配置行：`GLM-5.2 · AgentX replay · 4 nodes / 32 GPUs · DEP · concurrency 64/96/128`。
- 三个指标定义：
- `Ideal = reusable ÷ input`
- `Actual = engine hits ÷ input`
- `Gap = recompute (capacity)` —— 差距来自容量不足导致的重算。
- 柱状图（Ideal vs actual hit rate）：

- 纵轴 `KV hit rate (%)`，刻度 `100 / 80 / 70 / 60` (?)（OCR 刻度零散）。
- 横轴三档并发：`c64`、`c96`、`c128`；图例 `Ideal` / `Actual`。
- 数值标签：`100`、`96.3`、`96.2`、`95.8`、`91.9 (-3.9 pt)`。
- 读法：`c128` 档 ideal ≈95.8%、actual ≈91.9%，**差距 −3.9 pt**（即 `96.3→96.2→95.8` 与 `91.9` 的对应关系为推测 **(?)**）。
- 分层保留（retention）容量标注：`DEP · G2/G1 retention = G2 = 37M` (?)（OCR 作 `G22G1`）。
- **G1 GPU** • `53GB/rank`：`30.6M` token（`1,710GB`）；`evict` ↔ `refill`。
- **G2 Host** • `512GB/node`：`37M` token（`2,048GB`）；另有 `18M`（`1,024GB`）一档；`evict` ↔ `refill`。
- 关键归因（c128 档）：`c128: spills past G2 → G3 (1 TiB)`，`holds 41 min of writes → L3 hit 1.2%`，`recompute 8.2% → -3.9 pt`，`TTFT p95 290 s`。
- 图描述：三档并发下 actual 命中率随并发升高而下降（96.3→96.2→91.9），与 ideal 的缺口在 c128 达到 −3.9 pt；原因不是算法而是**容量**：c128 的工作集溢出 G2，G3（1 TiB）只能承接约 41 分钟的写入，L3 命中只有 1.2%，于是 8.2% 的输入需要重算，直接把 TTFT p95 推到 290 s。

## 第 87 页 | KV capacity for diff session cc

- 配置行：`GLM-5.2 · AgentX workload · 4 nodes / 32 GPUs · DEP`。
- 核心公式：`KV to keep = sessions × context × headroom`。
- 参数假设（retention moves to G3 in-flight）：`150K`（每会话上下文）· `session peak ~300K` · `57KB/token` · `headroom 1.5x` · `router sees G1 + G3 only`。
- 容量表（四列：会话并发 `Sessions cc` / `In flight (HBM)` / `KV to keep · 300K × 1.5` / `G2 host · per node`）：
- 表头另有两列并列：`staging only`（in-flight / node）与 `buffer_only · external linker · no host cache mode · today`，以及 `G3 pool holds KV to keep`（`DRAM→L3 segment`）。
- **cc=16**：`2.4M` / `7.2M` / `460GB`（`400GB`）/ ≈ `35GB` (?) / `400GB`（另见 `135GB` 括号值）。
- **cc=32**：`4.8M` / `14.4M` / `490GB`（`810GB`）/ ≈ `70GB` / `810GB`。
- **cc=64**：`6M` (?)（按 4.8M×2 应为 9.6M，OCR 疑缺位）/ `28.8M` / `550GB`（`1,610GB`）/ `135GB` / `1610GB`。
- **cc=128**：`57.6M` / `650GB` / `270GB` / `3,230GB`（`3,230GB`）。
- 图描述：随会话并发翻倍，`KV to keep` 呈线性放大（7.2M → 14.4M → 28.8M → 57.6M token），而本机 G2 host DRAM 容量（`512GB/node`）增长跟不上，差额被迫下沉到 **G3 pool（DRAM→L3 segment）**，其需求量从 400GB 一路涨到 3,230GB；结论是"today"的 `buffer_only / no host cache` 模式在 ≥64 并发就装不下工作集。表内部分被 OCR 挤到同一行的数值与括号值归属不清，标 `(?)`。

## 第 88 页 | Goal 2: decode concurrency must keep up

- 配置行：`GLM-5.2 · same AgentX load · 4 nodes / 32 GPUs`；**X 轴 = set session concurrency (lanes)**；注：**in-flight counts requests, incl. subagents**（in-flight 计的是请求数，含 subagent）。
- 两种部署对照：

- **左：TP8 – KV replicated on all 8 GPUs**：画了 8 个（4 组）`KV ×1` 方块，标注 `x1 x1 x1 x1`、`#2 #3 #4` (?)；容量 `1.26M tokens/node`；结论 `ceiling 33 requests in flight`。
- **右：DEP (dp8 + ep8) • no replication**：16 个 `KV` 方块；容量 `7.6M tokens/node (6x)`；结论 `scales to 100+ sessions`。
- 两张折线图：
- 左图纵轴 `Output throughput (tok/s)`，右图纵轴 `Requests in flight`；横轴均为 `set session concurrency (lanes)`，刻度 `c8`、`c16`、`c32`、`c64`、`c96`、`c128`（OCR 有 `c4B`、`c95`、`C4B` 等形近错字）。
- 数值标签：`900`、`158`、`160`、`615`、`20`、`009` (?)、`53`、`24`、`101`，图例 `DEP (dp8+ep8)` / `TP8`、`DEP · requests in flight` / `TP8 · requests in flight`。
- 结论行：`set sessions (1:1)`、`TP8 ceiling ≈33`。
- 图描述：TP8 因为 8 卡各持一份 KV 复制、单节点只有 1.26M token，请求在飞行数早早触到 **33** 的天花板，吞吐随之见顶；DEP 去掉复制后单节点 7.6M token（**6x**），in-flight 随并发线性上升并支撑 100+ 会话。即"**解码并发必须跟得上**"。

## 第 89 页 | Hard affinity mode DEP 下的负载均衡问题

- 配置行：`AgentX · DEP dp8+ep8 · cc 64 · hard session affinity · 1 hour · 4 node 32 dp ranks`。
- 主图：`Device KV occupancy of the 32 dp ranks, one cell per minute (red = ≥90%, full)` —— 32 个 dp rank × 每分钟一格的 occupancy 热力图。
- 纵轴分组：`node`、`node2`、…、`ep ou` (?)（疑为 `ep0..`）；横轴 `Minutes since the start of the measurement window (c64, 3600 s)`，刻度 `20 30 40 50 60`。
- 右上角统计：`Minutes at ≥90%`，标注 `(of 61)`。
- 色带：`0%` → `30%` → `60%` → `90% full` → `100%`。
- 结论性注脚：`KV aware routing for first round, then session affinity hard mode`。
- 图描述：首轮用 KV-aware 路由把会话摊平，之后切到 hard 亲和（会话钉死在某个 dp rank），热力图随后出现明显**热点**——部分 rank 长期停在红色（≥90%，满），其余 rank 偏冷；`Minutes at ≥90% (of 61)` 用于量化"有多少分钟处于饱和"。这就是 **hard affinity 模式下的负载均衡问题**：亲和换来了 KV 命中，却牺牲了排队公平性与整体利用率。

## 第 90 页 | Goal 3: Agentic Routing Strategy

- 该页对比三种 **Agentic 路由策略**，左侧表格 + 右侧 Dynamo KV Router 流程图。
- 三列策略对比：
- **KV-aware Router**：每轮独立打分，看全局；看的是**缓存前缀块 + in-flight 负载**。
- **Hard Affinity**：首轮定终身，不看负载；看的是 `X-Dynamo-Session-ID`。
- **Soft Affinity**：默认留原地，过载 `reroute` + `KVCR`；把绑定当**建议** + 过载值。
- 右侧流程图（Dynamo KV Router）三步机制，编号 ①②③：
- ① `route this turn: rank1 full → rank6`
- ② `hint: prefix is in rank1 host DRAM`

- ③ `NIXL peer-to-peer pull`
- 图中元素标签：`GPU KV`、`bound session`、`soft → rank1`、`KVCR host DRAM`、`full` / `free`、`deliver → GPU · prefill only the new tail`、`free prefix copy`、`G3 pool · one copy`。
- 图描述：GPU KV 侧 rank1 已满、rank6 空闲，路由把本轮放到 rank6；通过 `X-Dynamo-Session-ID` 的 hint 得知前缀在 rank1 的 host DRAM，于是用 `NIXL` 做点对点拉取，**只对新增尾部做 prefill**，从而在"负载均衡"与"前缀复用"之间取得折中；`G3 pool · one copy` 说明跨 rank 共享时只需一份副本。

## 第 91 页 | Goal 4 – Clever Session Gateway – Thunder Agent

- 主题：**ThunderAgent**——一个"聪明的会话网关"，用 **program 状态机 + 全局控制回路**调度。
- 控制回路：**每 `Δt = 5s` 一个 tick，动作是 `resume → pause`**。
- KV 利用率门限（横轴示意 `0.80 / 0.85 / 0.95`）：
- **`0.95` = pause 阈值**
- **`0.85` = resume 上限（ceiling）**
- **`0.80` = pause target**
- 图中实测点：`97%`、`55%`。
- 会话状态机：`REASONING` ↔ `ACTING`（`ACTING 等 tool`）；`tool call 结果返回` 触发状态推进；状态迁移被标注为 `step 未暂停` 的边界条件（与页 92 的"只在 tool 边界迁移"一致）。
- 执行位置：`Worker1`、`Worker2`；暂停后 `KV 已释放`（OCR 作 `DKV已释放`），会话进入 `全局等待队列（paused）`。
- 调度优先级（OCR 残缺，按语义还原）：
- **Resume**：`REASONING` 优先，其次是"占余量最大"者(?)（OCR 作 `装利余最大`）。
- **Pause**：最小的 `ACTING` 优先；`worker` 侧同样参与选择。
- **`ACTING` 计入 util 时权重 `2^(-t)`**，即 **等 tool 越久越先让位**（权重随时间衰减，等待久的 ACTING 最先被 pause）。
- 图描述：左侧是程序状态机，右侧是 KV util 曲线穿越 0.80/0.85/0.95 三条门限的示意，中间是多个 worker 与一个全局 paused 队列；核心思想是把"对话里等工具的时间"转化为可被撤销的 KV 占用。

## 第 92 页 | ThunderAgent • how it decides

- 决策规则总述：
- `one tick every 5 s`；`util = Σ of active programs ÷ GPU KV pool`。
- 每个 tick 做两类动作：`admit / resume`（进入）与 `pause`（退出）。
- 三个门限：`0.80 target`、`0.85 resume ceiling`、`0.95 pause`。
- 打分与顺序：
- **Pause order:** `S = 1/c + 1{ACTING} · ACTING weight × 2^(-t)` —— 以 `1/c`（上下文越小越该被牺牲）加上 ACTING 指示项，ACTING 项再乘时间衰减。
- 选择逻辑：先 `waiting on a tool`，再 `smallest context → cheapest to recompute`。
- Resume：`util < 0.85` 时触发，`S = 1/c + 1{REASONING}`；优先给 `worker with most free KV`。

- 兜底：`wait > 1800 s → forced resume`。
- 状态机约束：`State machine per program, transitions only at tool boundaries` —— 每个 program 一台状态机，只在工具调用边界迁移。
- 示例（四个 program 的暂停顺序，`c` 为上下文大小）：
- `P2 ACTING c=40K` → **第 1 个**被 pause
- `P3 ACTING c=120K` → **第 2 个**
- `C=30K` → **第 3 个**(?)（该条与 P4 的归属 OCR 串行，标 `(?)`）
- `P4 REASONING`、`P1 REASONING c=180K` → **最后**被 pause（REASONING 受保护）
- `util > 0.95` 触发 pause，`util < 0.85` 触发 resume，暂停者进入 `PAUSED`。
- 图描述：一组 program 在 REASONING / ACTING / PAUSED 之间沿 `tool call` → `tool returns` 的箭头迁移；示例中 ACTING 且上下文小的 program 先出队，上下文最大的 REASONING `P1`（`c=180K`）被留到最后，体现"重算代价最小者先让位"。

## 第 93 页 | Agentic Serving — ThunderAgent Benchmarks

- 测试设置：`SWE-Bench Lite · GLM 5.2 · 8x NVIDIA Hopper GPUs · session concurrency = 32`。
- 四张柱状图，指标（箭头为优劣方向）：
- `TTFT p50 (ms) ↓`
- `TPOT p50 (ms) ↓`
- `gen 吞吐均值 (tok/s) ↑`
- `e2e 均值 (s) ↓`
- 三组对照（图例）：`B·TA`（Baseline ThunderAgent(?)）、`A·LRU`、`SLRU`。
- 图上数值标签（OCR 顺序，未与柱子一一绑定）：`116.3`、`502`、`847`、`837`、`1866`、`112.5`、`1729`、`305`、`55.9`、`241`、`409`、`250`。
- 总结行（原文）：`时延：e2e 均值 -51% · TTFT p50 -87% · TPOT 减半 | 吞吐：均值 ×1.6`。
- 图描述：四联柱状图横排，每组三根柱（`B·TA` / `A·LRU` / `SLRU`）；随横轴从基线到 LRU/SLRU 再到 `B·TA`，三个时延指标一致下降、吞吐指标上升，右下角用一行结论给出 e2e −51%、TTFT p50 −87%、TPOT 减半、吞吐 ×1.6 的整体收益。

## 第 94 页 | ThunderAgent + KV offload

- 主题：**ThunderAgent 与 KV offload（分层缓存）的配合**，右侧为 program table，左侧为 G1/G2/G3 三层示意。
- 程序状态与所在层：
- `A · decoding` / `A running` → `G1 GPU`（标注 `G1 95%`）；`resume → refill`
- `D · idle, kept hot` → 仍留在 **G1**（`idle, kept hot`：空闲但保热）
- `B · waiting tool` → `G2 Host`（`A copy`、`B copy`；`G2 60%`）
- `C · offloaded` / `paused` → `offloaded`（`pause → cold → offloaded`）
- `G3 Pool`：`A B C · one shared copy`

- 门限：`pause when 95%`；`admit while …`(?)（OCR 截断，未取到门限值）。
- 关键结论（原文）：`TA never moves KV · it only changes who is cold · the tiers do the moving` —— ThunderAgent 自己从不搬运 KV，只改变"谁变冷"，真正的搬运由分层缓存完成。
- 图描述：右侧 program table 逐行列出各 program 的 phase 与冷热；左侧三条水平层（`G1 GPU` 95% 占用、`G2 Host` 60% 占用、`G3 Pool` 单副本）标出 A/B/C 三份 KV 的落点，箭头表示 `pause → cold → offloaded` 与 `resume → refill` 的双向流动。

## 第 95 页 | ThunderAgent in RL rollout – Dynamo VeRL Integration

- 测试设置：`Uni-Agent SWE-Bench · Qwen3-Coder-30B-A3B-Instruct · 8x NVIDIA Hopper GPUs · rollout-only`。
- 折线图：纵轴吞吐（刻度 `500 / 1000 / 1500 / 2000 / 2500 / 3000`），横轴 `Agent admission concurrency (cc)`，刻度 `32 / 64 / 80 / 96 / 128 / 256 / 384 / 512`。
- 三条曲线（图例）：`Default LB`、`Dynamo Python TA`、`Dynamo native Session Aware`。
- 增益标注：`+16.5%` **VSTA**(?)、`+6.8%`（`VSTA` 三字 OCR 不清，标 `(?)`）。
- 图上文字：`Default LB collapse.` 与 `high-concurrency regime` —— 高并发区间基线发生塌陷。
- 结论行（原文，OCR 有粘连）：`Dynamo SessionAware (TA based) can be leveraged in agentic RL rollout: throughput driven boost, tict / tpoi h limit`(?)（后半段 `trougnputdrivenbosttictttpoihimit` 严重粘连，疑为 "throughput-driven boost, … limit"）。
- 交付与致谢：`Deliver … with VeRL: github.com/verl-project/verl-recipe/tree/main/dynamo`；参与人/联系人一行：`…ingyi Yang, Jinyan Chen, Aichen Feng, Yilin Hou etc.`（首个姓氏 OCR 残缺，疑为 `Jingyi Yang` **(?)**）。
- 图描述：三条吞吐曲线随 `cc` 增大而上升，`Default LB` 在中间并发段即出现"塌陷"（掉头下降），而 `Dynamo Python TA` 与 `Dynamo native Session Aware` 在高并发区间继续抬升，两条增量分别标 `+6.8%` 与 `+16.5%`；页码框为 `3264x2448`，为投影屏截取而非手机翻拍。

## 第 96 页 | Assets & Future Work

- **该页为收尾页**：分 `Assets`（已有产出）与 `Ongoing Works`（在研方向）两栏，并附 PR 与代码链接。
- **Assets — Docs**:
- **Flash Indexer – Dynamo Digest**: `docs.nvidia.com/dynamo/digest/flash-indexer`
- **ThunderAgent Program Scheduler – User Guide**: `docs.nvidia.com/dynamo/user-guides/agents/thunder-agent-program-scheduler`
- **Assets — PRs • ThunderAgent & SessionAware**:
- `#11447` `feat(router): ThunderAgent admission strategy`
- `#11787` `refactor(router): policy-class admission`
- `#11616` `feat(admission): session-aware admission policy`
- 汇总链接：`github.com/ai-dynamo/dynamo/pull/11447 · /11787 · /11616`
- **Assets — 代码仓库**:

- `agent-plugins`：`…/ai-dynamo/agent-plugins/tree/main/pi-plugin`(?)（OCR 作 `gin-agent-plugins` / `pi-plugin`，路径片段残缺）
- `verl-recipe`：`…/verl-project/verl-recipe/tree/main/dynamo`（另见 `/pull/126`(?) 片段）
- **Ongoing Works**（五个方向）：
- **Simulation for Agentic Infra**：`simulate, tuning engine, scheduling and KV management before deploy` —— 部署前用仿真做引擎、调度与 KV 管理的调优。
- **Enhance program aware routing** —— 增强 program 感知的路由。
- **More advanced admission control**：`ThunderAgent v2`，`deal with moderate cc`（处理中等并发区间）。
- `Soft affinity mode + KVCR`。
- `Programmatic KV block hints` **(SGLang RFC** `#27574` **) etc.**：Dynamo router 下发 `Share / Prefetch / Demote / Pin / Retain` 等 soft hint。
- 该页为**总结/资源页（非致谢页）**，无数据图表。

# 附录 A：逐页索引表（96 张）

下表为全部 96 张照片的索引：序号即整理编号（与影印本、第六章逐页整理一一对应），时间为照片拍摄时刻，标题为识别出的首页标题行，字数为该页识别到的有效字符数。重复拍摄页与文字量偏少的页面在备注中标出。

| 序 | 拍摄 | 该页识别标题 | 字数 | 备注 |
|---|---|---|---|---|
| 1 | 20:26:10 | MegaMoE recap | 449 | |
| 2 | 20:26:10 | MegaMoE support matrix | 267 | |
| 3 | 20:26:10 | MegaMoE on Blackwell | 450 | |
| 4 | 20:26:10 | MegaMoE on Blackwell | 396 | |
| 5 | 20:26:10 | MegaMoE on Blackwell | 411 | 与第 4 页为同一张投影片的重复拍摄 |
| 6 | 20:26:10 | MegaMoE on Blackwell | 203 | |
| 7 | 20:26:10 | MegaMoE on Blackwell | 347 | |
| 8 | 20:26:10 | From a 32x4 Trick to a Full TMEM Transpose | 965 | |
| 9 | 20:26:10 | CuteDSL MegaMoE on Blackwell | 443 | |
| 10 | 20:26:10 | CuteDSLMegaMoEonBlackwell | 472 | |
| 11 | 20:26:10 | CuteDSL MegaMoE on Blackwell | 426 | |
| 12 | 20:26:10 | MegaMoE on Blackwell | 467 | |
| 13 | 20:26:10 | MegaMoEonBlackwell | 547 | |
| 14 | 20:26:10 | MegaMoE on Blackwell | 284 | |
| 15 | 20:26:10 | CuteDSL MegaMoE on Blackwell | 370 | |
| 16 | 20:26:10 | MegaMoEonBlackwell | 512 | |
| 17 | 20:26:10 | Overall Architecture: One Kernel Does Everything | 954 | |
| 18 | 20:26:10 | Warp Specialization | 597 | |
| 19 | 20:26:10 | The swapAB Variant: the Cure for Small Token Counts | 1246 | |
| 20 | 20:26:10 | Fp8 Quantization: per_tensor vs. blockwise | 654 | |
| 21 | 20:26:10 | Optimization 1: Cooperative Warp-group Scheduling | 1171 | |
| 22 | 20:26:10 | Optimization 2: Task-tile Ping-pong | 994 | |
| 23 | 20:26:10 | Optimization 3:Multi-CTA CGA +TMA Multicast | 767 | |
| 24 | 20:26:10 | Optimization 4: Mainloop & Store Pipelines | 1373 | |
| 25 | 20:26:10 | Optimization 5: All-Register Epilogue Fusion Chain | 1158 | |
| 26 | 20:26:10 | Why Gate/Up Interleave = 8: the WGMMA Register Layout | 2317 | |
| 27 | 20:26:10 | Optimization 6: A Single Active Dispatch Warp | 1126 | |

| 序 | 拍摄 | 该页识别标题 | 字数 | 备注 |
|---|---|---|---|---|
| 28 | 20:26:10 | Optimization 7: Folding the Producer Warpgroup + Register | 1006 | |
| 29 | 20:26:10 | Optimization 8: Early fc1_done Publish | 1090 | |
| 30 | 20:26:10 | Optimization9:BlockwiseFC2Dual-Accumulator Overlap | 1302 | |
| 31 | 20:26:10 | Optimization 10: Scheduler Groups and the fc1_done Spin | 701 | |
| 32 | 20:26:10 | Optimization 11: 16-byte FC2 Output Stores | 1375 | |
| 33 | 20:26:10 | Token Sweep Results | 1024 | |
| 34 | 20:26:10 | Summary | 319 | |
| 35 | 20:26:10 | Two Challenges Define Training at Scale | 411 | |
| 36 | 20:26:10 | One Playbook, Two Workloads | 294 | |
| 37 | 20:26:10 | DeepSeek-V4-Pro Overview | 434 | |
| 38 | 20:26:10 | Model Features Drive Optimizations | 616 | |
| 39 | 20:26:10 | Choose the Parallel Layout | 581 | |
| 40 | 20:26:10 | Choose the Parallel Layout | 590 | 与第 39 页为同一张投影片的重复拍摄 |
| 41 | 20:26:10 | Context Length Drives Memory Policy | 338 | |
| 42 | 20:26:10 | Context Length Drives Memory Policy | 354 | 与第 41 页为同一张投影片的重复拍摄 |
| 43 | 20:26:10 | The Starting Points | 408 | |
| 44 | 20:26:10 | STEP 1: Runtime Overhead and Memory Trade-offs | 410 | |
| 45 | 20:26:10 | WheretheStepTime Goes | 212 | |
| 46 | 20:26:10 | STEP 2:Optimize the Hybrid Attention | 1006 | |
| 47 | 20:26:10 | STEP 2: Optimize the Hybrid Attention | 364 | |
| 48 | 20:26:10 | STEP 3:Optimize the MoE | 650 | |
| 49 | 20:26:10 | STEP 3: Optimize the MoE | 524 | |
| 50 | 20:26:10 | STEP 4:Balance CP Workloads | 258 | |
| 51 | 20:26:10 | STEP4I Balance CP Workloads | 610 | |
| 52 | 20:26:10 | Lessons for Scalable Training | 815 | |
| 53 | 20:26:10 | Motivation: KV Cache as a Serving Bottleneck | 163 | |
| 54 | 20:26:10 | KV Cache Is a Serving Bottleneck | 484 | |
| 55 | 20:26:10 | KV Cache Compression Methods | 591 | |
| 56 | 20:26:10 | Where KV Compression Fits | 782 | |
| 57 | 20:26:10 | TurboQuant: Online KV Quantization | 802 | |
| 58 | 20:26:10 | TurboQuant: Layout Compressed KV for Decode | 575 | |
| 59 | 20:26:10 | TurboQuant: Tiled MMA Decode Attention on 4-bit KV | 669 | |

| 序 | 拍摄 | 该页识别标题 | 字数 | 备注 |
|---|---|---|---|---|
| 60 | 20:26:10 | TurboQuant:Decode Performance | 641 | |
| 61 | 20:26:10 | KVTC: Compression for Storage and Transfer | 832 | |
| 62 | 20:26:10 | KVTC: Integration and Pipelined Decompression | 717 | |
| 63 | 20:26:10 | KVTC: Single-Request TTFT | 583 | |
| 64 | 20:26:10 | KVTC: Throughput Under Memory Pressure | 1312 | |
| 65 | 20:26:10 | nvCOMP-ANS: Lossless KV Cache Transfer | 647 | |
| 66 | 20:26:10 | nvCOMP-ANS: Compression and Serving Results TTFTgains are strongestat low concurrency in thisbenchmark | 667 | |
| 67 | 20:26:10 | Takeaways | 529 | |
| 68 | 20:26:10 | What Dynamo can do today | 663 | |
| 69 | 20:26:10 | KVRouter在生产环境中被广泛验证 | 300 | |
| 70 | 20:26:10 | 请求进入后，KVRouter用两类信号为Worker打分 | 270 | |
| 71 | 20:26:10 | Flash Indexer 在链路里做什么 | 341 | |
| 72 | 20:26:10 | 索引里存的是一张“位置到worker"的地图 | 295 | |
| 73 | 20:26:10 | GEN01/06 THROUGHPUT | 750 | |
| 74 | 20:26:10 | GEN04/06 | 690 | |
| 75 | 20:26:10 | GEN06/06 THROUGHPUT | 866 | |
| 76 | 20:26:10 | 性能数据 | 535 | |
| 77 | 20:26:10 | Standalone KV Indexer: 定位 | 300 | |
| 78 | 20:26:10 | Standalone KV Indexer:架构 | 383 | |
| 79 | 20:26:10 | SLAs of New Agentic Era | 364 | |
| 80 | 20:26:10 | AgentX Trace | 279 | |
| 81 | 20:26:10 | Single AgentX Session Characteristics | 351 | |
| 82 | 20:26:10 | 3 Layer Software Stack of Agentic Inference | 874 | |
| 83 | 20:26:10 | AgentX Dashboard | 869 | |
| 84 | 20:26:10 | Observability - Dynamo Provider Plugin | 674 | |
| 85 | 20:26:10 | How many tokens fit in 1 GB | 249 | |
| 86 | 20:26:10 | Goal 1: actual KV hit rate theoretical max | 549 | |
| 87 | 20:26:10 | KV capacity for diff session cc | 736 | |
| 88 | 20:26:10 | Goal 2: decode concurrency must keep up | 716 | |
| 89 | 20:26:10 | Hard affinity mode DEP 下的负载均衡问题 | 388 | |
| 90 | 20:26:10 | Goal3: Agentic Routing Strategy | 482 | |
| 91 | 20:26:10 | Goal 4 - Clever Session Gateway - Thunder Agent | 316 | |

| 序 | 拍摄 | 该页识别标题 | 字数 | 备注 |
|---|---|---|---|---|
| 92 | 20:26:10 | ThunderAgent · how it decides | 586 | |
| 93 | 20:26:10 | Agentic Serving ThunderAgent Benchmarks | 326 | |
| 94 | 20:26:10 | ThunderAgent + KV offload | 336 | |
| 95 | 20:26:10 | Thunderagent in RL rollout - Dynamo VeRL Integration | 550 | |
| 96 | 20:26:10 | Assets & Future Work | 934 | |

# 附录 B：OCR 处理与可靠性说明

## B.1 处理方法

| 环节 | 做法 |
| --- | --- |
| 预处理 | 读取 EXIF 方向 → 转 RGB → 生成最长边 2600 px 的识别副本（保留原图坐标映射，便于把识别框回贴到影印本的隐匿文本层） |
| 识别引擎 | RapidOCR 1.4.4（PP-OCRv4 检测 + 方向分类 + 识别模型，ONNX Runtime CPU 推理），中英混合字典 |
| 并行 | 9 进程 × 每进程 2 个 ONNX 线程；96 页总耗时约 4 分钟 |
| 阅读顺序还原 | 按识别框垂直中心聚类成"行"，行内按左边界排序；解决双栏/多块式投影片的串读问题 |
| 质地检 | 统计每页文本量、平均置信度、低分框（<0.60）数量；对最弱 8 页做 3800 px 二次识别并择优 |
| 人工校核 | 逐条核对英文单词粘连、符号（`·`）、乱码与中英混排；不确定处标 `(?)` |

## B.2 质量统计

| 指标 | 数值 |
| --- | --- |
| 页数 | 96（无缺页） |
| 识别字符总数 | 60,024 |
| 平均置信度 | 0.947（仅第 31 页低于 0.80） |
| 重复/重拍页 | 3 组：4–5、39–40、41–42 |
| 文字量最少 | 见附录 A"备注"列 |

## B.3 二次（高分辨率）识别补充

以下页面的第一次识别质量偏低，已用最长边 3800 px 重跑，互为补充：

- **第 31 页（Optimization 10: Scheduler Groups and the `fc1_done` Spin）** 二次识别恢复了三条关键信息：① 调度器**按组**派发，"先派一组专家的全部 FC1 tile，再派它们的 FC2 tile"，默认 group = 一个波次；② 小 bucket 下每个专家只有 1 个 token tile 却有 48 个 FC1 tile，一个组只装得下 1–3 个专家，于是 FC2 tile 紧跟在自家 FC1 tile 后面、在其尚未算完时到达；③ **把 group 扩大到 264（≈6 个专家）后，FC2 tile 到达时 `fc1_done` 已完成，自旋消失。**原第一次识别只保留了"activation TMA producer 约 25% 时间在自旋"这一条。
- **第 84 页（Observability – Dynamo Provider Plugin）** 第一次识别被界面元素严重分割；二次识别确认该页是一张 **trace 可视化截图**：时间轴 `00:02:20 → 00:04:00`、lane 名形如

`swebench_pl_8task...`、`traj-astropy_astropy-693/774`、`traj-django_django-109`、`tool:bash`，并可见模型标签 `Qwen3.5-27B` 与"NVIDIA"标识。该页**不含技术论点**，属于工具/可观测性演示。

- **第 26 页（Why Gate/Up Interleave = 8: the WGMMA Register Layout）** 二次识别得到 171 个文本框（首次仅 15 个），确认该页是一张 `T32–T35` 线程与 `(d0,d1)` / `(d4,d5)` 数据对的寄存器布局表；具体逐格对应仍**不保证正确**，正文只引用其结论（interleave = 8 由 WGMMA 寄存器布局决定）。
- **第 18 页（Warp Specialization）** 的 `Warps / setmaxnreg / Duty` 三列表格存在串行，逐行数值不确定；只有页面明写的 `20 warps / 640 threads` 可靠。

## B.4 引用时的注意事项

1. 正文中的**加粗数字**均取自幻灯片文字层或图注，未做换算；若与你的复现结果不符，请先核对原图（影印本第 5–100 页为逐页原图）。
2. 图表类页面（如第 33、45、60、63、64、76、83、86、88、93 页）的**柱高/曲线高度**没有可靠数值，本文只引用幻灯片上用文字写出的结论倍数，不根据像素估计。
3. 中文页（第 68–96 页为主）存在少量识别别字（如"缓存"→"媛存"），已按上下文修正，不影响结论。

# 附录 C：术语与缩写对照

| 缩写/术语 | 全称或含义 | 在材料中的语境 |
| --- | --- | --- |
| MoE | Mixture-of-Experts | 细粒度专家层（DeepSeekMoE） |
| MegaMoE | — | 本场提出的融合 dispatch/FC1/FC2/combine 的单持久内核 |
| EP / TP / PP / CP / DP | Expert / Tensor / Pipeline / Context / Data Parallel | 并行维度；本例初始布局 PP4 + EP64，CP16（64K）/ CP64（1M） |
| dispatch / combine | all-to-all 发送与收回 | MoE 的通信两段 |
| FC1 / FC2 | MoE 中的两层专家 FFN | FC1 含 SwiGLU 与 FP8 cast |
| swapAB | 交换 A/B 操作数（把 token 映射到 N 维） | 小 token 场景的 tile 利用率手段 |
| tile / tileN | 计算分块 / N 方向分块 | 256×256、`M64×N128`、`M128/256×N8` |
| WGMMA | Warpgroup Matrix-Multiply-Accumulate | SM90 起的 warp-group 级 MMA |
| TMA | Tensor Memory Accelerator | 异步张量搬运；S2G / G2S |
| TMEM | Tensor Memory | Blackwell 上的张量存储；TMEM-DP 为其数据通路 |
| LDTM / STTM | Load/Store TMEM | 出入 TMEM 的指令 |
| R2S | Register to Shared | epilogue 中寄存器到 SMEM 的搬运 |
| STG.32 / STG.128 | 32-bit / 128-bit 全局写指令 | 第 32 页把 8×STG.32 合成 1×STG.128 |
| setmaxnreg | 设置每 warp 寄存器上限 | warp specialization 的预算分配 |
| MEMBAR / UREDGR | 内存屏障 / 异步归约指令 | 同步优化（batch-commit、async UREDGR） |
| SMEM / HBM / SM | 共享内存 / 高带宽显存 / 流式多处理器 | 228 KB SMEM 预算、53 GB/rank 等 |
| NVFP4 / MXFP8 / FP8 / BF16 / FP32 | 数值格式 | 计算与 combine 的精度组合 |
| per_tensor / blockwise | 量化缩放粒度 | per-expert 标量 vs 128×128 块 |
| DeepGEMM / CUTLASS / CuTe / CuteDSL | 内核库/DSL | baseline 与实现载体 |
| NVSHMEM / NCCL / GIN | 单边通信库 / 集合通信库 / GPU-initiated 网络 | dispatch/combine 与 all-gather |
| NIXL | NVIDIA 推理传输库 | 点对点 KV 拉取 |

| 缩写/术语 | 全称或含义 | 在材料中的语境 |
| --- | --- | --- |
| CGA | Cluster Group Array（SM90 簇） | 2×2 CGA + TMA multicast |
| TTFT / TPOT / ISL / OSL | 首 token 时间 / 每输出 token 时间 / 输入长度 / 输出长度 | 服务指标 |
| E2E / SLA / QPS | 端到端 / 服务等级协议 / 每秒查询 | 智能体场景的轨迹级指标 |
| KV Cache | 键值缓存 | 推理显存的主要占用者 |
| TurboQuant / KVTC / nvCOMP-ANS | 三条 KV 压缩路线 | 在线量化 / PCA 压缩 / 无损熵编码 |
| KV Router / Flash Indexer | 缓存感知路由 / 前缀索引服务 | Dynamo 体系组件 |
| Dynamo | NVIDIA 推理服务框架 | 含 KV Router、provider plugin、VeRL 集成 |
| DEP | Disaggregated Expert Parallel（材料中与 KV offload/亲和路由一起出现） | 第 85–89 页的容量对比对象 |
| MLA / DSA | Multi-head Latent Attention / DeepSeek Sparse Attention | 每 token KV 体积拆分（43.9 / 9.8） |
| CSA / HCA | 混合注意力的两条路径 | DeepSeek-V4 结构图 |
| MTP | Multi-Token Prediction | 结构图中的 MTP Modules（MTP Loss） |
| AgentX | 智能体会话 trace 基准/仪表盘 | 第 79–83 页 |
| ThunderAgent | 会话网关（program 状态机 + 全局控制回路） | 第 91–96 页 |
| AA-AgentPerf | 满足 SLA 的最大并发 agent 数 | 第 79 页 SLA 表 |
| VeRL | 强化学习训练框架 | 第 95 页 rollout 集成 |
| SWE-Bench Lite | 软件工程智能体基准 | 第 93、95 页实测 |
| nsys / Nsight | NVIDIA 性能分析器 | block size 由 silicon time 决定 |

# 附录 D：产出文件与复现方式

| 文件 | 说明 |
| --- | --- |
| `大模型训练与推理_图片影印本.pdf` | 96 页原图影印（1 页封面 + 3 页索引 + 96 页图片），每页附**隐形 OCR 文本层**，可全文检索；书签按页标题生成，页脚标注原始文件名与拍摄时刻 |
| `大模型训练与推理_内容整理与总结分析.pdf` | 本文件：摘要 → 资料与方法 → 全篇脉络 → 技术主线剖析 → 总结 → 分析与评论 → 逐页要点整理 → 附录 |
| `大模型训练与推理_内容整理与总结分析.md` | 本文的可编辑源稿（Markdown），便于再加工或导出其它格式 |
| `大模型训练与推理_讲稿整理（录音对照）.pdf` | 配套文件：现场录音转写与投影片的时间轴对照、逐页口头要点、四场问答，以及本文的修订清单 |
| `_build/ocr/text/*.txt` | 每页的 OCR 纯文本（阅读顺序） |
| `_build/ocr/page_*.json` | 每页的识别框、文本与置信度（原图坐标），用于生成影印本文本层 |
| `_build/notes/seg0*.md` | 六段逐页结构化摘要的中间稿 |
| `_build/page_stats.json` | 逐页统计（字数、置信度、重复检测）与标题索引 |
| `_build/*.py` | 处理脚本：`ocr_batch.py`（批量识别）、`ocr_hires.py`（弱页二次识别）、`make_scan_pdf.py`（影印本生成）、`build_doc.py`（本文生成）、`finish_pdf.py`（页眉页脚与书签）、`html2pdf.py`（HTML→PDF） |

**复现要点**：先运行 `python _build/ocr_batch.py`（约 4 分钟，9 进程），再 `python _build/page_stats.py`，然后 `python _build/make_scan_pdf.py` 生成影印本，最后 `python _build/build_doc.py` 生成本文。`.py` 依赖：`rapidocr-onnxruntime`、`onnxruntime`、`numpy`、`Pillow`、`PyMuPDF`、`markdown`，以及本机 Chrome（headless 打印 PDF）。